Datos, costes y registro de la búsqueda

AlgoProject — manual de uso. Actualizado el 2026-10-14 con `tools/manual.py`.

1. 13. La librería de barras — un solo dato guardado, todos los timeframes gratis
2. 57. Calidad del feed M1 — cuándo el feed es de fiar, y cuánto beneficio depende de sus fallos
3. 59. Spread real — el spread de Darwinex medido tick a tick
4. 25. Actualizar los datos — el botón "Update all", automatizado y con red
5. 24. Costes por activo — qué le cobras a cada mercado, y en qué unidad
6. 43. El libro mayor de la búsqueda — cuántas cosas se han probado de verdad
7. 12. Catálogo de rendimiento — qué cuesta cada parte del proyecto, y si hoy cuesta más que ayer
8. 64. El archivo de estrategias — guardar una estrategia con todos sus resultados y recuperarla sin recalcular nada
9. 109. El almacén de estrategias — guardar lo que pasa la puerta y comprobar que está entero
10. 73. Catálogo de fondeo — qué vende cada empresa, a qué precio y con qué reglas
11. 74. Ofertas de fondeo — el aviso diario y «¿merece la pena?»

13. La librería de barras — un solo dato guardado, todos los timeframes gratis

Qué pregunta responde

¿De dónde saca Python los precios? De aquí. La librería guarda **las barras de un minuto** de cada mercado que usas, y nada más. Cuando un análisis pide M30, H1, H4 o diario, se calculan en el momento a partir del minuto y se quedan guardados para la siguiente vez.

Guardar solo el minuto no es un atajo: está comprobado que **el M30 calculado en Python es idéntico, decimal a decimal, al M30 que exporta SQX**. Medido el 21-09-2026 sobre los 7.708.823 minutos de oro: mismas 274.832 barras, mismo índice, diferencia máxima 0,000000 en apertura, máximo, mínimo y cierre. Así que tener los dos guardados sería tener el mismo dato dos veces.

Cuándo lo usas, y cuándo no

Lo usas cuando empiezas a trabajar con un mercado nuevo, y cuando actualizas la data de un mercado dentro de SQX. En los dos casos es el mismo comando y él solo decide qué hay que bajar.

No lo usas para nada más. Los análisis no lo llaman: ellos piden barras y la librería responde. Tampoco sirve para datos de tick — SQX tiene feeds de tick y el comando los ignora a propósito, porque un tick no es una barra y aquí no se remuestrea nada desde ahí.

Antes de empezar

- **El worker tiene que poder arrancar.** El comando lo levanta él solo y lo deja parado al acabar. No toca el master, así que **puedes lanzarlo con la GUI de SQX abierta**.
- **El mercado tiene que estar declarado** en `assets/_markets.yaml`, o ya estar en la librería. Si añades un activo nuevo ahí, el siguiente `sync_bars` lo baja sin que hagas nada más.
- No hace falta que exportes nada antes. Las barras son un hecho del mercado, no de un backtest.

Cómo se ejecuta

Primero mira qué falta, que no cuesta nada:

```
python3 -m sqx.export.sync_bars --check
```

```
$ python3 -m sqx.export.sync_bars --check
  ok   BRENTCMDUSD_ftmo                library 4,105,534  SQX 4,105,534
  ok   XAGUSD_DukasM1_Infinox          library 7,034,806  SQX 7,034,806
  ok   XAUUSD_DukasM1_Infinox          library 7,708,823  SQX 7,708,823
0 feed(s) to pull
```

Y cuando quieras que lo baje de verdad, el mismo comando sin `--check`:

```
python3 -m sqx.export.sync_bars
```

flag	obligatorio	qué hace
`--check`	no	dice qué bajaría y no baja nada. Sin él, baja

No hay flags de fechas a propósito: **cada mercado se baja sobre su propio rango completo**, el que SQX declara para él. Si le pusieras una ventana fija, el mercado que tuviera datos más allá de esa fecha se marcaría como desactualizado para siempre.

Cuánto tarda: unos 35 segundos por mercado. Los tres de oro juntos, menos de dos minutos.

Qué produce

ruta	qué es
`~/Desktop/AlgoData/bars/<feed>/M1.parquet`	las barras de un minuto. 128 MB el oro, 107 el plata, 65 el Brent
`~/Desktop/AlgoData/bars/manifest.json`	qué tiene la librería: barras, rango y la huella de cada mercado
`~/Desktop/AlgoData/barsDerived/<feed>/<TF>-<huella>.parquet`	los timeframes ya calculados

La huella en el nombre del archivo es lo que hace que no tengas que acordarte de nada: si actualizas la data de un mercado en SQX y vuelves a sincronizar, la huella cambia y **los timeframes viejos dejan de usarse solos**. No se sirven barras antiguas por error nunca.

Cómo se lee el resultado

En `--check`, una línea por mercado:

- **`ok`** — la librería tiene exactamente las barras que tiene SQX. No hay nada que hacer.
- **`PULL`** — falta el mercado, o SQX tiene más barras que tú. Se va a bajar entero.
- **`??`** — el mercado está declarado en `assets/_markets.yaml` pero SQX no lo tiene como feed de minuto. Es un nombre mal escrito, casi siempre.

La comparación es **el número de barras**, no la fecha. Eso es lo que hace que detecte también un relleno de huecos o una corrección en medio del histórico, y no solo que haya crecido por el final.

Y lo que cuesta pedir un timeframe, medido de verdad sobre el oro:

```
$ python3 -c 'from core import barstore; barstore.read("XAUUSD_DukasM1_Infinox", TF)'
  timeframe      barras    1a vez   cacheado
  M30           274,832     1.32s     0.031s
  H1            137,785     1.15s     0.025s
  H4             35,299     1.01s     0.012s
  D1              5,912     0.99s     0.005s
```

La primera vez de cada timeframe cuesta **un segundo largo**: ahí está leyendo los 7,7 millones de minutos. A partir de ahí, centésimas. Un informe que analiza 757 estrategias no nota ese segundo; el panel interactivo tampoco, porque solo lo paga la primera vez que lo abres tras actualizar data.

Un ejemplo completo

Actualizaste el Brent en SQX y quieres que Python lo vea:

```
python3 -m sqx.export.sync_bars
```

```
$ python3 -m sqx.export.sync_bars
  PULL BRENTCMDUSD_ftmo                library 4,018,086  SQX 4,105,534
  ok   XAGUSD_DukasM1_Infinox          library 7,034,806  SQX 7,034,806
  ok   XAUUSD_DukasM1_Infinox          library 7,708,823  SQX 7,708,823
  BRENTCMDUSD_ftmo                4,105,534 bars  2013-01-01 → 2026-09-07  65 MB
1 feed(s) refreshed; library holds 3
```

Lee lo que pasó: el oro y la plata estaban al día y **no se tocaron**. El Brent tenía 4.018.086 barras en la librería contra 4.105.534 en SQX, así que se bajó entero — hasta el 7 de septiembre, que es hasta donde llega su data. Los timeframes del Brent que hubiera calculados quedaron huérfanos en ese momento y se recalculan la próxima vez que alguien los pida.

Qué NO te dice

- **No te dice si la data de SQX es buena.** Copia lo que SQX tenga. Si un feed tiene huecos, la librería tendrá los mismos huecos.
- **No te avisa de que tu data está vieja.** Solo compara tu librería contra SQX. Si en SQX llevas ocho meses sin actualizar un mercado, los dos dirán `ok` tan contentos. Eso se mira en SQX. *(A 21-09-2026 el oro y la plata llegan hasta el 16-01-2026 y el Brent hasta el 07-09-2026: los metales llevan ocho meses sin actualizar en SQX, y esto no te lo iba a decir el comando.)*
- **`export_bars.py` ya no existe** (retirado el 23-09-2026). Un timeframe suelto se pide a `core.barstore.bars(feed, TF)`, que lo calcula desde el minuto en un instante y lo cachea.
- **El volumen no es exacto al de SQX.** Calculado desde el minuto, difiere en 13 barras de 274.832 y como mucho en 2 unidades — redondeo de SQX al agregar. Los precios sí son exactos. Si algún día haces algo que dependa del volumen al detalle, tenlo presente.

Si algo falla

- **`KeyError: '<feed>'` al pedir barras** — ese mercado no está en la librería. Corre `sync_bars --check` y mira si sale como `??`: entonces el nombre en `assets/_markets.yaml` no coincide con el de SQX.
- **El comando se queda colgado al arrancar** — el worker no levantó. `bin/sqx-worker.sh stop` y vuelve a lanzarlo.
- **Un mercado sale `PULL` una y otra vez aunque acabe de bajarlo** — eso sería un fallo de verdad y hay que mirarlo, no repetirlo. Pasó una vez, con una ventana de fechas fija que recortaba el feed; por eso ya no hay flags de fechas.

57. Calidad del feed M1 — cuándo el feed es de fiar, y cuánto beneficio depende de sus fallos

Qué pregunta responde

Dos preguntas, una en cada punto del WORKFLOW.

Paso 4, antes de construir: ¿está sano el feed M1 de este activo, y desde qué año? El módulo busca en cada feed de la librería de barras cuatro tipos de anomalía: **picos** (en el cierre y en la mecha) que se deshacen en tres minutos, que es la firma de un tick malo; **congelados** (10 velas seguidas idénticas); **huecos** (5 minutos o más sin vela en sesión), y **caídas del proveedor** (todos los feeds de Dukascopy callan a la vez). Los cuenta por año y por mes, dice desde qué año el feed es estable y qué meses fueron un episodio del proveedor. El preflight (`python3 -m core.assets <SÍMBOLO>`) lo lee en voz alta.

Paso 8, después de la puerta: de una estrategia que ha pasado, ¿qué parte de su beneficio sale de operaciones que tocaron una de esas anomalías, y es más de lo que daría el azar? No recalcula ninguna métrica sin esas operaciones (decisión 13): eso sería limpiar el backtest con otro nombre. Dice el **% del beneficio en riesgo** y si salta la **alarma**.

Cuándo lo usas, y cuándo no

El escaneo (paso 4) se corre cuando la librería de barras cambia: un feed nuevo, o una actualización de datos. Tarda un minuto para los 13 feeds.

La atribución (paso 8) se corre sobre la misma cosecha que acaba de juzgar la puerta (`studies.screening.gate.report`), igual que el edge por coste (capítulo 50).

No lo uses para «limpiar» una estrategia. Casi todo lo que el detector marca es **real**: el flash crash del yen de 2019, el Brexit, el SNB, las intervenciones del BoJ. Una estrategia que ganó el día del Brexit no es sospechosa por eso. La alarma dice «estas operaciones ganan más de lo que ganaría un grupo cualquiera del mismo tamaño», no «esto es un error del feed».

No esperes que la alarma salte a menudo. Con los K congelados, una estrategia del oro M30 con ~1.000 operaciones toca de media 7 anomalías; una de USDJPY H1, 1 o 2. Con menos de 10 marcadas el informe dice «insuficiente» y no juzga. Sobre las 115 estrategias del ejemplo del oro: 102 insuficientes, 13 sin alarma, ninguna alarma. **Hoy esta criba casi nunca cambia una decisión**; su valor es avisar el día que una estrategia sí dependa de minutos anómalos.

Antes de empezar

- La librería de barras al día (capítulo 13): `~/Desktop/AlgoData/bars/<feed>/M1.parquet`.
- Los umbrales en `ledger/thresholds.yaml`, sección `feedQuality.`: son tus respuestas del 2026-09-26. El K y la sesión de cada feed ya están congelados ahí.
- Para el paso 8: una cosecha hecha (`/oos-gate`), y los `.sqx` de sus estrategias todavía en alguna instalación: de ahí se lee si la estrategia entra con orden stop o límite y si lleva stop. Una estrategia sin `.sqx` sale «sin .sqx» y no se juzga.
- **No toca SQX en absoluto.** Se puede correr con todo abierto.

Cómo se ejecuta

```
python3 -m studies.data.feedQuality.scan
python3 -m core.assets XAUUSD
python3 -m studies.data.feedQuality.report --project XAU_ISOOS_ejemplo --databank Results --feed
XAUUSD_DukasM1_Infinox
python3 -m studies.data.feedQuality.report --project XAU_ISOOS_ejemplo --databank Results --feed
XAUUSD_DukasM1_Infinox --strategy "Strategy 23.22.82"
```

comando	flag	obligatorio	qué hace
`scan`	`--feed`	no	un feed; sin él, los 13
`scan`	`--workers`	no	procesos en paralelo (6 por defecto, ~2,6 GB en total)
`report`	`--project`, `--databank`	sí	la cosecha (el databank de build); siempre la más reciente
`report`	`--feed`	sí	el feed con el que se construyó, p. ej. `XAUUSD_DukasM1_Infinox`
`report`	`--strategy`	no	una sola estrategia leída en detalle
los dos	`--set`	no	prueba un umbral sin tocar el ledger: `--set alarm.p_max=0.05`

Hay otros dos comandos que no se corren en el día a día:

- `python3 -m studies.data.feedQuality.calibrate` **mide** el K y la sesión de cada feed con tu criterio y **propone** las filas del ledger; no las escribe. Se corre una vez cuando entra un feed nuevo, o si cambia el proveedor. Tarda 45 s.
- `python3 -m studies.data.feedQuality.inject` **planta** anomalías de tamaño conocido en una copia de cada feed (tu diseño de 2.16) y mide qué encuentra el detector. Tarda unos 5 min.

Tiempos medidos: `scan` 53 s para los 13 feeds; `report` 7 s para 115 estrategias.

Qué produce

`~/Desktop/AlgoData/feedQuality/`:

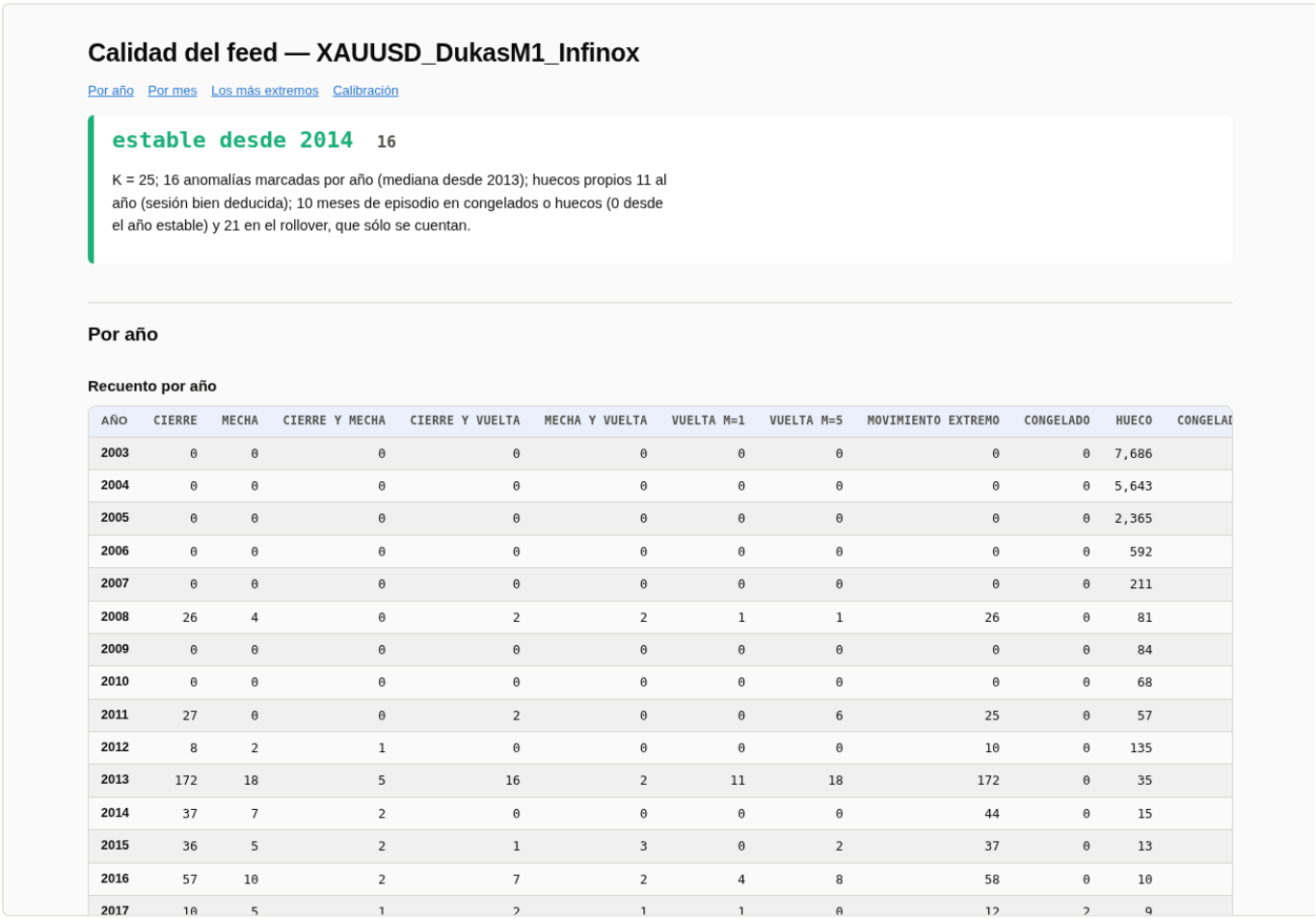
fichero	qué es
`<feed>/events.parquet`	una fila por anomalía: tipo, clase, minuto, tamaño. Lo que lee el paso 8
`<feed>/summary.json`	los números que lee el preflight
`<feed>/feedQuality.json` / `.html` / `.md`	el informe del feed: por año, por mes, los 20 más extremos por columna, cómo se eligió K
`calendar.parquet`	los silencios de toda la librería: festivos, cierres parciales, caídas
`calibration.json`, `injection.json`	la calibración y la inyección
`review.csv`	la revisión a mano de los 20 picos más extremos de cierre y de mecha de cada feed

Además reescribe la sección **Feed quality** de `~/Desktop/AlgoData/INDEX.md` (una fila por feed) cada vez que se corre `scan`.

La atribución escribe en `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/feedQuality/`: `feedQuality.json/.html/.md` (el panel), `verdict.csv` (`strategy`, `identity`, `verdict`, `alarm`, `p`, `at_risk`, `flagged`, `column`) y `estrategias/<nombre>.json/.html` con `--strategy`.

Cómo se lee el resultado

El informe de un feed



Calidad del feed — XAUUSD_DukasM1_Infinox

Por año Por mes Los más extremos Calibración

estable desde 2014 16

K = 25; 16 anomalías marcadas por año (mediana desde 2013); huecos propios 11 al año (sesión bien deducida); 10 meses de episodio en congelados o huecos (0 desde el año estable) y 21 en el rollover, que sólo se cuentan.

Por año

Recuento por año

AÑO	CIERRE	MECHA	CIERRE Y MECHA	CIERRE Y VUELTA	MECHA Y VUELTA	VUELTA M=1	VUELTA M=5	MOVIMIENTO EXTREMO	CONGELADO	HUECO	CONGELAD
2003	0	0	0	0	0	0	0	0	0	7,686	
2004	0	0	0	0	0	0	0	0	0	5,643	
2005	0	0	0	0	0	0	0	0	0	2,365	
2006	0	0	0	0	0	0	0	0	0	592	
2007	0	0	0	0	0	0	0	0	0	211	
2008	26	4	0	2	2	1	1	26	0	81	
2009	0	0	0	0	0	0	0	0	0	84	
2010	0	0	0	0	0	0	0	0	0	68	
2011	27	0	0	2	0	0	6	25	0	57	
2012	8	2	1	0	0	0	0	10	0	135	
2013	172	18	5	16	2	11	18	172	0	35	
2014	37	7	2	0	0	0	0	44	0	15	
2015	36	5	2	1	3	0	2	37	0	13	
2016	57	10	2	7	2	4	8	58	0	10	
2017	10	5	1	2	1	1	0	12	2	9	

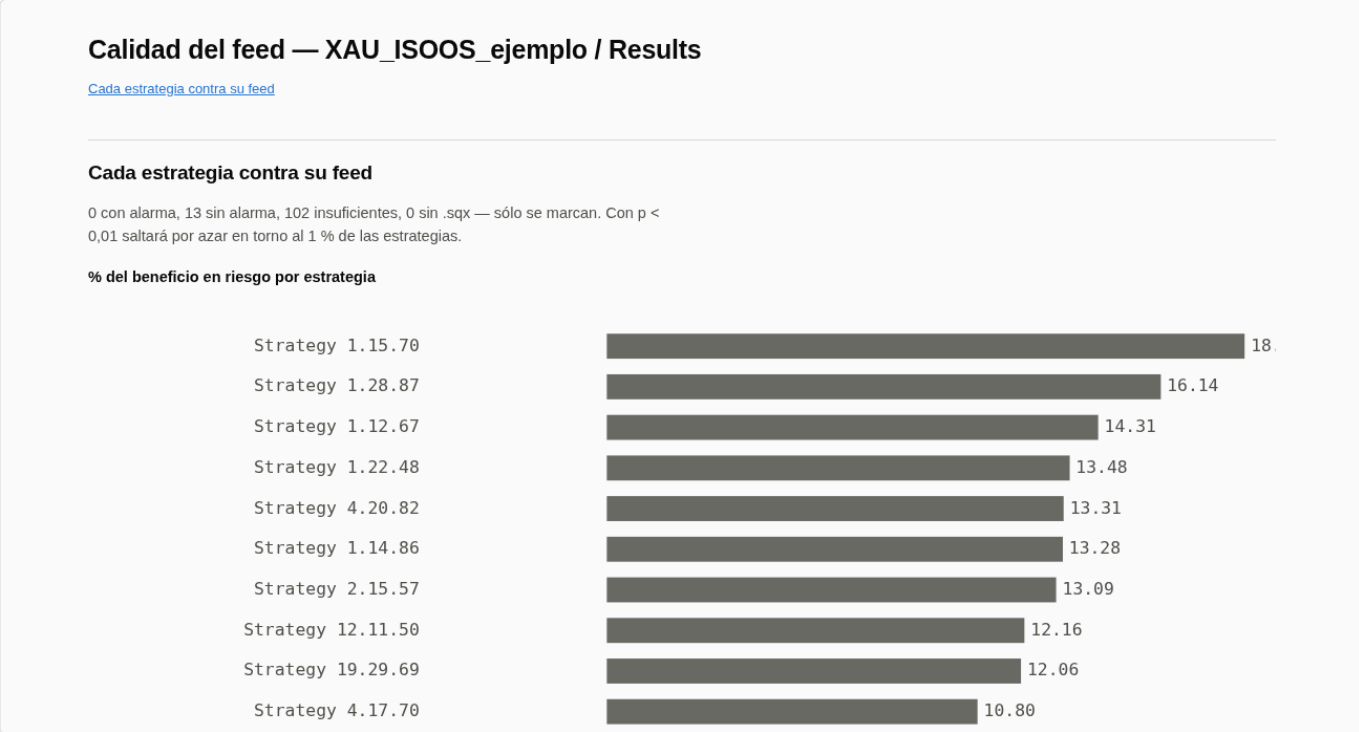
- **«estable desde 2014»**: el primer año desde el que ningún año tiene más huecos que max(3 × la mediana desde 2013, 30). Antes, «calidad B» (hasta 10 × la mediana) o «inestable».
- **marcadas por año**: lo que la atribución del paso 8 lee — picos-y-vuelta de cierre y de mecha, congelados, huecos propios y caídas del proveedor. 16 al año en el oro.
- **huecos propios**: si pasan de 30 al año, la sesión está mal deducida (tu respuesta 2.10).
- **episodios**: un mes con más de 5 veces la mediana de los 36 anteriores. Los del rollover sólo se cuentan: no marcan operaciones.
- Verde: sesión bien y ningún episodio desde el año estable. Ámbar: algo de eso falla.

El aviso del preflight

```
calidad del feed XAUUSD_DukasM1_Infinox: 16 anomalías marcadas por año (mediana desde 2013, K = 25), estable desde 2014 (escaneado 2026-09-26).
AVISO: la construcción empieza en 2008, antes del año estable 2014: esos años son calidad B y inestable.
episodios de hueco: 2009-12, 2010-01, 2010-11, 2010-12, 2011-04, 2012-01, 2012-04, 2012-11, 2012-12, 2013-06.
episodios de hueco en rollover: 2011-11, 2012-12, 2013-03, 2014-03, 2014-04, 2015-03, 2016-03, 2016-11, 2017-03, 2018-03, 2019-03, 2020-03 y 9 más.
```

Avisa, no bloquea (tu respuesta 3.1): mover el inicio de la construcción es de quien fija las ventanas, no de este módulo. Los episodios del rollover de cada marzo son las semanas en que EE. UU. ya ha cambiado la hora y Europa todavía no: el rollover se desplaza una hora.

El panel de la atribución



- **alarma** (rojo): p < 0,01 con al menos 10 marcadas. Con cientos de estrategias saltará por azar en ~1 % de ellas: por eso **marca y no descarta** (`action: mark`).
- **sin alarma** (verde): las marcadas no ganan más que un grupo al azar.
- **insuficiente** (gris): menos de 10 marcadas; no se juzga.
- La barra es el **% del beneficio en riesgo**: el neto de las marcadas entre el neto total.

Un ejemplo completo

El escaneo de la librería:

```
$ python3 -m studies.data.feedQuality.scan
BRENTCMDUSD_ftmo           K=20  estable 2018  161 marcadas/año  huecos 109/año  4 episodios
XAGUSD_DukasM1_Infinox     K=20  estable 2020  60 marcadas/año  huecos 33/año  35 episodios
XAUUSD_DukasM1_Infinox     K=25  estable 2014  16 marcadas/año  huecos 11/año  10 episodios
USDJPY_DukasM1_the5ers     K=30  estable 2005  14 marcadas/año  huecos 2/año  4 episodios
EURUSD_DukasM1_the5ers     K=25  estable 2005  9 marcadas/año  huecos 2/año  4 episodios
...
-> /home/sergioguslw/Desktop/AlgoData/feedQuality  ·  281 meses de episodio en 13 feeds; INDEX.md al día
```

(«episodios» en esta línea cuenta también los del rollover.) El Brent (109 huecos al año) y la plata (33) están por encima de 30: su sesión, deducida del feed, no recoge todas sus pausas.

La atribución de la cosecha de ejemplo, y una estrategia en detalle:

```
$ python3 -m studies.data.feedQuality.report --project XAU_ISOOS_ejemplo --databank Results --feed
XAUUSD_DukasM1_Infinox
115 estrategias: {'insuficiente': 102, 'sin alarma': 13} ->
/home/sergioguslw/Desktop/AlgoData/reports/XAU_ISOOS_ejemplo/Results/2026-09-26/feedQuality

$ python3 -m studies.data.feedQuality.report ... --strategy "Strategy 23.22.82"
**sin alarma** (0.06689) — 10 de 947 operaciones tocan una anomalía del feed y llevan 7.1 % del
beneficio neto. p = 0.0669 contra grupos al azar del mismo tamaño. De las marcadas, 30 % tocan
un pico-y-vuelta (posible error de precio) y el resto un congelado o un hueco. Columna que manda:
cierre.
```

Esta estrategia es la que más se acerca a la alarma de las 115: sus 10 operaciones marcadas ganan 7.164 $, un 7 % del total, y un grupo al azar de 10 gana eso o más en el 7 % de los casos. No salta.

Cómo se tomó cada decisión

Todas son tuyas: las dieciséis respuestas del 2026-09-26 y cinco preguntas más del mismo día. Lo que el módulo hace con cada una:

- **La escala**: el movimiento típico de esa hora de la semana en las 52 semanas anteriores (MAD × 1,4826), nunca por debajo de 3 ticks del feed ni de 0,25 veces la mediana de las horas de esa semana. El **tick es el del feed**, no el de SQX: en los pares, SQX llama tick al pip y el feed cotiza a una décima de pip. Con el tick de SQX el suelo tapaba casi todo.
- **K por feed**, el menor de {15, 20, 25, 30} con menos de 52 picos de cierre al año en 2013–2019: sale **20** en Brent, plata, CADJPY y EURJPY; **25** en oro, AUDJPY, AUDUSD, EURUSD, GBPJPY, USDCAD y USDCHF; **30** en USDJPY y GBPUSD. Más alto que los provisionales, como anticipaba tu respuesta 1.2: la escala de 52 semanas es menor en los años tranquilos.
- **La sesión se deduce del feed**: los minutos con vela en al menos la mitad de las semanas de 2015–2019. Pares: de lunes 00:00 a sábado 00:00. Oro y plata: pausa diaria 00:00–01:00. Brent: 00:00–03:00.
- **El rollover (23:00–01:59)** no cuenta ni para congelados ni para huecos, y sus congelados y huecos se cuentan aparte, sin marcar: así sale el episodio de EURUSD de 2021–2023, que estaba entero a las 00:xx.
- **La alarma** lee IS y OOS juntas.

Qué NO te dice

- **Si una anomalía es un error o un suceso real.** Por su forma no se distinguen: el flash crash del oro de 2021 volvió como un tick malo. De los 520 picos más extremos revisados a mano (los 20 mayores de cierre y de mecha de cada feed), 383 son sucesos identificados, 95 sin identificar (muchos en la reapertura del oro o en el rollover) y 42 sospechosos, 38 de ellos el episodio de CADJPY de 2006–2008 (`review.csv`).
- **Lo que ganaría la estrategia con datos buenos.** No lo calcula, a propósito.
- **Nada sobre los años anteriores a que empiece la escala**: las primeras 13 semanas de cada feed quedan «sin escala».
- **Los festivos propios de los metales** (Navidad del oro y la plata) salen como hueco del símbolo: la regla sólo llama festivo a un día en que callan todos los feeds.

Si algo falla

- `feedQuality: <feed> se escaneó con K=… y otra sesión; vuelve a correr …scan`: el ledger cambió después del último escaneo. Corre `scan` y repite.
- `FileNotFoundError …/feedQuality/<feed>/summary.json`: ese feed nunca se ha escaneado.

- Muchas estrategias «sin .sqx»: sus ficheros ya no están en ninguna instalación (el proyecto se retiró). No se juzgan; la cosecha sigue sirviendo para la puerta.

59. Spread real — el spread de Darwinex medido tick a tick

Qué pregunta responde

¿Cuánto spread cobra de verdad el mercado, y se mantiene constante respecto al precio? Lo mide en los ticks de Darwinex (que llevan ask y bid) y te dice qué coste declarar en SQX en cada tramo (build, oos1, oos2), ya con tu factor de seguridad de 1,25.

Antes este capítulo tenía una segunda mitad, el reajuste de cada estrategia al spread real (`report`). Se borró el 2026-10-07: el paso 8 reprecia ahora con `engines/market/reprice.py`, dentro de la puerta.

Cuándo lo usas, y cuándo no

- **Paso 4 (preflight)**, antes de decidir los costes de un activo: `scan`.
- Hoy solo están configurados **XAUUSD y USDJPY**.

Antes de empezar

- SQX no tiene que estar cerrado: se lee el fichero de ticks del maestro en solo lectura (`SQX/user/data/History/<feed>/<feed>_TICK.dat`) sin tocar ninguna instancia.
- Las velas M1 de Dukascopy del activo tienen que estar en la librería (`bars/`).

Cómo se ejecuta

```
python3 -m studies.data.spread.scan                      # los dos activos
python3 -m studies.data.spread.scan --symbol XAUUSD      # uno
```

flag	obligatorio	qué hace
`--symbol`	no	un activo de `config.yaml`; sin él, todos

Cuánto tarda. La primera vez, `scan` decodifica los ticks: 13 s para USDJPY (368 millones de ticks) y 25 s para el oro (647 millones). Después los guarda por minuto y el `scan` entero tarda 5 s. No toca SQX.

Qué produce

- `AlgoData/spread/<feed de ticks>/`:
- `minutes.parquet`: una fila por minuto, con el spread del primer tick, el ponderado por tiempo, el mínimo, el máximo y el bid. Se rehace sola si SQX actualiza los ticks.
- `daily.parquet`: el spread relativo de cada día, medido o modelado.
- `hours.parquet`, `summary.json` y el informe `spread.html/.md/.json`.

No escribe `assets/`. Propone; los costes los cambias tú desde la ventana.

Cómo se lee el resultado

Spread real — XAUUSD

Propuesta para SQX · Por año · Por hora · Modelo · Cola

relativo NO constante → modelo volatilidad `0.1877`

Peor año del spread relativo a 1.96 × la mediana de los años (tolerancia ±20%); en puntos 1.91 ×. Propuesta con factor 1.25: comisión build 0.0131 %, oos1 0.0138 %, oos2 0.0118 %. Hoy assets/ declara spread_is 5.0 puntos, spread_oos 10.0 puntos, comisión 0.001 (pct_of_notional).

Propuesta para SQX

Propone, no escribe: assets/ es del dueño (core.assetwrite).

Por tramo

TRAMO	DESDE	HASTA	% DÍAS MEDIDOS	SPREAD MEDIO (PB)	COMISIÓN % PROPUESTA	PRECIO MEDIANO	PUNTOS EQUIVALENTES
build	2008-01-01	2017-12-29	2.45	1.05	0.01313	1,251	16.44
oos1	2018-01-02	2022-12-30	100	1.10	0.0138	1,705	23.52
oos2	2023-01-03	2026-08-28	99.79	0.9431	0.01179	2,641	31.14

Los días sin Darwinex (antes de octubre de 2017) llevan el spread del modelo; «% días medidos» dice cuánto del tramo es medición.

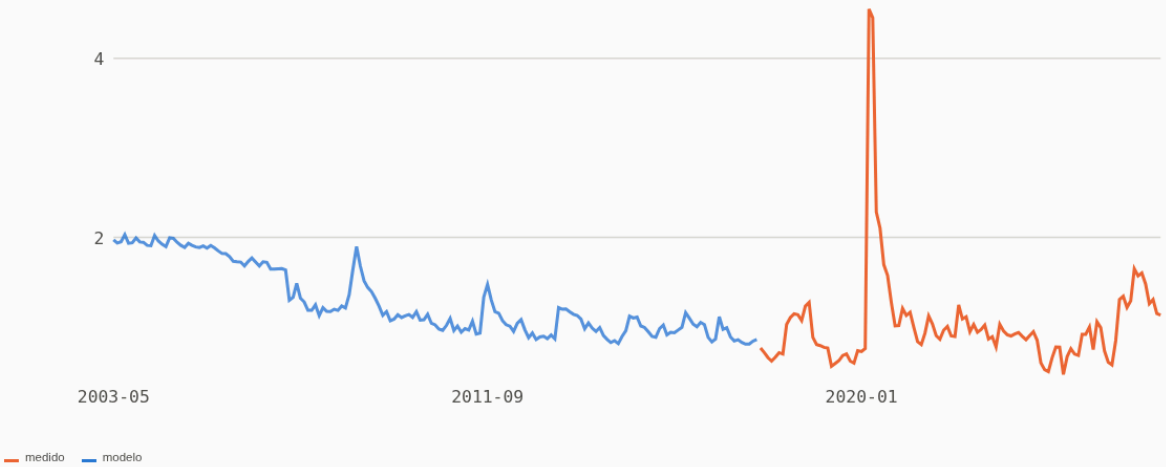
Por año

Spread por año

AÑO	PUNTOS MEDIANA	PUNTOS MEDIA	PB MEDIANA	PB MEDIA	PRECIO MEDIANO	MINUTOS
2017	9	9.12	0.7233	0.7142	1,278	87,652
2018	12	12.19	0.9356	0.9685	1,264	351,511
2019	10	9.53	0.6901	0.6852	1,406	353,332
2020	23	32.82	1.23	1.89	1,774	354,600
2021	18	17.59	0.9872	0.978	1,794	353,544
2022	17	17.95	0.9576	0.994	1,805	353,575
2023	17	17.62	0.887	0.9077	1,945	352,540
2024	15	16.12	0.638	0.668	2,381	355,070
2025	29	33.59	0.8766	0.9546	3,343	350,277
2026	56	63.37	1.23	1.39	4,527	259,228

Primer y último año, parciales. Puntos = spread ÷ el tick de assets/.

Spread relativo medio por mes

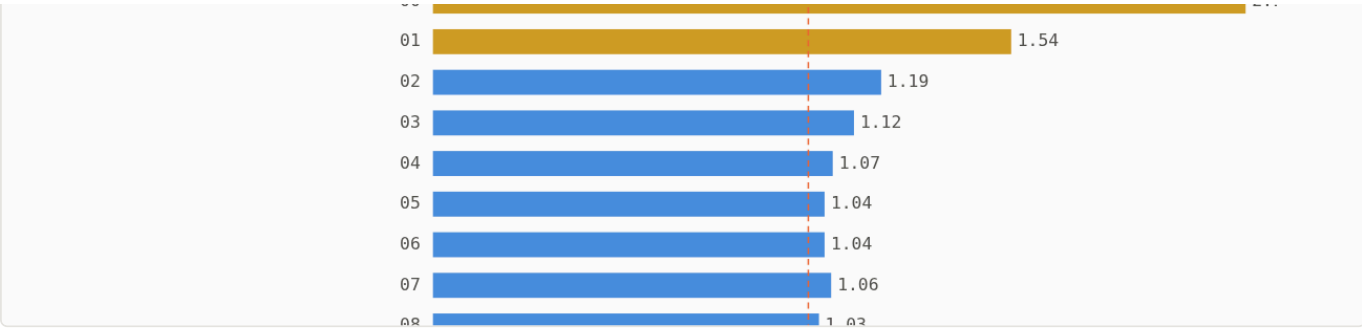


Por hora

Hora del reloj del feed. El reajuste de cada operación multiplica el spread del día modelado por el de su hora.

Spread de cada hora ÷ spread medio del día





- **El veredicto** dice si el spread relativo al precio es constante (cada año completo dentro de ±20 % de la mediana de los años). Si no lo es, dice qué modelo lo reconstruye mejor para los años sin Darwinex.
- **«Por tramo»** es la propuesta:
- **comisión %**: spread medio × 1,25, en % del nocional. SQX la cobra **una vez** por operación, sobre el precio de apertura, y una operación paga un spread entero entre ida y vuelta.
- **puntos equivalentes**: el mismo coste como spread fijo al precio mediano del tramo.
- **% días medidos**: cuánto del tramo es medición y cuánto modelo.
- **La curva mensual**: en naranja lo medido, en azul lo modelado hacia atrás.
- **«Por hora»**: cuánto más caro es cada hora que la media del día. A las 00 el oro cuesta 2,1 veces más: es el rollover.
- **«Modelo»**: el error de cada modelo en años que no vio. «Hacia atrás» es el que importa.

Spread real — USDJPY

Propuesta para SQX · Por año · Por hora · Modelo · Cola

relativo NO constante → modelo volatilidad_precio `0.09607`

Peor año del spread relativo a 1.44 × la mediana de los años (tolerancia ±20%); en puntos 1.77 ×. Propuesta con factor 1.25: comisión build 0.0048 %, oos1 0.0052 %, oos2 0.0078 %. Hoy assets/ declara spread 0.1 puntos, comisión 0.0 (usd_per_lot).

Propuesta para SQX

Propone, no escribe: assets/ es del dueño (core.assetwrite).

Por tramo

TRAMO	DESDE	HASTA	% DÍAS MEDIDOS	SPREAD MEDIO (PB)	COMISIÓN % PROPUESTA	PRECIO MEDIANO	PUNTOS EQUIVALENTES
build	2008-01-01	2017-12-29	2.43	0.385	0.004813	99.73	0.48
oos1	2018-01-02	2022-12-30	98.48	0.4148	0.005185	109.86	0.5697
oos2	2023-01-02	2026-08-28	98.24	0.6245	0.007807	149.60	1.17

Los días sin Darwinex (antes de octubre de 2017) llevan el spread del modelo; «% días medidos» dice cuánto del tramo es medición.

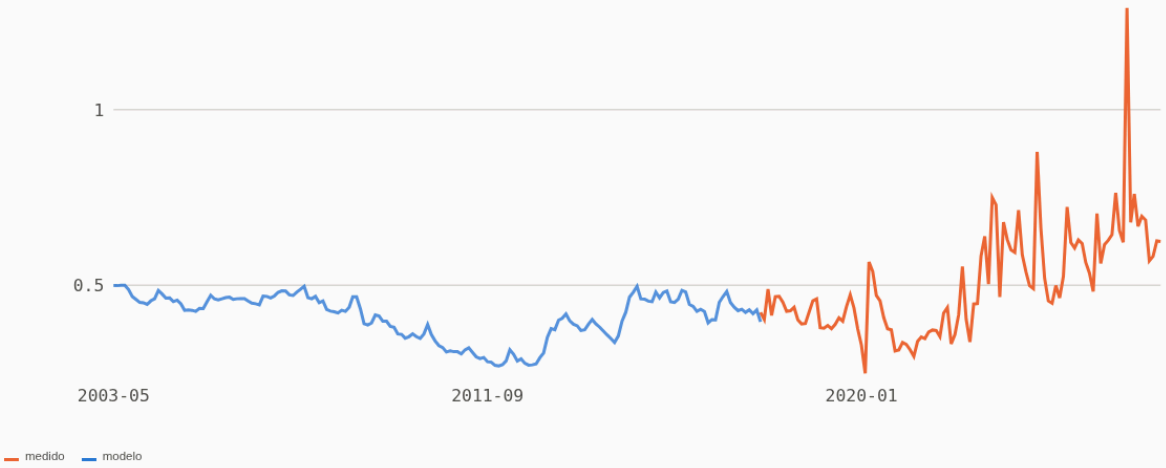
Por año

Spread por año

AÑO	PUNTOS MEDIANA	PUNTOS MEDIA	PB MEDIANA	PB MEDIA	PRECIO MEDIANO	MINUTOS
2017	0.4	0.4747	0.3563	0.4205	112.86	92,362
2018	0.4	0.4746	0.3609	0.4302	110.80	370,741
2019	0.4	0.4456	0.3665	0.409	108.79	369,732
2020	0.3	0.4246	0.2864	0.3975	106.83	370,910
2021	0.3	0.3962	0.2753	0.3598	109.82	370,147
2022	0.5	0.6384	0.3608	0.4798	133.80	369,773
2023	0.6	0.8475	0.4558	0.6052	140.87	368,795
2024	0.7	0.8553	0.4543	0.5665	151.42	369,865
2025	0.7	0.9835	0.4744	0.6567	148.52	367,732
2026	0.8	1.04	0.5048	0.6569	158.92	271,149

Primer y último año, parciales. Puntos = spread ÷ el tick de assets/.

Spread relativo medio por mes

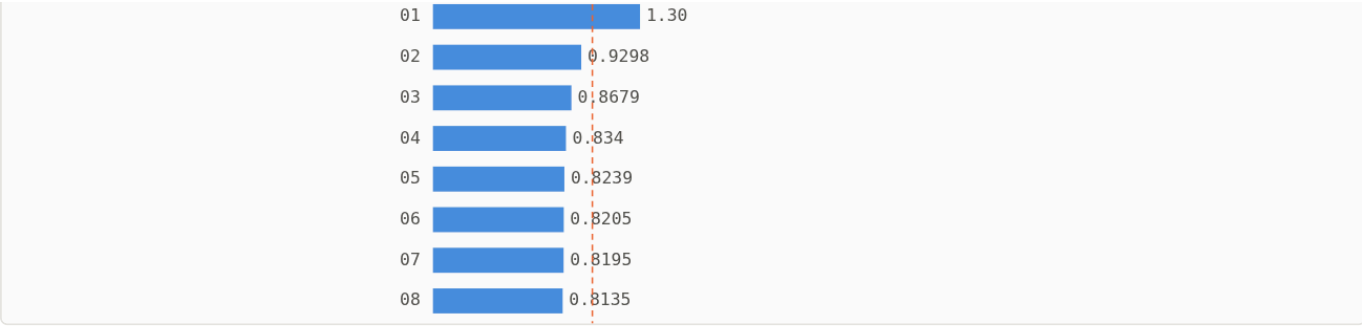


Por hora

Hora del reloj del feed. El reajuste de cada operación multiplica el spread del día modelado por el de su hora.

Spread de cada hora ÷ spread medio del día





En el informe de cada activo, **«Spread en puntos medio por mes»** es la curva del spread: medida donde hay ticks y modelada hacia atrás.

Un ejemplo completo

```
$ python3 -m studies.data.spread.scan
XAUUSD: relativo NO constante (peor año 1.96×) → modelo volatilidad, error hacia atrás 19%. Hoy:
spread_is 5.0 puntos, spread_oos 10.0 puntos, comisión 0.001 (pct_of_notional)
       % días medidos  spread medio (pb)  comisión % propuesta  puntos equivalentes
build          2.4484             1.0507                0.0131              16.4371
oos1         100.0000             1.1038                0.0138              23.5210
oos2          99.7881             0.9431                0.0118              31.1381

USDJPY: relativo NO constante (peor año 1.44×) → modelo volatilidad_precio, error hacia atrás 10%. Hoy:
spread 0.1 puntos, comisión 0.0 (usd_per_lot)
       % días medidos  spread medio (pb)  comisión % propuesta  puntos equivalentes
build          2.4317             0.3850                0.0048               0.4800
oos1          98.4825             0.4148                0.0052               0.5697
oos2          98.2402             0.6245                0.0078               1.1679
```

Cómo se lee:

- **Oro:** hoy declaras 5 y 10 puntos. Darwinex, con el margen de 1,25, pide unos 16 en el build, 24 en oos1 y 31 en oos2. En comisión serían 0,013 %, frente al 0,001 % actual.
- **USDJPY:** declara 0,1 puntos, y el real con margen es 0,5–1,2.

La banda del spread: el rango del MC Retest

```
python3 -m studies.data.spread.bands                    # todos los de band.assets, cada uno por
separado
python3 -m studies.data.spread.bands --symbol DAX40
```

Para cada activo, cada día de ticks de Darwinex es un punto: su precio y su spread medio en puntos. Se ajusta la curva más sencilla, `spread = a · precio^b` (una recta en log-log), a la media y a los cuantiles 2,5 % y 97,5 %. Esos dos cuantiles son el **Min y el Max** que se escriben en la tarea de MC Retest de spread. SQX sortea **un** spread por simulación y lo aplica a todo el backtest, así que lo que importa es el spread medio de un día, no el de un minuto suelto. Tarda 8 s los siete activos y deja `AlgoData/spread/<feed de ticks>/band.html`.

Banda del spread — DAX40

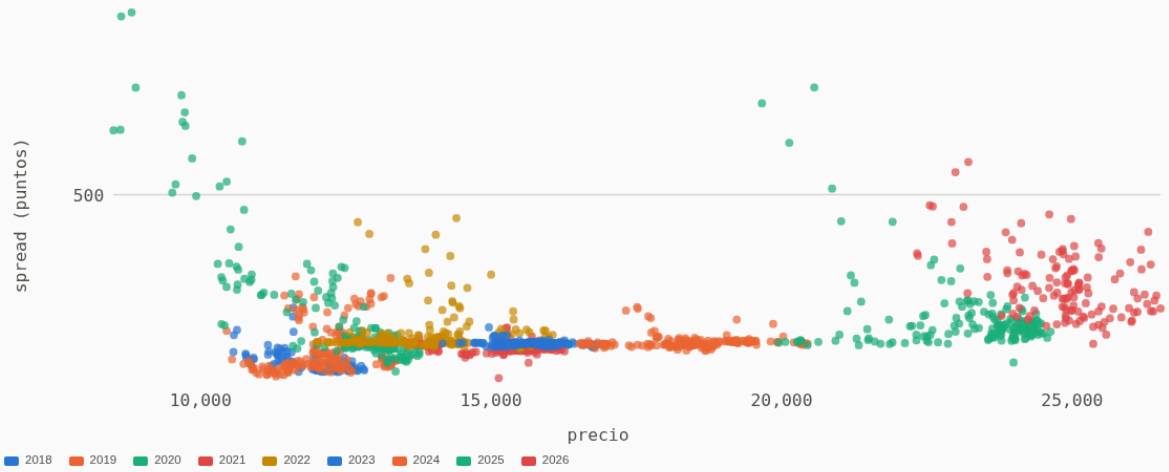
Curva Rangos para el MC Retest Calibración

MC Retest de spread hoy: 163.8 – 366.3 puntos

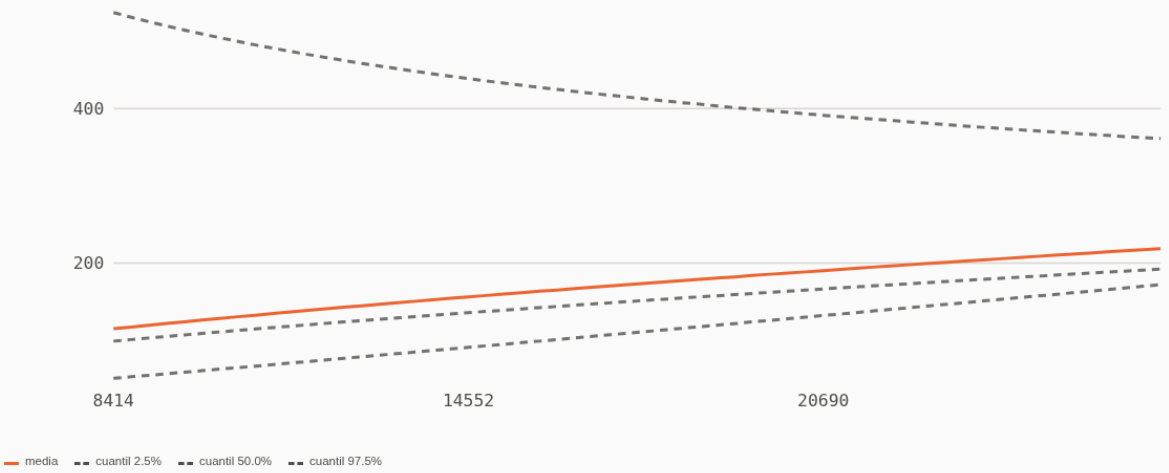
Al último precio (25,360.7) la curva media da 213.1 puntos y la banda 0.025–0.975, 163.8 a 366.3. Exponente de la media b = 0.56 (1 = proporcional al precio, 0 = fijo en puntos). Peor año fuera de la banda por un lado: 13 % de sus días (lo esperado, 2,5 %).

Curva

Spread medio de cada día contra el precio



Curvas ajustadas: media y banda



Coeficientes

CURVA	A	B
media	0.6696	0.562

- **b** dice cuánto sigue el spread al precio: 1 es proporcional, 0 es fijo en puntos.
- **«Rangos para el MC Retest»**: la banda al precio típico de cada año y al último precio. Eliges la fila del precio típico del tramo.
- **«Calibración»**: qué parte de los días de cada año cae fuera de la banda. Lo esperado es un 2,5 % por cada lado. Un año muy por encima es un régimen que el precio no explica.

```
DAX40      b=+0.56  hoy (precio 25,360.7): hoy media 213.1  hoy 0.025 163.8  hoy 0.5 187.1  hoy 0.975
366.3
DJ30       b=-0.01  hoy (precio 51,411.0): hoy media 286.1  hoy 0.025 178.5  hoy 0.5 294.3  hoy 0.975
391.6
NIKKEI225  b=-0.63  hoy (precio 65,775.0): hoy media 389.3  hoy 0.025 171.6  hoy 0.5 274.5  hoy 0.975
898.1
USA500     b=-0.10  hoy (precio 7,688.1): hoy media 37.6  hoy 0.025 29.2  hoy 0.5 29.5  hoy 0.975 60.2
USATEC     b=-0.14  hoy (precio 30,384.3): hoy media 85.5  hoy 0.025 81.9  hoy 0.5 82.9  hoy 0.975
137.5
XAUUSD     b=+1.18  hoy (precio 4,274.5): hoy media 49.2  hoy 0.025 17.8  hoy 0.5 49.3  hoy 0.975 83.4
USDJPY     b=+2.26  hoy (precio 158.6): hoy media 1.0  hoy 0.025 0.7  hoy 0.5 1.0  hoy 0.975 1.5
```

Ojo: en los índices, salvo el DAX, el precio apenas explica el spread (b ≈ 0, o negativo en el Nikkei). El spread va por regímenes: 2020 y 2026 salen por encima de la banda en un 9–12 % de sus días. La banda describe la dispersión que hubo, pero no la predice desde el precio.

Dar de alta un activo nuevo (o rehacer sus spreads)

Todo lo anterior, de una vez, con tus reglas por defecto (en `onboard:` del `config.yaml` del estudio). Se lanza pidiéndoselo a Claude («haz lo mismo para el cruce EURGBP», skill `/asset-onboard`) o a mano:

```
python3 -m studies.data.spread.onboard --symbol EURGBP --kind forex          # solo enseña el plan
python3 -m studies.data.spread.onboard --symbol EURGBP --kind forex --write   # lo escribe en assets/
python3 -m studies.data.spread.onboard --symbol XAUUSD --kind metal --write --spread-only
```

flag	obligatorio	qué hace
`--symbol`	sí	el activo, como en SQX (`EURGBP`)
`--kind`	sí	`index` , `metal` o `forex` : decide tramos, comisión, swap y triple swap
`--bars` , `--ticks`	si hay varios brokers	el feed DukasM1 y el DarwTick a usar
`--write`	no	lo escribe; sin él solo imprime el plan
`--spread-only`	no	activo existente: solo spreads, slippage y rango del MC Retest

- **Qué calcula:** el spread de cada tramo (media de Darwinex × 1,25, con el modelo donde no hay ticks), el slippage (la mitad), la comisión (0 en índices; 8 USD por lote, pasados a % en metales), el swap (−8 % en índices, −7 % en metales, la media de los brokers de SQX en forex), el triple swap (viernes en índices) y el rango del MC Retest.
- **Cuánto tarda:** 15–40 s la primera vez, porque decodifica los ticks. No toca SQX.

```
EURGBP (forex) — EURGBP_DukasM1_the5ers + EURGBP_DarwTick_the5ers
  modelo del build                    relativo
  spread por tramo (puntos, × factor)  {'build': 1.0, 'oos1': 1.04, 'oos2': 1.03}
  spreads a declarar                  {'spread': 1.02}
  slippage                            {'slippage_is': 0.51, 'slippage_oos': 0.51}
  comisión                            8.0
  swap largo/corto                    (-5.2, 0.89)
  triple swap                         WEDNESDAY
  MC Retest spread (puntos)           (0.72, 1.56)
```

Qué NO te dice

- **El spread antes de octubre de 2017 no se mide: se modela.**
- El build 2008–2017 es un 97 % modelo.
- El error del modelo solo se puede medir sobre 2017–2026: 19 % en el oro y 10 % en USDJPY.
- Si los brokers eran más caros antes, el modelo se queda corto justo ahí. El 1,25 es el único margen contra eso.
- **El modelo de USDJPY usa el nivel de precio**, que entre 2017 y 2026 subió con el tiempo. Aplicado a 2011–2012, con el yen a 76–80, predice los spreads más finos de toda la historia. Míralo con desconfianza.
- **El spread de Darwinex no es el de tu broker.** Es el que tienes, y el más restrictivo.

Si algo falla

- `KeyError` con el símbolo: el activo no está en `config.yaml` (hoy solo XAUUSD y USDJPY).

25. Actualizar los datos — el botón "Update all", automatizado y con red

Qué pregunta responde

Ninguna: esto no analiza nada. Hace una cosa y la hace con cuidado — descarga histórico nuevo en el maestro, igual que el botón **"Update all"** de la GUI, y después deja `assets/_policy.yaml` diciendo la verdad sobre hasta dónde llega cada feed.

Las dos mitades van juntas a propósito. Actualizar sin refrescar las fechas deja el proyecto creyendo que tu histórico acaba donde acababa hace meses.

Cuándo lo usas, y cuándo no

Lo usas cuando quieras la descarga desatendida — de madrugada, sin nadie delante — o cuando prefieras que el refresco de fechas vaya encadenado y no se te olvide.

No lo uses si sólo quieres refrescar las fechas sin descargar nada. Para eso está `python3 -m core.assets --dataranges`, que pregunta al conductor, no toca el maestro y **es seguro con tu GUI abierta**.

Y no lo uses con la GUI del maestro levantada, porque no te va a dejar.

Antes de empezar

Cierra StrategyQuant en el maestro. No hay forma de saltárselo y es deliberado. Si está abierto:

```
SQX está corriendo desde /home/sergioguslw/Desktop/SQX (PID 234724, 235060, 235064, …).
Ciérralo tú y vuelve a lanzarlo: la regla dura 2 prohíbe sqcli en el maestro
con la GUI levantada, y este comando acaba en un sync que borra .sqx de disco.
```

Sale con 1 y no toca nada. **Nunca mata el proceso**: lo cierras tú.

Cómo se ejecuta

```
python3 -m sqx.data.update              # ENSAYO: te dice qué haría y no hace nada
python3 -m sqx.data.update --apply      # descarga de verdad
python3 -m sqx.data.update --apply --symbol XAUUSD_DukasM1_Infinox
```

flag	obligatorio	qué hace
`--apply`	no	sin él es un ensayo. Es lo único que descarga
`--symbol`	no	un solo símbolo. Por defecto, todos los que el maestro tenga configurados

El ensayo imprime el comando exacto que lanzaría y cuántas estrategias hay en juego:

```
7546 .sqx en disco bajo /home/sergioguslw/Desktop/SQX/user/projects
ENSAYO. Lanzaría: ./sqcli -data action=update  (cwd /home/sergioguslw/Desktop/SQX)
Añade --apply para hacerlo de verdad.
```

Por qué cuenta los `.sqx` — esto es lo importante

Toda ejecución de `sqcli` termina así, la mires o no:

```
Synchronizing databanks to files
```

Y la **regla dura 1** dice qué significa eso: *cada sync de SQX borra los `.sqx` de disco que el databank no tiene en memoria*. En tu maestro eso son **7.546 estrategias y 3,4 GB** expuestos cada vez que se lanza cualquier comando.

Por eso el comando fotografía `user/projects` entero antes y después, guarda las dos fotos en `AlgoData/backups/data-update/` y compara:

```
foto guardada en …/AlgoData/backups/data-update/2026-09-22T213045Z.json
…
regla 1 ok: los 7546 .sqx siguen en disco
```

Si algo desapareciera lo dice con nombre y tamaño, y tienes la foto para saber exactamente qué era. Un informe vacío es el único resultado aceptable.

Por qué en el maestro y no en un worker

Porque una descarga hecha en un worker **se pierde**. `bin/sqx-worker.sh` hace esto en cada arranque:

```
rsync -a --exclude='History/' "$MASTER/user/data/" "$WORKER/user/data/"
```

Maestro → worker. El `History` es un enlace compartido, así que los ficheros de barras nuevos caerían bien, pero las tres bases H2 que un backtest lee de verdad son copias por instalación: la copia fresca del worker la machaca la vieja del maestro en el siguiente arranque, y tu GUI sigue construyendo con barras antiguas. Silencioso. Por eso la descarga va donde va.

Qué hace al terminar

Encadena el refresco de fechas él solo, así que no hay un segundo paso que olvidar:

```
XAUUSD: data: {from: 2003-05-05, to: 2026-01-16}   # …, 7.708.823 barras M1
     →  data: {from: 2003-05-05, to: 2026-09-22}   # …, 7.949.285 barras M1
```

Qué NO hace

- **No cierra SQX por ti**, ni lo mata. Se niega y te lo dice.
- **No toca ningún proyecto.** Sólo descarga datos y reescribe las fechas de `_policy.yaml`.
- **No decide ventanas.** Que un tramo `oos2` llegue más allá de los datos es un aviso del preflight, no algo que esto arregle. Ver `03-datos-costes-y-registro.pdf › 24-costes`.

Ojo

 Probada de punta a punta el 2026-09-25. Sin `--symbol` actualizó **los 67 símbolos** que tiene el maestro en **16 minutos**, y los 7.546 `.sqx` siguieron en disco. Los diecisiete activos pasaron de acabar el `2026-09-22` a acabar el `2026-09-25`.

⚠ **Algún día puede fallar por límite de descargas.** Aquella vez, dos días de `MSFT_DukasM1_ICMarkets` dieron `ERROR: download failed. Error code: 429`: el servidor de datos cortó por exceso de peticiones. No hace falta arreglar nada; la siguiente ejecución los baja. Búscalo así en la salida:

```
grep 'ERROR: download failed' <salida>
```

Haz igualmente una copia de `user/projects` antes (regla dura 1): la foto que guarda el comando es un inventario, no una copia.

Automático, cada sábado

Desde el 2026-09-25 cron lo lanza **los sábados a las 02:00** con `bin/weekly-data-update.sh`, sin modelo de por medio. Hace tres cosas en orden y, si una falla, no hace la siguiente:

1. Copia `user/projects` del maestro entero (unos 4 GB) en `AlgoData/snapshots/weekly-data-update-AAAA-MM-DD`, y comprueba que la copia tiene los mismos `.sqx`. Guarda las tres últimas copias y borra las anteriores, solo las suyas.
2. Lanza `python3 -m sqx.data.update --apply`. **Si tu GUI del maestro está abierta, no hace nada**, ni la copia.
3. Apunta los días que el servidor de datos rechazó (el `429` de arriba).

Lo que pasó lo lees en `AlgoData/logs/weekly-data-update.log`, ya sin las miles de líneas de ruido de `sqcli`. Para probarlo sin lanzar nada:

```
bin/weekly-data-update.sh --dry-run
```

Si un worker arranca durante la descarga, copia las bases de datos del maestro a medio escribir. Por eso a esa hora no debería haber nada arrancando.

El primer sábado del mes: `oos2` avanza solo

Pediste que los periodos acaben en fechas redondas, a fin de mes, sin tener que cambiar a mano los diecisiete `oos2` cada mes. Desde el 2026-09-30 lo hace `bin/monthly-oos2-roll.sh`, también sin modelo:

- Cron lo lanza **todos los sábados a las 02:05**. **El primer sábado del mes** mueve las ventanas; **los demás sábados sólo recoge al activo que se quedó atrás** ese día, con la misma fecha de final. Si no quedó ninguno, escribe tres líneas en el log y no cambia nada. (Hasta el 2026-10-03 los demás sábados no hacía nada, y un activo con el feed atrasado esperaba un mes entero.)
- **Espera a que termine la descarga de las 02:00** (el cerrojo de `weekly-data-update.sh`), porque es la descarga la que pone al día la línea `data:` de cada activo.
- Pone el final de `oos2` en **el último día del mes anterior**: el primer sábado de octubre, el 30 de septiembre.
- **Sólo mueve un activo si sus datos ya llegan** al último día laborable de ese mes (si el mes acaba en fin de semana, basta con el viernes). El que no llegue se queda donde estaba, aparece con ⚠ en el log, el script sale con 1 y te llega un aviso al escritorio. **Un activo atrasado no frena a los demás**: cada uno se mueve por su cuenta, y el sábado siguiente, con los datos ya al día, se mueve él.
- **No toca** los activos con `oos2` sin decidir (`to: null`) — esa ventana es tuya — ni mueve nunca un `oos2` hacia atrás. No toca `build` ni `oos1`.

Por dentro llama a:

```
python3 -m sqx.data.roll_oos2           # ENSAYO: qué movería, sin escribir
python3 -m sqx.data.roll_oos2 --apply   # lo escribe en assets/_policy.yaml
bin/monthly-oos2-roll.sh                # el script entero hoy (espera a la descarga, escribe y avisa)
```

El ensayo del 2026-09-30 decía:

```
oos2 → hasta 2026-08-31 (último día del mes anterior)
  AUDJPY: 2026-08-30 → 2026-08-31   (datos hasta 2026-09-25)
  …
  XAUUSD: 2026-08-30 → 2026-08-31   (datos hasta 2026-09-25)
ENSAYO. Añade --apply para escribirlo.
```

Lo que pasó lo lees en `AlgoData/logs/monthly-oos2-roll.log`. El cambio queda en `assets/_policy.yaml` sin commitear, como todo lo que hacen los agentes.

⚠ Los proyectos ya creados **no cambian**: llevan las fechas con las que se configuraron. El nuevo `oos2` vale para lo que se configure después.

El otro servidor (gorilla) recibe una copia

Desde el 2026-10-10 hay un segundo servidor, **gorilla**, que corre workflows con los mismos datos que éste (**leaseWeb**). Gorilla nunca descarga nada: los datos se bajan aquí y se copian allí. Así los dos calculan sobre las mismas barras, instrumentos y sesiones.

La copia la hace sola la actualización del sábado, al terminar bien. A mano, después de añadir un feed o si el sábado falló:

```
bin/mirror-data.sh --dry-run   # dice qué copiaría, no copia nada
bin/mirror-data.sh             # copia user/data del maestro de aquí al maestro de gorilla
```

Qué servidores reciben la copia está en `config/machine.yaml`, bloque `mirrors:` (el alias de SSH y la carpeta de SQX de allí).

⚠ **No copia nada si hay un SQX abierto en cualquiera de los dos lados**, y lo dice. Una copia con SQX escribiendo sale rota, y a un workflow en marcha en gorilla no se le pueden cambiar las barras a mitad. Si pasa el sábado, llega un aviso al escritorio: se cierra lo que esté abierto y se lanza a mano.

⚠ Lo que haya en la carpeta de datos del SQX de gorilla y no esté aquí **se borra**: es un espejo, no una mezcla.

Los workers de gorilla no necesitan nada más: cogen los datos nuevos de su maestro la próxima vez que arrancan.

Lanzar algo en gorilla: siempre con el código al día

Todo lo que se corre en gorilla pasa por un solo comando, lanzado desde aquí:

```
bin/remote.sh gorilla <comando>            # lo corre allí y espera a que acabe
bin/remote.sh --detach gorilla <comando>   # lo deja corriendo allí (un workflow) y vuelve
```

Antes de correr nada hace `git pull` en gorilla, así que allí siempre corre el último código subido a GitHub. Si el `git pull` falla, **no corre nada** y lo dice.

⚠ Si en gorilla hay un SQX en marcha, **no hace el `git pull`** y avisa: a un workflow que está corriendo no se le cambia el código a mitad. El comando que pidas sí se ejecuta.

⚠ Gorilla sólo recibe lo que está commiteado y subido. Lo que aquí esté sin commitear, allí no existe.

Dónde está el detalle

- `sqx/data/README.md` — el porqué del maestro y del rsync.

- `knowhow/costs/refreshing-sqx-costs.md` — la referencia de verbos y el flujo de datos entre instalaciones.
- `03-datos-costes-y-registro.pdf › 24-costes` — el refresco de fechas por separado, sin descargar nada.

24. Costes por activo — qué le cobras a cada mercado, y en qué unidad

Qué pregunta responde

Antes de construir o retestear nada hay que saber qué paga esa estrategia por operar: el spread, la comisión y el swap. SQX trae sus propios números y **casi nunca son los que te cobra tu bróker**, así que esa diferencia se escribe una vez en `assets/` y se consulta siempre. Esta página dice cómo está organizado eso, qué unidad lleva cada campo, y por qué un activo de forex se rellena distinto que el oro.

También contesta la otra mitad: en qué otros mercados hay que comprobar que la estrategia sigue funcionando, que es `assets/_markets.yaml`.

Cuándo lo usas, y cuándo no

Lo usas antes de tocar cualquier proyecto, tarea o plantilla de un símbolo. Es la regla dura 5 del `CLAUDE.md` y no es opcional: si el activo tiene un coste sin decidir, el comando falla y el trabajo se para hasta que tú des la cifra.

No lo usas para saber qué pagó de verdad un backtest que ya corrió. Eso no se supone, se recupera de las propias operaciones (`gross − P/L`), y lo hace `portfolio/common/monteCarlo/inputs/costs.py`. `assets/` dice lo que hay que aplicar de aquí en adelante, no lo que se aplicó.

Antes de empezar

Nada. No hace falta SQX abierto ni datos exportados: son ficheros de texto y un comando que los lee.

Cómo se ejecuta

```
python3 -m core.assets XAUUSD          # el preflight de un activo
python3 -m core.assets --index         # regenera assets/INDEX.md
python3 -m core.assets --dataranges    # relee de SQX hasta dónde llega cada feed
```

El tercero es el que usas **después de darle a "Update all" en el maestro**: pregunta a SQX qué histórico tiene ahora de cada feed y reescribe el `data:` de los 17 en `_policy.yaml`. Sólo imprime lo que se movió:

```
XAUUSD: data: {from: 2003-05-05, to: 2026-01-16}   # …, 7.708.823 barras M1
     →  data: {from: 2003-05-05, to: 2026-09-22}   # …, 7.949.285 barras M1
```

Pregunta al **conductor**, no al maestro, así que puedes lanzarlo con tu GUI abierta sin ningún riesgo. Ve los datos del maestro igualmente porque `bin/sqx-worker.sh` copia su `user/data` al worker en cada arranque, y el `History` es un enlace al del maestro.

Salida real del primero:

```
# XAUUSD — clase `no_forex`, overrides a aplicar

SQX symbol: XAUUSD_DukasM1_Infinox   verificado: 2026-09-03

- **spread_is**: usar `10.0` points (SQX lleva hoy `10`) — PROVISIONAL 2026-09-21 — default de SQX…
- **spread_oos**: usar `10.0` points (SQX lleva hoy `10`) — PROVISIONAL 2026-09-21 — default de SQX…
- **commission**: usar `0.00229` pct_of_notional (SQX lleva hoy `{'method': 'SizeBased', 'value': 8}`)…
- **swap_long**: usar `SIN DECIDIR` pct_annual (SQX lleva hoy `{'type': 'points', 'value': -73.42}`)
- **swap_short**: usar `SIN DECIDIR` pct_annual (SQX lleva hoy `{'type': 'points', 'value': 38.76}`)
- **segmento build**: 2008-01-01 a 2017-12-31 inclusive (dateFrom 1199145600000, dateTo 1514764800000)
- **segmento oos1**: 2018-01-01 a 2022-12-31 inclusive (dateFrom 1514764800000, dateTo 1672531200000)
- **segmento oos2**: 2023-01-01 a 2026-12-31 inclusive (…)  ⚠ RESERVADO para WFC, WFM
- **retest family**: XAGUSD_DukasM1_Infinox, BRENTCMDUSD_ftmo
- **retest structural**: (vacío)
```

Los tres códigos de salida, que son el mecanismo entero:

sale con	qué significa	qué haces
`0`	todo decidido	sigues, diciendo en voz alta qué valores aplicas
`2`	algún coste obligatorio sigue en `use: null`	**paras y preguntas.** No eliges tú un número
`3`	el fichero no cuadra con su clase	lo arreglas: te dice exactamente qué campo falta o sobra

Cómo está organizado

Tres ficheros compartidos arriba y los diecisiete de instrumento en `symbols/`, para que la carpeta se vea de un vistazo:

fichero	qué guarda	cuántos hay
`_policy.yaml`	qué es cada tramo IS/OOS (una vez) **y dónde empieza y acaba en cada activo**, con la fecha de datos de SQX al lado	uno, para todos
`_classes.yaml`	los dos esquemas de coste: qué campos, en qué unidad, en qué ajuste de SQX	uno, para todos
`_markets.yaml`	el universo de retest: por activo main, mercados `family` y `structural`	uno, para todos
`symbols/<SÍMBOLO>.yaml`	lo propio del instrumento: sus costes y sus rangos de MC Retest	uno por activo, **en su subcarpeta**

La idea es que abras un fichero de activo y **no veas nada repetido**. Los segmentos no están diecisiete veces, están una. Lo que quedó dentro es lo que de verdad cambia de un mercado a otro.

Los tramos IS/OOS: la plantilla se comparte, las fechas son de cada activo

Los históricos no empiezan a la vez. El oro tiene M1 desde el 2003-05-05 y tú construyes desde 2008; el DAX40 no tiene nada antes del 2013-09-30, así que una ventana que empiece en 2008 se quedaría corta **sin avisar**. Por eso `_policy.yaml` parte el asunto en dos:

- `segments_default` — **qué es** cada tramo: su papel y qué spread usa. Una sola copia.
- `segments` — **dónde empieza y acaba en cada activo**, con `data` al lado. Los 17 juntos, para poder compararlos y ajustarlos de una vez en lugar de abrir diecisiete ficheros.

```
segments:
  XAUUSD:
    data: {from: 2003-05-05, to: 2026-01-16}   # leído de SQX, no se edita a mano
    build: {from: 2008, to: 2017}
    oos1:  {from: 2018, to: 2022}
    oos2:  {from: 2023, to: 2026}
  DAX40:
    data: {from: 2013-09-30, to: 2026-01-16}
    build: {from: null, to: null}              # sin decidir
```

Un extremo puede ser **un año o una fecha**. El año significa el año entero, ambos extremos incluidos: `from: 2008, to: 2017` es del 1 de enero de 2008 al 31 de diciembre de 2017 completo. La fecha significa ese día, también incluido, y es como se corta un tramo a mitad de año: `to: 2026-08-30` llega hasta el último instante del 30 de agosto.

Las ventanas de forex y metales, decididas el 2026-09-22:

tramo	desde	hasta
`build`	2008-01-01	2017-12-31
`oos1`	2018-01-01	2022-12-31
`oos2`	2023-01-01	2026-08-30

Los seis índices siguen sin ventana a propósito: sus históricos empiezan en 2011-2013 y una ventana de 2008 no cabría — el comando la rechazaría.

`data` lo lee el proyecto de SQX con `bin/sqx-worker.sh run -symbol action=list` y está ahí justo para que veas si la ventana que pones cabe. **Si no cabe, el comando falla:**

```
ESQUEMA ROTO en assets/symbols/DAX40.yaml: el tramo `build` empieza en 2008 y los datos de
`DAX40_DukasM1_Infinox` empiezan en 2013-09-30
```

→ exit 3. Eso es fatal: esa ventana no se puede llenar nunca. En cambio, que un tramo **acabe después** del último dato es sólo un aviso, porque es el estado normal del tramo más reciente mientras no sincronizas:

```
AVISO: el tramo `oos2` llega a 2026-08-30 y los datos de `XAUUSD_DukasM1_Infinox`
acaban en 2026-01-16
```

Y si las dejas en `null`, nadie se inventa una ventana: `window()` se niega y el comando avisa.

Lo que tiene SQX hoy, leído el 2026-09-22:

activo	datos desde		activo	datos desde
EURUSD, GBPUSD, USDCHF, USDJPY, XAUUSD	2003-05-05		NIKKEI225	2011-09-19
AUDUSD, EURJPY, GBPJPY, USDCAD	2003-08-04		USA500, USATEC	2012-01-19
AUDJPY	2003-12-01		DAX40, DJ30	2013-09-30
CADJPY	2004-10-25		**SP500ft**	**no existe en SQX**

Las dos clases, que es lo que decide cómo se rellena

	`forex`	`no_forex` **(índices y metales)**
spread	`spread` , **un** valor en puntos	`spread_is` **y** `spread_oos` , en puntos
comisión	`commission` , **$ por lote** (`SizeBased`)	`commission` , **% del nocional** (`PercentageBased`)
slippage	`slippage` , en puntos	igual
swap	`swap_long` / `swap_short` , **puntos por noche**	`swap_long` / `swap_short` , **% ANUAL**

Por qué dos spreads fuera de forex. El precio de estos activos se multiplica a lo largo del histórico: el oro va de 800 a 3500. Un spread fijo en puntos cuesta siempre lo mismo en dólares, así que en porcentaje cuesta **el triple en 2010 que hoy** — y eso mete una pendiente dentro de la misma muestra que ve el generador. Declarando uno para el tramo de construcción y otro para los dos fuera de muestra, esa pendiente desaparece. Se puede hacer porque **SQX guarda los costes dentro de cada tarea**, no una vez por proyecto: la tarea de build lleva un `defaultSpread` y las de retest otro.

Por qué el % fuera de forex. Un porcentaje escala con el precio él solo, que es justo lo que le falta a un número fijo en puntos.

Los rangos del MC Retest

La tarea **MC Retest** vuelve a correr el backtest entero contra una entrada perturbada, y dos de sus métodos sortean un valor dentro de un rango: el spread y el slippage. Ese rango lo pones tú, por activo, al final de su fichero:

```
mc_retest:
  spread:
    min: 5                    # puntos
    max: 12
    sqx_now: [{min: 1, max: 5}, {min: 5, max: 12}]   # varias tareas, rangos distintos
  slippage:
    min: 0
    max: 10
    sqx_now: [{min: 0, max: 5}, {min: 0, max: 10}]
```

Los dos van en puntos absolutos, no en múltiplos del spread real. Eso es lo que obliga a que sean por activo: un rango de 1 a 5 puntos es razonable en EURUSD y no significa nada en el NIKKEI225.

Si los dejas sin decidir **no se bloquea nada**: el comando avisa y sale con 0, porque un rango sin fijar sólo deja sin interpretar esa tarea concreta del MC Retest, no el resto del trabajo.

```
AVISO: los rangos MC Retest spread, slippage no están decididos. No bloquea,
pero esa tarea del MC Retest no es interpretable hasta que el dueño los fije.
```

⚠ Mirando el maestro el 2026-09-22, **casi todos los instrumentos llevan el rango de fábrica de SQX** —spread 1,0–5,0 y slippage 0,0–5,0— que es idéntico en todos los mercados. En el NIKKEI225, cuyo spread real son 1100 puntos, eso perturba entre 1 y 5: unas 200 veces más barato que el propio backtest. Donde sí los has puesto a mano están a la escala del activo: el oro lleva spread 5–12 contra un spread real de 10, y el NIKKEI225 lleva 80–200 y 120–400 en sus otras tareas.

(`RandomizeMinDistance` tiene la misma forma y rango de fábrica 0,0–10,0. No está en `assets/` porque pediste estos dos.)

Las dos trampas, que cuestan dinero de verdad

⚠ **El swap en % es ANUAL, no por noche.** SQX divide entre 100 y entre 360 antes de aplicarlo. Si escribes ahí el porcentaje de una noche, te equivocas por 360 veces. Para pasar un swap que hoy está en puntos (y, por la misma cuenta, una comisión que hoy está en $/lote):

```
pct_anual = puntos × tick_size × 36000 / precio_de_referencia
pct_comisión = usd_por_lote / (point_value × precio_de_referencia) × 100
```

⚠ **El precio de referencia es la MEDIANA del precio del tramo que se está cargando — `oos1`, no el último cierre** (decisión del dueño, 2026-09-29, OPEN.md #36). Un activo cuyo precio se mueve mucho a lo largo de los años puede errar el % casi 5 veces si se usa el cierre de hoy en vez de la mediana del tramo que de verdad se retestea: la plata cerró a 66,6 el día que se escribió esto pero cotizó una mediana de sólo 18,5 en oos1 (2018-2022), y su comisión salía **0,00863 %** con la mediana contra **0,00252 %** al cierre — casi 5 veces menos. `studies/data/spread/onboard.py` (`_oos1_price`) ya lo hace así para toda comisión nueva; el oro y la plata quedaron recalculados con la mediana el 2026-09-29.

Ejemplo con Brent (`BRENTCMDUSD_ftmo`, el único no-forex que sigue derivando el swap del punto — el oro y la plata ya llevan un `-7 %` anual fijo del dueño, sin relación con el precio): los `-25.76` puntos del corto, con tick `0.001` y la mediana de `oos1` (68,578), son **-13,52 % anual**; al último cierre de entonces (97,235) salían -9,54 %, casi 1,5 veces menos negativos de lo que el tramo retesteado paga de verdad.

⚠ **La comisión en % puede estar cobrándose por pata o por operación, y no está verificado.** El código de SQX sólo cobra al abrir, pero lo medido en los exports con `SizeBased 8` son 16 $ ida y vuelta. Es un factor de 2 sobre toda la comisión de los activos no-forex. Antes de dar por buena una cifra, hay que comprobarlo con un backtest y el residual `gross − P/L`.

La comisión de un activo no-forex es la de Darwinex, EN TODO EL WORKFLOW

Decisión del dueño, 2026-09-29, corrigiendo la misma tarde una lectura anterior ("aplica el máximo de cada a la hora de buildear y testear", entendida como elegir el bróker más caro tramo a tramo): en un activo `no_forex` con comisión pactada (XAUUSD, XAGUSD, BRENT), **todo** el workflow de SQX — la construcción y cada retest, de OOS a WFM y las variantes — usa la comisión de **Darwinex, 0,005 % del nocional**, la misma en `build`, `oos1` y `oos2`. El forex sigue en 5 USD/lote fijos en los tres tramos; los índices siguen en 0 %: ninguno de los dos cambia. Se escribe como cualquier otro coste, con `core.assetwrite.set_cost` repartiendo la cifra plana sobre los tres tramos en el método propio de la clase:

```
costs:
  commission:
    use:                    # el MISMO {method, value} en los tres tramos
      build: {method: PercentageBased, value: 0.005}
      oos1:  {method: PercentageBased, value: 0.005}
      oos2:  {method: PercentageBased, value: 0.005}
    brokers:
      darwinex: {method: PercentageBased, value: 0.005, unit: pct_of_notional,
                 source: https://help.darwinex.com/execution-costs, date: '2026-09-29', confirmed:
true,
                 pct_now: 0.005, price_now: 4369.12, price_date: '2026-09-22'}
      infinox:  {method: SizeBased, value: 8.0, unit: usd_per_lot,
                 source: "dueño, 2026-09-29", date: '2026-09-29', confirmed: true,
                 pct_now: 0.001831, price_now: 4369.12, price_date: '2026-09-22'}
```

Las cifras del dueño, 2026-09-29, aplicadas:

clase	activos	comisión de `use`
forex	los diez pares	**5 USD/lote, SizeBased**, todos los brokers, los tres tramos igual
índices	DAX40, DJ30, NIKKEI225, USA500, USATEC	**0 %, PercentageBased**, todos los brokers, los tres tramos
oro/plata/crudo	XAUUSD, XAGUSD, BRENT	**Darwinex 0,005 % PercentageBased**, todos los tramos igual

La cifra de cada bróker sigue existiendo aparte, en `costs.commission.brokers`, para el test final del paso 26 (MT5 sobre el feed de cada firma de fondeo) y para la conciliación semanal del `weeklyReconciler` — nunca para `use`. Cada bróker confirmado lleva además su propia comisión convertida a un porcentaje **a precio de hoy**: `pct_now`, `price_now` y `price_date`, que `python3 -m core.commission --refresh` recalcula cada semana (justo después de `bin/weekly-data-update.sh`) leyendo la última vela que `core.barstore` tiene del feed del activo — su cierre, no la mediana de un tramo, porque aquí la pregunta es "qué me cobraría ese bróker hoy", no "qué pagó el tramo que se retesteó". Un bróker en $/lote sin barras sincronizadas todavía (los cinco índices) se deja sin precio en vez de inventarlo; uno en % no necesita precio.

```
python3 -m core.commission --refresh
```

```
XAUUSD: darwinex 0.005%, ftmo 0.0014%, fundednext 0.0016%, infinox 0.001831%
BRENT: ftmo 0.0014%, darwinex 0.005%, infinox 0.079965%
```

El código que hace la conversión, `core.commission.commission_pct(method, value, price, point_value)`, es el mismo que antes valoraba el ganador de cada tramo — sólo que ahora nadie lo usa para decidir `use`, y lo lee `core.commission.broker_pct(asset, broker)`, el único accesor que el paso 26 y el `weeklyReconciler` usan para leer una comisión de bróker. En Brent, cuyo precio (≈100) es bajo frente a su `point_value` (100), el 8 $/lote fijo de Infinox convierte a casi un 0,08 % — muy por encima del 0,005 % de Darwinex; en el oro, al revés, Infinox convierte a menos del 0,002 %. Es justo la comparación que antes decidía el ganador de cada tramo, ahora guardada sólo como referencia.

Los dos tipos de mercado adicional

`_markets.yaml` separa en dos, y la diferencia importa porque responden a preguntas distintas:

- **`family`** — mismo motor económico que el activo principal. Oro y plata, por ejemplo. Es la prueba fácil: **pasarla no demuestra gran cosa**, porque puede que los dos mercados sean la misma operación con otro nombre. Fallarla sí dice algo.
- **`structural`** — parecidos en estructura (volatilidad, horario, ruido) pero **sin ningún motor compartido**. Es la prueba dura: si el edge transfiere aquí, es que la lógica capta algo del movimiento del precio y no una exposición a un factor macro.

La lista se cierra antes de mirar ningún resultado. Elegir los mercados después de ver dónde funciona convierte la prueba en una selección, y sus p-valores en decoración.

Cómo se rellena un valor

Abres `assets/<SÍMBOLO>.yaml` y en el bloque del campo pones el número en `use:` y el motivo en `why:`. `sqx_now` no se toca: es lo que SQX lleva hoy, y está ahí para que veas de qué partes — ojo, **muchas veces en otra unidad** que la que va en `use`.

```
costs:
  spread_is:
    use: 14                  # points — tramo `build`
    sqx_now: 10              # points, el unico que hay hoy
    why: "Infinox, media de la sesión de Londres 2008-2017, confirmado 2026-XX-XX"
```

Sólo **el spread y la comisión bloquean**. El slippage y los swaps no, y los dos llevan ya una cifra PROVISIONAL: el slippage es **la mitad del spread** de cada instrumento.

No es una medición y no puede serlo — un backtest aplica el slippage que le das, así que de un export no se recupera nunca el real. Es una convención, y se eligió ésa porque es la única que aguanta entre clases: un punto vale `tick_size × point_value`, que va de 0,0065 $ en el Nikkei a 11,53 $ en USDCHF, así que 2,5 puntos planos serían 0,02 $ en uno y 28,82 $ en el otro. La mitad del spread deja el coste entre 0,33 $ y 5,00 $ por lote y pata, y en el oro cae en 5 puntos: el punto medio del rango 0-10 que tu propio MC Retest ya sortea.

Si escribes `PROVISIONAL` en el `why`, el valor **decide pero no bloquea**: el comando sale con 0 y todo informe construido encima queda marcado, para que dentro de seis meses nadie confunda un número de relleno con una cifra pactada con el bróker.

Cómo se sabe que está bien

`python3 -m core.assets <SÍMBOLO>` sale con 0 y el índice lo marca:

```
python3 -m core.assets --index && head -8 assets/INDEX.md
```

`` decidido, `⚠ provisional` decidido con un número de relleno, `–` bloqueado.

Qué NO hace

- **No escribe en SQX.** `core.assets.sqx_settings()` devuelve lo que una tarea tendría que llevar, pero meterlo en el proyecto es otro trabajo. Hoy hay que hacerlo a mano en la GUI.
- **No adivina.** Si no hay cifra pactada, hay `null` y el trabajo se para. Un fichero con valores inventados es peor que no tener fichero: parece decidido.
- **No sabe qué te cobró un backtest ya corrido.** Eso se recupera de las operaciones.

Dónde está el detalle

- `assets/RULES.md` — la regla y el esquema, para una sesión de Claude.
- `knowhow/costs/` — qué puede cobrar SQX y en qué unidad, con las fórmulas decompiladas.
- `assets/_classes.yaml` — los dos esquemas, uno al lado del otro.

43. El libro mayor de la búsqueda — cuántas cosas se han probado de verdad

> **Cambio del 2026-09-28.** La regla de «no quemar el oos2» ya no se aplica a humanos: puedes mirar el `oos2` y los pasos 17, 18 y 19 cuando quieras, desde la ventana o el terminal. Sólo un agente autónomo que decida solo (`ALGO_AUTONOMOUS=1`) sigue encontrando la puerta cerrada. El ledger sigue contando cada mirada.

Qué pregunta responde

Cada paso de la cadena cuenta su propio embudo y se le olvida. Cuando llegas al paso 17 con tres supervivientes, **nadie sabe si salieron de diez mil o de cincuenta** — y esa diferencia no es contabilidad, es el resultado.

Este módulo es el libro que no se olvida. Escribe **una línea por búsqueda** —una construcción, un retest, una criba de Python, un umbral aplicado— durante toda la vida de un estudio, y con eso contesta tres cosas que hoy nadie puede contestar:

1. **El embudo entero**: de cuántas a cuántas, paso a paso, con los umbrales de cada corte.
2. **Cuánta historia se ha gastado**: qué tramos ha mirado alguien, cuántas veces, y si el `oos2` sigue siendo virgen de verdad.
3. **Cuántas cosas se han probado en total**, que es el número que el Sharpe desinflado necesita y que hoy recibe subestimado — porque se le da la dispersión de un lote de variantes en vez de la de todo el estudio.

Y hace cumplir dos reglas que hasta ahora dependían de la buena voluntad de quien corriera la cadena: **nadie mira el `oos2` si no es el paso 17 o el 19**, y **el paso 20 no se lee hasta que 17, 18 y 19 estén los tres hechos**. No avisa: se niega.

Cuándo lo usas, y cuándo no

Lo usas siempre. Un estudio sin ledger no puede decir lo que vale su superviviente.

No lo usas para decidir nada por ti: no puntúa estrategias ni las descarta. Cuenta.

Y **no edites `ledger/thresholds.yaml` para que algo pase**. Un umbral movido después de ver los resultados es la manera más barata de sobreajustar la cadena entera.

Antes de empezar

Nada. El fichero de un estudio se crea solo la primera vez que se le escribe, en `~/Desktop/AlgoData/ledger/<estudio>.jsonl`.

Un **estudio** es un activo + un timeframe + una plantilla o familia de plantillas: `XAUUSD_M30_DirectionalMomentum`. Es la unidad a la que se le debe la corrección por comparaciones múltiples: todas las búsquedas que han tocado ese activo con esa plantilla cuentan juntas, venga del paso que venga.

Cómo se ejecuta

Leer un estudio:

```
python3 -m ledger.report --study XAUUSD_M30_DirectionalMomentum
```

Comprobar que cada umbral llega a su módulo desde el ledger:

```
python3 -m ledger.report --check-thresholds
```

Reconstruir hacia atrás un estudio que ya corrió, desde lo que dejó en disco:

```
python3 -m ledger.backfill \
    --gate ~/Desktop/AlgoData/reports/XAU_ISOOS_ejemplo/Results/2026-09-23/gate \
    --symbol XAUUSD --timeframe M30 --family DirectionalMomentum --write
```

Reconstruir los pasos 17, 18 y 19, que no se apuntan solos en el ledger (sin sus filas la puerta ciega del paso 20 no se abre nunca). Primero sin `--write`, para ver las filas; luego con él, **una sola vez** — si ya están, se niega a escribirlas otra vez:

```
python3 -m ledger.backfill --blind USDJPY_workflow_profiling_v1 --wfm-databank WFM \
    --symbol USDJPY --timeframe H1 --family crossAboveHMA_v1 --write
```

flag	obligatorio	qué hace
`--study`	sí (en `report`)	el identificador del estudio
`--check-thresholds`	—	dice de cada umbral si su módulo lo lee del ledger o de una copia, y sale con error si una copia diverge
`--equity` + `--identity`	no	añade el Sharpe desinflado de una superviviente concreta
`--gate` o `--blind`	uno de los dos (en `backfill`)	el directorio del informe de la puerta, o el proyecto cuyos WFC, CSCV y WFM se reconstruyen (una fila por tramo que leyó cada uno)
`--wfm-databank`	no (con `--blind`; por defecto `WFM`)	el databank del WFM que leyó el paso 19
`--write`	no	sin él, `backfill` enseña lo que haría y no escribe

Segundos, y no toca SQX.

Qué produce

Un `.jsonl` por estudio bajo el data root: **una línea JSON por búsqueda, y nada la reescribe nunca**. Se añade al final; no hay borrado ni edición, porque un libro que se puede editar después contesta una pregunta distinta de aquella para la que se hizo.

Cómo se lee el resultado

```
XAUUSD_M30_DirectionalMomentum · 8 búsquedas ·
/home/…/AlgoData/ledger/XAUUSD_M30_DirectionalMomentum.jsonl

-- el embudo, paso a paso
        ts  step        criterion segment  n_in  n_out   kept
2026-09-23     8   studies/screening/gate/presencia    oos1   120    115 0.9583
2026-09-23     8     studies/screening/gate/sanidad    oos1   115    112 0.9739
2026-09-23     8   studies/screening/gate/estaticas    oos1   112     64 0.5714
2026-09-23     8 studies/screening/gate/degradacion    oos1    64     45 0.7031
2026-09-23     8       studies/screening/gate/forma    oos1    45     45 1.0000
2026-09-23     8        studies/screening/gate/mono    oos1    45     45 1.0000
2026-09-23     8     studies/screening/gate/familia    oos1    45     45 1.0000
2026-09-23     8 studies/screening/gate/redundancia    oos1    45     45 1.0000
de 120 entraron a 45 supervivientes

-- qué historia se ha gastado
segment  searches steps      first       last
   oos1         8   [8] 2026-09-23 2026-09-23
  build  2008 → 2017  leído 0x
  oos1   2018 → 2022  leído 8x
  oos2   2023 → 2026-08-30  leído 0x  RESERVADO para WFC, WFM

-- la puerta ciega del paso 20
  17 pendiente · 18 pendiente · 19 pendiente
  ledger: el paso 20 es ciego hasta que 17, 18 y 19 estén los tres hechos; faltan [17, 18, 19].
  Mirar dos antes de correr el tercero contamina la decisión de correrlo, y el oos2 es la última bala

-- lo que se ha probado en total: N = 115 candidatos sobre 1 búsqueda(s), sigma = 0.3506 (annualised)
  benchmark con N de la última búsqueda (  115): 0.0570 (por día)
  benchmark con N de el estudio entero  (  115): 0.0570 (por día)
  los dos coinciden: este estudio sólo tiene una búsqueda con distribución registrada
```

Desde el 2026-10-02 el último bloque cuenta también las **ideas gastadas**. Así sale hoy en `XAUUSD_H4_freeShellLong`:

```
-- lo que se ha probado en total: N = 15996 = 15967 candidatos de 2 búsqueda(s) + 29 hipótesis medidas
en 1 idea(s) (long), sigma = 0.0211 (per_period)
  aparte, sin sumar: 441 medidas del perfil de mercado y su barrido, corregidas entre sí
  benchmark con N de la última búsqueda  (10998): 0.082084 (por día)
  benchmark con N de todas las búsquedas (15967): 0.083981 (por día)
  benchmark con N de búsquedas + ideas   (15996): 0.083990 (por día)
```

El embudo se lee de arriba abajo. `kept` es qué fracción sobrevivió a ese corte. Aquí las estáticas se llevaron casi la mitad (112 → 64) y la degradación otro 30 %.

El mapa de gasto es el que contesta «¿sigue siendo holdout mi holdout?». `oos1` leído 8 veces ya no es fuera de muestra en ningún sentido útil; `oos2` con 0 lecturas sí lo es.

La puerta ciega dice qué pasos de 17, 18 y 19 están hechos. Para ti está siempre abierta (dueño, 2026-09-28): sólo con `ALGO_AUTONOMOUS=1` —un agente autónomo— un módulo que llame a `gate.allow_read` revienta mientras falte uno, que es lo que enseña la salida de ejemplo de arriba.

El total probado es la razón de ser del módulo: `sigma` es la dispersión de los candidatos entre los que se eligió, y `N` cuántos fueron. Los dos alimentan el Sharpe desinflado. En este estudio los dos benchmarks coinciden porque **sólo una búsqueda registró su distribución** — el `backfill` sólo puede recuperar lo que la puerta dejó escrito, y de los pasos anteriores no queda registro. Con dos búsquedas ya se separan: medido en `tests/test_ledger.py`, al pasar de N=1.600 a N=2.000 el listón sube de 0,1995 a 0,2031 y el Sharpe desinflado baja de **0,6301 a 0,6000**.

Las ideas también cuentan (decisión tuya, 2026-10-02). Antes de que exista ninguna búsqueda, el experto en ideas ya ha mirado los datos: cada hipótesis que midió sobre `build` es un ensayo. El informe las suma a `N` y las enseña aparte, «N candidatos + K hipótesis». Tres reglas:

- **Cuenta la hipótesis medida, no la idea.** Un fichero de ideas con 3 ideas y 29 hipótesis medidas suma 29, y **una sola vez por estudio** aunque varias de sus ideas caigan en él. Si el fichero tiene un añadido con un total nuevo («29: 24 de antes + 5 nuevas»), vale el mayor.
- **Largos y cortos no comparten cuenta.** Una idea larga sólo sube el listón de los estudios largos de ese activo y timeframe; la dirección del estudio es la de su plantilla. Una idea que aún no tiene plantilla cuenta en todos los estudios de su activo, timeframe y dirección; un estudio que no es de una plantilla (`mt5verify`) recibe las dos direcciones.
- **Las medidas del perfil de mercado no entran.** Se enseñan en la línea «aparte, sin sumar»: ya se corrigen entre sí y relanzar el perfil repite filas sin mirar nada nuevo.

Las hipótesis suben `N` y no tocan `sigma`: no tienen Sharpe que agrupar. **Lo que cuesta hoy: nada medible.** Con 15.967 candidatos ya contados, 29 hipótesis mueven el listón de 0,083981 a 0,083990 y el Sharpe desinflado del mejor candidato de 0,2021 a 0,2020; ninguna estrategia cambia de lado. Pesa cuando el estudio es pequeño: en `tests/test_ledger_ideas.py`, 29 hipótesis sobre 50 candidatos bajan el Sharpe desinflado de 0,4025 a 0,3764. La ventana (Registro de búsquedas) sigue enseñando sólo las búsquedas.

⚠ **Las unidades no se mezclan.** SQX guarda Sharpes anualizados y las fórmulas de `core/significance.py` trabajan por observación. Cada línea apunta en qué unidad venían sus candidatos, y agrupar dos unidades distintas **lanza** en vez de dar un número creíble y falso.

Un ejemplo completo

La comprobación de umbrales, con salida real:

```
                              key  declarado lee_de  coincide fijado_por         el
          gate.sanidad.min_trades      20.00 ledger      True      dueño 2026-09-23
    gate.degradacion.min_retention       0.00 ledger      True      dueño 2026-09-23
gate.degradacion.max_concentration       1.00 ledger      True      dueño 2026-09-23
           gate.forma.max_dd_ratio       5.00 ledger      True      dueño 2026-09-23
                   gate.mono.max_p       0.50 ledger      True      dueño 2026-09-23
                gate.familia.alpha       0.05 ledger      True      dueño 2026-09-23
         snoopingScreen.stepm.fwer       0.05 ledger      True      dueño 2026-09-25
             entryQuality.dcr_high       0.30 ledger      True     agente 2026-09-24
          parameterCloud.rank_high       0.95 ledger      True     agente 2026-09-24
        parameterCloud.plateau_low       0.15 ledger      True     agente 2026-09-24
            parameterCloud.rho_low       0.20 ledger      True     agente 2026-09-24
                       cscv.blocks      12.00 ledger      True      dueño 2026-09-23

14 umbrales declarados: 14 los lee su módulo del ledger y 0 son copias que coinciden
```

Desde el 2026-09-26 **los módulos leen cada umbral de aquí**: donde su `config.yaml` tenía el número ahora pone `ledger:<clave>`, y el módulo lo sustituye al leer. La columna `lee_de` dice `ledger` cuando es así y `copia` cuando un módulo todavía guarda su propio número; una copia que no coincide, o un hueco que apunta a otra clave, sale como divergencia y la orden termina con error — que es como está pensado para meterse en un guion. Con los seis módulos migrados no queda ninguna copia. **Para cambiar un umbral, cámbialo aquí**, con tu nombre y la fecha: el `config.yaml` ya no tiene el número. `--set` sigue sirviendo para probar otro valor en una corrida sin tocar nada.

Qué NO te dice

- **No sabe lo que no se registró.** Una búsqueda que no llamó al ledger no existió para él, y los pasos anteriores a que esto existiera sólo se recuperan si dejaron un artefacto por escrito. Las líneas reconstruidas van marcadas `backfill` y llevan la fecha del informe, no la de hoy.
- **No cuenta las ideas que no dejaron fichero.** Lee `AlgoData/ideas/<ACTIVO>/*.md` y su línea «Hipótesis medidas: N»; una idea pensada fuera de ahí, o un fichero sin esa línea, suma cero.
- **No decide nada.** No puntúa, no descarta y no ordena.

Si algo falla

- `PermissionError: el paso N no puede mirar oos2` — **no es un fallo, es la puerta**. Si crees que ese paso sí debería poder, se arregla en `assets/_policy.yaml`, que es tuyo, y no ensanchando la comprobación.
- `ValueError: las búsquedas mezclan unidades de score` — alguien registró Sharpes anualizados y otro por observación en el mismo estudio. Hay que decidir cuál y rehacer el registro de esa fase.
- `KeyError: '<clave>: declared 0 times in thresholds.yaml'` (o `2 times`) — un módulo pide un umbral que el registro no tiene, o que dos ramas añadieron dos veces. Se arregla en `ledger/thresholds.yaml`: una fila por clave. No se arregla poniendo el número en el `config.yaml`.
- `sin búsquedas registradas todavía` — el estudio no tiene fichero: o el identificador no coincide (mira `~/Desktop/AlgoData/ledger/`) o nadie ha escrito nada aún.

Filas que la ventana y la WFM apuntan solas, y la calibración

Desde 2026-10-04 el build (paso 6, al lanzarlo desde la ventana) apunta su fila con las estrategias que SQX generó y aceptó, y la WFM (paso 19) apunta las suyas al informar; `ledger.report` enseña el N del estudio y el N global entre todos. Para apuntar a posteriori las poblaciones `Test_Calib_*`:

```
python3 -m ledger.backfill --calib ~/Desktop/AlgoData/scratch/calib_final/populations.csv --dry-run
```

Sin `--write` (o con `--dry-run`) solo imprime las filas y su total; con `--write` las escribe.

Lo probado, no sólo lo elegido: `ledger.tried`

Cuántas cosas se probaron decide cuánto hay que descontar al ganador. Si de un barrido sólo queda el elegido, la corrección se hace a ciegas. `ledger.tried` apunta **una línea por cada valor probado** en `~/Desktop/AlgoData/ledger/tried/<estudio>.jsonl` (contrato L2; el ledger de siempre sigue siendo una línea por búsqueda). Para lo que pruebas a mano —percentiles, un umbral, otra plantilla parecida, un bloque descartado tras mirarlo—:

```
python3 -m ledger.tried --study XAUUSD_M30_DirectionalMomentum --name atr_percentil \
    --values 50,60,70,80 --scores 0.011,0.014,0.009,0.012 --chosen 60 --why "mejor Sharpe en IS"
```

```
$ python3 -m ledger.tried --study XAUUSD_M30_DirectionalMomentum --name atr_percentil --values 50,60,70,80 --…
XAUUSD_M30_DirectionalMomentum: 4 pruebas apuntadas (b32956031256) · probados 4, elegido 60, cuentan 4
```

- `--chosen` se deja fuera si el barrido sólo miró: **un barrido sin elegido no cuenta para N**.
- `--fix` marca un barrido repetido sólo porque el código falló: se escribe y no cuenta.
- `--kind` (parameter, threshold, config, template, block), `--provenance` (propia, libro y página, paper, web) y `--step` completan la línea.
- **Las reejecuciones se detectan solas** en la puerta OOS: correrla otra vez sobre la misma cosecha con otro `--set` se apunta como un ensayo más del mismo barrido, sin que nadie lo pida.
- `ledger.report` añade la tabla «probados K, elegido 1» y suma esas pruebas al N. Las puntuaciones en Sharpe (por observación o anualizado) entran también en la sigma; las demás suman a N y no se mezclan.

12. Catálogo de rendimiento — qué cuesta cada parte del proyecto, y si hoy cuesta más que ayer

Qué pregunta responde

Este proyecto es caro: miles de estrategias, cientos de miles de simulaciones, y una máquina que comparte con la GUI de StrategyQuant X. El catálogo mide **cuánto tarda y cuánta memoria pide** cada parte cara del código, guarda la medida con la fecha y el commit, y la próxima vez te dice si algo se ha vuelto más lento.

También mide el disco: cuánto ocupa cada rama de `AlgoData`, qué ficheros están duplicados, y cuánto costaría leer los mismos datos guardados en otro formato.

No es un test. No dice si el código está bien: dice lo que cuesta.

Cuándo lo usas, y cuándo no

Úsalo después de tocar cualquier módulo de análisis, antes de lanzar una corrida larga que tiene que caber en memoria, y cuando quieras saber dónde merece la pena invertir tiempo de programación.

No lo uses para comparar con la medida de otra máquina: los números son de este servidor. Y no lo uses como prueba de que un cambio es correcto — puede ser tres veces más rápido y estar mal.

No hace falta cerrar SQX. Ningún objetivo toca la instalación: todos leen ficheros ya exportados bajo `~/Desktop/AlgoData`. Pero si la GUI está construyendo estrategias, la máquina está ocupada y las medidas saldrán peores; `history.csv` guarda la carga del sistema con cada fila para que eso se vea.

Antes de empezar

Tiene que existir un export de trabajo bajo el raíz de datos. Por defecto el catálogo mide sobre:

qué	dónde	se cambia en
proyecto y activo de la muestra	`Test_XAUUSD_H1_ruptura_wtc2`, XAUUSD (10.000 estrategias construidas el 2026-10-13)	`sample.project`, `sample.asset`
operaciones exportadas y sus `.sqx`	**ninguna**: esta población no tiene carpetas `trades/` ni `strategies/` (sólo el formato nuevo, `trades.parquet`). Hay que elegir una exportación que las tenga	`sample.trades_databank`
batería de retest ya ingerida	`raw/Test_XAUUSD_H1_ruptura_wtc2/MCR_All/2026-10-14/`	`sample.retest_databank`
velas	`XAUUSD_M1`	`sample.bars_feed`
cosecha del criba (paso 8)	`harvest/Test_XAUUSD_H1_ruptura_wtc2/Results/2026-10-14/`	`sample.harvest`, `sample.gate`
hechos del autopiloto	el mismo proyecto, paso 8	`sample.facts`

Siempre usa **el export más reciente** de cada uno. Si no tienes alguno, exporta primero (`01-empezar.pdf › 00-empezar`) o cambia esas claves en `perf/config.yaml`.

Cómo se ejecuta

```
python3 -m perf.catalogue
```

```
$ python3 -m perf.catalogue --only core
midiendo en 779d2e1-dirty
  core.trades_read ...
  core.bars_read ...
  core.sqx_xml ...
  core.sqx_stats ...
target                    ahora     antes   tiempo  memoria  veredicto
core.trades_read          0.17s     0.17s     1.6%    -1.4%  noisy
core.bars_read            0.41s     0.41s     0.5%    -5.3%  noisy
core.sqx_xml              0.03s     0.04s    -0.7%     0.4%  noisy
core.sqx_stats            0.05s     0.05s     0.1%     0.4%  noisy
montecarlo.stream         0.04s     0.04s    -1.0%     1.0%  noisy
montecarlo.analyse       10.10s    10.08s     0.3%    -1.2%  noisy
retest.load_sims          0.09s     0.10s    -6.8%     4.2%  noisy
crossmarket.paired        0.45s     0.44s     2.0%     3.0%  noisy

sin regresiones
```

Una línea por objetivo. `ahora` es el tiempo de esta medida, `antes` el de la anterior, `tiempo` el cambio **por unidad de trabajo** y `memoria` el cambio en el pico de memoria.

flag	obligatorio	qué hace
`--only`	no	mide sólo esos objetivos o esa área (`core` , `strategies` , `montecarlo.analyse`)
`--scaling`	no	vuelve a medir el techo de memoria de la máquina; tarda unos minutos
`--hotspots OBJETIVO`	no	en vez de medir todo, perfila uno y dice dónde se le va el tiempo
`--set CLAVE=VALOR`	no	cambia cualquier valor de `perf/config.yaml` sólo para esta corrida

El comando **sale con código 1 si algo ha empeorado**, así que puede ir en cron sin nadie delante.

El otro comando:

```
python3 -m perf.disk.report      # inventario de AlgoData, presupuestos y qué se puede borrar
```

```
$ python3 -m perf.disk.report
AlgoData: 2.35 GB en 49 ramas

     2153.8 MB    4014 ficheros     1.0 d  raw
     2153.8 MB    4014 ficheros     1.0 d  raw/XAUUSD
     1599.8 MB    2121 ficheros     9.0 d  raw/XAUUSD/WFM_Stability
      310.3 MB    1517 ficheros     1.5 d  raw/XAUUSD/MC_Trades
      107.4 MB     104 ficheros     0.5 d  logs
      107.1 MB      81 ficheros     0.5 d  logs/SQX
      107.0 MB      34 ficheros     0.5 d  logs/SQX/StrategyQuant
       75.4 MB      91 ficheros     2.0 d  raw/XAUUSD/MCR_All
       65.9 MB      21 ficheros     1.0 d  raw/XAUUSD/SPP_IS
       43.5 MB     154 ficheros     6.5 d  raw/XAUUSD/Retest_Markets_-_Family
       43.2 MB      69 ficheros     9.0 d  raw/XAUUSD/WFM
       36.8 MB       4 ficheros    11.5 d  bars

duplicados: 1 grupos, 12.8 MB recuperables
  x2      12.8 MB  raw/XAUUSD/SPP_IS/2026-09-10/bars/bars_M30.csv | raw/XAUUSD/MC_Trades/2026-09-19/bars/bars_M30.csv
```

El presupuesto de disco

Desde el 2026-09-21, `perf.disk.report` **también sale con código 1 si alguna rama de** `AlgoData` **se pasa de su presupuesto**, o si el total se pasa del suyo. Igual que `perf.catalogue` con las regresiones: así puede ir en cron y creerte lo que dice.

```
AlgoData: 2.47 GB de 60 GB (4%) en 55 ramas

           ok      1.96 GB /   40 GB      5%  raw
           ok      0.30 GB /    3 GB     10%  bars
           ok      0.11 GB /    1 GB     11%  logs
           ok      0.06 GB /    5 GB      1%  reports
```

Debajo de la tabla, el informe imprime también cuánto lleva ocupado `/tmp`, que no es parte de `AlgoData` sino una partición propia de 3,9 GB, y **sale con código 1 si pasa del 80 %** (`disk.tmp_max_share`):

```
/tmp: 2.15 GB de 4.13 GB (52%), tope 80%
```

Se llenó al 100 % el 2026-09-26 y a todas las sesiones de Claude les falló el shell. Lo que la llena son las carpetas de sesión de Claude (`/tmp/claude-1000/...`); `bin/weekly-claude-cleanup.sh` borra las de más de 3 días y corre cada día. Lo temporal del propio proyecto va ya a `AlgoData/tmp/` , no a `/tmp` , y ese mismo script poda lo que una corrida caída dejó allí. Si ves `FUERA DE PRESUPUESTO: /tmp` , ejecuta ese script (con `--dry-run` primero si quieres ver qué borraría).

Los techos están en `perf/config.yaml` , en `disk.budget_gb` . Se dimensionaron con 2,3 GB en uso y dejando sitio para el estudio de variantes (~200 MB de operaciones por estrategia madre y mercado, así que el crecimiento va a caer en `raw/`).

Una rama **sin** presupuesto sale marcada `sin presupuesto` , no aprobada. Que aparezca una rama nueva y empiece a engordar es justo lo que esta tabla existe para que veas.

Qué se puede borrar

Y una última sección dice **qué se podría borrar y cuánto liberaría**:

```
candidatos a borrar: 6, 0.47 GB  (nada se borra aquí)
      331.9 MB  strategy_copies    raw/XAUUSD/WFM_Stability/2026-09-10/wfm/strategies
      118.9 MB  strategy_copies    raw/XAUUSD/MC_Trades/2026-09-19/strategies
        0.9 MB  superseded_export  raw/XAUUSD/Retest_Markets_-_Family/2026-09-09
```

Cinco reglas, y **sólo una de ellas es un veredicto**:

regla	qué encuentra	cuánto te puedes fiar
`collected_variants`	databanks que el ledger del pipeline dice que ya se exportaron **y se comprobó el hash**	**veredicto** — está probado que el dato sobrevivió
`superseded_export`	un export fechado cuyo hermano más nuevo contiene todo lo que él tiene	candidato
`strategy_copies`	ficheros `.sqx` dentro de exports, que duplican lo que ya guarda el databank	candidato
`intermediates`	carpetas `raw/` y `trades/` de CSV al lado de un `trades.parquet` que ya las contiene (los exports posteriores al 23-09-2026 las borran solos)	candidato
`stale_branch`	nada escrito en 90 días	la más floja — viejo no es lo mismo que sobrante

⚠ **Nada del proyecto borra ninguna de estas cosas.** La lista sólo le pone un número a una decisión que sigue siendo tuya.

Un detalle que importa: `superseded_export` compara **contenidos**, no fechas. La primera versión comparaba fechas y propuso borrar `raw/XAUUSD/SPP_IS/2026-09-10` porque existe un `2026-09-19` — pero el del 19 sólo tiene `wfc_pairs/`, mientras que el del 10 tiene las tablas de permutaciones que lee todo el estudio SPP. Habría propuesto borrar la única copia de los datos de entrada. Ahora un export sólo cuenta como sustituido si el nuevo lo contiene **entero**.

(`perf/measure/runner.py` también se puede ejecutar, pero es interno: es el subproceso que el arnés arranca para medir un objetivo aislado. No lo llames a mano.)

Qué salidas produce, y qué significan

Todo vive en `~/Desktop/AlgoData/profiling/`:

fichero	qué guarda
`history.csv`	una fila por medida, para siempre. **Sólo crece**: nunca se edita ni se borra
`scaling.csv`	las curvas de escalado de la máquina
`disk.csv`, `duplicates.csv`, `formats.csv`	el inventario del disco
`budgets.csv`	cada rama contra su presupuesto, una fila por corrida
`reclaimable.csv`	los candidatos a borrar, con sus bytes y su motivo

Las columnas de `history.csv`

Columna por columna:

- **segundos** — lo que tardó la medida mediana de las tres repeticiones.

- **microsegundos/unidad** — lo mismo dividido entre el trabajo hecho. **Ésta es la que decide.** Si el export crece de 500 a 3.000 operaciones, los segundos suben y esta columna no: el código no se ha vuelto más lento, hay más datos.
- **trabajo** — cuántas operaciones, filas, velas o ficheros procesó, con su unidad.
- **MB pico** — el pico de memoria residente **del árbol de procesos entero**, incluidos los trabajadores que el objetivo arranque. Es la cifra que decide si una corrida cabe en la máquina.
- **dispersión %** — cuánto se separaron entre sí las repeticiones de esta misma medida. Es el ruido del instrumento.
- **veredicto** — uno de cinco:

veredicto	qué significa
`first`	primera vez que se mide; no hay con qué comparar
`regression`	más del 15 % más caro por unidad, o más del 15 % más de memoria
`improvement`	más del 15 % más barato
`steady`	cambió poco, y ese poco es mayor que el ruido de la medida
`noisy`	cambió menos de lo que el instrumento puede distinguir. **No es "igual": es "no lo sé"**

El techo de la máquina (`scaling.csv`)

Las dos curvas son el mismo cálculo moviendo los mismos bytes. La única diferencia es si el dato cabe en la caché del procesador o hay que ir a buscarlo a la memoria principal.

Que la línea `cache` siga subiendo hasta 96 procesos y la `dram` se quede plana desde 16 significa que **los núcleos que sobran no están calculando: están esperando memoria**. La consecuencia práctica: en un módulo cuyos datos no caben en caché, pedir más procesos no lo hace más rápido — sólo gasta más RAM.

El disco

Las ramas más grandes son sólo las hojas: una carpeta que contiene a otra de la lista no aparece, porque su barra sería la suma de sus hijas otra vez. Los **duplicados son candidatos, no una sentencia**: dos ficheros que coinciden en tamaño y en 64 KB de cada extremo son casi seguro el mismo fichero, pero nada aquí lo confirma y **nada aquí borra nada**.

Dónde se le va el tiempo a un módulo

```
python3 -m perf.catalogue --hotspots core.sqx_stats
```

```
$ python3 -m perf.catalogue --hotspots core.sqx_stats
core.sqx_stats — 0.07 s profiled

TIME (own)
   56.2%     0.039 s        100x  sqxstats.py:27(records)
    6.2%     0.004 s     19,020x  ~:0()
    4.2%     0.003 s     15,762x  ~:0()
    4.0%     0.003 s      6,248x  pathlib.py:675(__lt__)
    2.9%     0.002 s     12,496x  pathlib.py:654(_cparts)
    2.5%     0.002 s         20x  zipfile.py:1347(_RealGetContents)
    2.4%     0.002 s        140x  ~:0()
    1.8%     0.001 s         20x  ~:0()
    1.7%     0.001 s          1x  tracemalloc.py:477(_group_by)
    1.7%     0.001 s     11,620x  ~:0()
    1.5%     0.001 s         21x  ~:0()
    1.4%     0.001 s      3,780x  ~:0()

MEMORY (held at peak)
  /usr/lib/python3.10/pathlib.py:969  0.0 MB
  /usr/lib/python3.10/zipfile.py:1270  0.0 MB
  /home/sergioguslw/Desktop/AlgoProject/core/sqxstats.py:48  0.0 MB
```

Arriba, las funciones ordenadas por **tiempo propio** — el gastado dentro de su propio cuerpo, sin contar a quién llaman. Abajo, las líneas que tenían más bytes vivos en el momento de mayor consumo.

Dos avisos:

- Los segundos de un perfilado son más largos que los reales, porque perfilar cuesta. **Lee el orden, no los segundos.**
- La lista de memoria sólo ve lo que reservó Python en ese proceso. Los buffers de numpy dentro de un trabajador no salen ahí: para eso está la columna `MB pico`, que muestrea el árbol entero.

Qué se puede tocar

Todo está en `perf/config.yaml`. Lo que de verdad cambia el resultado:

clave	qué hace
`harness.repeats`	repeticiones por medida. Menos repeticiones, más ruido y más veredictos `noisy`
`regression.wall_pct`	a partir de qué porcentaje una medida se llama regresión
`sample.n_sims`	el presupuesto de simulación del objetivo de Monte Carlo
`sample.files`	cuántos ficheros muerden los objetivos de parseo
`disk.stale_days`	a partir de cuántos días sin tocar una rama se marca como obsoleta

Cambiar `sample.*` rompe la comparación con las medidas anteriores, porque ya no es el mismo trabajo. Si lo cambias, la historia previa de ese objetivo deja de valer.

Las trampas que este módulo existe para evitar

- **`ru_maxrss` miente cuando hay trabajadores.** De los procesos hijos informa del más grande, no de la suma. Medido el 2026-09-20: `montecarlo.analyse` da 535 MB por `ru_maxrss` y **2.340 MB** muestreando el árbol. La segunda es la buena.
- **El muestreo cada 50 ms puede perderse un pico más corto que eso.** La cifra es un suelo.
- **Una medida con la máquina ocupada no vale.** Por eso cada fila guarda la carga del sistema y la memoria libre que había.

- **Un commit `-dirty` no se puede reproducir.** `history.csv` lo guarda tal cual para que se vea.

Medido otra vez el 2026-10-14 — el catálogo entero sobre XAUUSD H1

Una pasada, con la máquina en reposo y el commit `2534971` con cambios sin guardar. Cada objetivo se midió tres veces y se guarda la mediana. **Este es el punto de partida nuevo**: el historial anterior se apartó el 2026-10-13 porque cambió la forma de medir la memoria.

Cómo leer las columnas. *Segundos* es el tiempo real de reloj. *CPU* es el tiempo de trabajo sumado de todos los núcleos que se usaron: si CPU es mucho mayor que los segundos, el objetivo reparte el trabajo entre muchos núcleos a la vez. *Memoria pico* es lo que ocupa en la RAM el proceso y todos sus hijos, contando una vez lo que comparten (se llama PSS). *Dispersión* es lo que se separaron las tres repeticiones: un número alto avisa de que esa medida es poco fiable.

objetivo	trabajo	segundos	CPU (s)	memoria pico (MB)	dispersión
`retest.load_sims`	2.098 filas	0,06	0,2	214	28 %
`feedquality.detect`	7,97 M velas	2,9	14,5	2.024	4 %
`autopilot.facts`	526.256 filas	6,5	6,9	1.856	1 %
`gate.stage`	3 estrategias	0,6	0,7	285	1 %
`gate.level0`	3 estrategias	0,6	0,7	279	1 %
`gate.level1a`	3 estrategias	0,6	0,7	301	3 %
`gate.level1b`	3 estrategias	0,7	0,8	332	1 %
`gate.level2`	3 estrategias	0,7	0,9	299	2 %
`gate.level3ab`	3 estrategias	3,3	11,9	378	1 %
`gate.monkey`	3 estrategias	2,4	13,9	460	1 %
`gate.kept`	3 estrategias	0,07	0,07	212	7 %
`falsepositives.donor`	32.900 operaciones	6,1	41,5	649	2 %
`profilepower.replication`	15.622 velas	5,3	5,7	1.408	6 %
`pooledprofile.asset`	15.622 velas	2,3	72,9	1.462	2 %
`portfolio.pairs_table`	124.750 pares	7,9	311	891	2 %
`portfolio.rank_search`	30.000 combinaciones	20,0	1.266	210	1 %
`portfolio.stage_b`	110.000 trayectorias	1,9	34,5	582	2 %
`portfolio.verdict_null`	10.000 sorteos	9,5	38,2	357	1 %
`portfolio.funded_bank`	2.610 configuraciones	12,9	92,3	1.052	1 %
`portfolio.weights_wf`	7 brazos	0,3	0,3	247	4 %
`core.bars_read`	7,97 M velas	0,7	1,1	1.436	25 %
`core.ticks_read`	369,7 M ticks	6,6	24,3	4.401	113 %

Tres cosas que esta tabla **no** dice, y conviene saber:

- **Los objetivos del criba (`gate.*`) sólo vieron 3 estrategias.** La cosecha de este proyecto guarda las 3 que quedaron tras el tope de la prueba funcional, no las 10.000. Sus segundos no sirven para planificar una población grande: dividen entre 3 y se extrapolan mal. Para eso siguen valiendo las medidas con 500 estrategias de más abajo, con su fecha.

- **`core.ticks_read` tiene 113 % de dispersión**: una repetición tardó mucho más que otra. Esa cifra es orientativa.
- **La carga del sistema de la columna `load1` de `history.csv` incluye a los propios hijos del objetivo.** Los objetivos de cartera (`portfolio.*`) usan todos los núcleos, así que verás cargas de 20 a 48 en sus filas sin que nada más estuviera corriendo.

Todos los objetivos que corrieron salieron `first` (no había medida anterior) y sin regresiones.

Lo que no se pudo medir el 2026-10-14

objetivo	por qué falló
`montecarlo.stream` , `montecarlo.analyse` , `montecarlo.analyse_long` , `core.trades_read` , `core.sqx_xml` , `core.sqx_stats`	necesitan `sample.trades_databank` , y esta población no tiene carpetas `trades/` ni `strategies/`
`snooping.superior`	busca el último día de la cosecha y encuentra la carpeta `_aparte` , que no tiene `equity.parquet` (error en `perf/inputs/workloads.py` línea 114)
`portfolio.universe_member`	apunta a velas `USDJPY_DukasM1_the5ers` que ya no existen
`portfolio.close_curve`	apunta a la cosecha `Test_Calib_USDJPY_H1_momentumL` , borrada (`perf/inputs/construct.py` línea 21)

Por eso la pasada entera de un solo comando (`python3 -m perf.catalogue`) **se para en el primer fallo** y no guarda nada: las cifras de arriba salieron de lanzar el mismo comando objetivo por objetivo con `--only` . Mientras esos siete no se arreglen, para tener la pasada entera hay que lanzarla así.

Lo que tardó el workflow entero en la prueba funcional del 2026-10-13/14

Esta es la prueba funcional del workflow de punta a punta: proyecto `Test_XAUUSD_H1_ruptura_wtc2` , XAUUSD H1, **10.000 estrategias** construidas, y tres madres (las estrategias que sobreviven y siguen) tras el paso 8. Fueron **79 minutos y medio** (4.764 s). SQX calculando suman 42,7 minutos; todo lo demás (Python, exportar, sincronizar, abrir y cerrar SQX) suma 36,7 minutos, el 46 %.

Los segundos son de reloj. *sqx* es SQX calculando, desde que se le da la orden hasta que dice «Project finished». *export* es sacar los datos de SQX. *sync* es que SQX recargue sus ficheros. *resto* es todo lo demás.

paso	total (s)	sqx	export	sync	resto
arranque	26	0	0	0	26
6 + 7 construir y retest fuera de muestra (SQX)	2.094	1.869	134	89	2
8 criba (Python)	481	0	0	0	481
8 juicio	67	0	0	0	67
9 otros mercados, familia (SQX)	10	4	3	0	2
10 otros mercados (Python)	9	0	0	0	9
11 CrossTF (SQX)	11	2	7	0	1
11.5 CrossTF (Python)	5	0	0	0	5
11.7 CrossTF M30 y H4 build (SQX)	9	1	6	0	1
12 otras temporalidades (Python)	7	0	0	0	7
13 vecinos	81	0	0	0	81
13 MC Retest (SQX)	17	7	8	0	2
14 MC Retest (Python)	3	0	0	0	3
15 SPP (SQX)	167	78	12	2	75
16 SPP (Python)	7	0	0	0	7
16.5 variantes	601	105	103	159	234
17 WFC a ciegas	5	0	0	0	5
18 CSCV a ciegas	26	0	0	0	26
19 WFM (SQX)	694	495	29	1	168
19 lectura de WFM	6	0	0	0	6
19.5 lectura en oos2	5	0	0	0	5
20 análisis conjunto	5	0	0	0	5
21 lectura	1	0	0	0	1
26 verificación en MT5	302	0	0	0	302
parada	125	0	15	0	110
TOTAL	**4.764**	**2.562**	**317**	**252**	**1.632**

Los pasos de menos de un segundo (juicios sin criterio, 18.5, guardado para fondeo, libro mayor) están en el fichero original y aquí se omiten. Las fases más largas:

fase	qué es	segundos
construcción + retest (6 + 7)	SQX construyendo 10.000 estrategias	1.869
WFM (19)	SQX	495
falsos positivos (8)	Python	413
calidad del feed (8)	Python	174
reapertura de SQX tras el WFM (19)	SQX volviendo a abrir el proyecto	167
criba por niveles (8)	Python	151

Dos lecturas: lo que manda es la construcción (casi la mitad del total), y el paso 8 en Python cuesta 8 minutos con 10.000 estrategias porque se hace **antes** de quedarse con tres. Fuente: `AlgoData/autopilot/Test_XAUUSD_H1_ruptura_wtc2/20261013-231422/` (`tiempos.md` y `resumen.md`).

> **Aviso sobre lo que sigue.** Todas las tablas de abajo son medidas más antiguas (2026-09-24 a 2026-09-29), de otras poblaciones y otros tamaños, y **no se han repetido el 2026-10-14**. Valen como orden de magnitud con su fecha, no como el estado de hoy. Las memorias de esas secciones se midieron con la memoria residente (RSS), no con la PSS de ahora.

Lo que cuesta el workflow de punta a punta — medido 2026-09-24

Esto **no** sale de `perf.catalogue` y no está en `history.csv`, a propósito: el catálogo compara por unidad de trabajo y no toca SQX nunca, y esto es reloj de pared de una cadena que sí lo toca. Es una referencia de planificación, no una serie temporal.

La corrida medida: USDJPY H1, plantilla `emaCloseAbove`, proyecto `TestUSDJPY_Workflow_v1` en el custodio (48 núcleos, 80 g de heap), embudo 50 → 17 → 12 → 8 → 4. Los tiempos escalan con la población, así que están anotados con el tamaño al que se midieron.

En SQX — sacado del propio log (`Task finished in`)

paso	tarea	población	tiempo
6	CONSTRUCCION (build, 2008–2017)	→ 50 estrategias	**16 s**
7	OOS (retest `oos1` 2018–2022)	50	**5,4 s**
9	Retest Markets - Family (9 pares)	17 × 9	**84 s**
11	crossTF H1+H4	24 celdas	**21 s**
13	MCR 1 Bar	8 × 1.000 sims	**24 s**
13	MCR 5 Params	8 × 1.000	**16 s**
13	MCR 6 Exits	8 × 1.000	**198 s**
13	MCR 7 OHLC	8 × 1.000	**695 s**
13	MCR 8 Stress (3 perturbaciones, 15 años)	8 × 1.000	**1.022 s**
15	SPP IS	4 madres	**79 s**
15	SPP OOS	4 madres	**42 s**

El MC Retest es el 89 % del tiempo de SQX de toda la cadena: 1.955 s de 2.202. Y eso con CINCO tareas; con las siete que ahora se pueden configurar será más. Dentro del MC Retest, `OHLC` y `Stress` son el 88 %: perturbar el histórico y correr las tres perturbaciones juntas sobre 15 años.

En Python — `/usr/bin/time`, un núcleo salvo donde se diga

paso	comando	población	tiempo	RSS pico
8	`studies.screening.gate.harvest`	50 + 17	**58 s**	1,6 GB
8	`studies.screening.gate.report` (8 cribas + 17 monos a 2.000 sorteos)	17	**2,4 s**	380 MB
10	`crossmarket.report` a 2.000 sorteos	8 × 9	**~13 min**	—
10	`crossmarket.report` a 10.000 sorteos (el de `config.yaml`)	8 × 9	**>50 min, no terminó**	—
10.5	`variants.scale`	12 madres → 12 hermanas	**0,7 s**	125 MB
12	`crossTF.report`	24 celdas	**29 s**	580 MB
14	`retest.ingest`	36 corridas, 35.972 sims	**10 s**	**3,2 GB**
14	`retest.report`	4 estrategias	**3,4 s**	1,1 GB
16	`sppUltra.report`	4 perfiles, 49.226 filas	**17 s**	910 MB
16.5	`variants.make`	1 madre, 60 variantes	**1,3 s**	300 MB

Exportaciones, que arrancan y paran el conductor por su cuenta:

comando	población	tiempo	RSS pico
`export_metrics`	8 estrategias	**15 s**	20 MB
`export_spp`	4 perfiles	**15 s**	1,4 GB
`export_retest`	8 × 9 mercados, 66 k operaciones	**17 s**	1,9 GB

El impuesto que no aparece en ninguna tabla

Cada etapa exige parar el custodio para reescribir el `project.cfx` (regla dura 4) y arrancarlo otra vez. Medido:

	tiempo
`sqx-worker.sh start` (el script vuelve)	2,6 s
hasta que la CLI responde	**21,5 s**
`sqx-worker.sh stop` (con su sincronización de cierre)	14,7 s
ciclo completo	**~39 s**

En esta corrida se pagó **ocho veces: unos 5 minutos** sólo en abrir y cerrar. No es evitable hoy: `startOnlyTask` no corre nada en este install, así que una etapa por arranque es la única forma de parar entre pasos para cribar.

Los dos números que hay que tener en la cabeza

1. **`crossmarket.report` es el cuello de botella de Python**, y por dos órdenes de magnitud: 13 minutos frente a segundos de todo lo demás. Un núcleo, 9 mercados × 10.000 sorteos por estrategia. Con una población de 100 en vez de 8 son horas. Es el primer candidato a paralelizar.
2. **`retest.ingest` pica 3,2 GB con 36 corridas.** Escala con corridas × simulaciones, así que una población de 100 por las ocho tareas (800 corridas) pediría del orden de 70 GB si la proporción se mantiene. Antes de correr eso, medirlo.

Medido otra vez el 2026-09-29: 21 estrategias, todo lanzado desde la ventana

USDJPY M30, plantilla `donchianUpperCrossUp`, proyecto `Test_USDJPY_donchianUpperCrossUp_M30` en el custodio (96 hilos, 80 g de heap). Cada paso con su «▶ SQX» de la raíl, así que el tiempo del trabajo incluye lo que la ventana hace alrededor: la copia del proyecto, arrancar, esperar a que SQX cargue sus databanks (~50 s), parar y el export automático.

paso	tarea	población	tarea en SQX	por estrategia
7	OOS	21	**6 s**	0,05 s
13	MCR 1 Bar	21 × 1.000 sims	**67 s**	2 s
13	MCR 2 Spread	21	**7 min 34 s**	19 s
13	MCR 3 Slippage	21	**7 min 27 s**	19 s
13	MCR 5 Params	21	**63 s**	2 s
13	MCR 6 Exits	21	**7 min 29 s**	19 s
13	MCR 7 OHLC	21	**28 min 12 s**	73 s
13	MCR 8 Stress	21	**42 min 48 s**	110 s
15	SPP IS	21	**10 min 14 s**	26 s
15	SPP OOS	21	**5 min 20 s**	13 s
19	WFM (30 celdas, exacta, 5.000 tests por paso)	21	**58 min 42 s**	150 s

trabajo completo de la ventana	reloj	de ello, fuera de SQX
paso 13 (las 7 MCR)	**1 h 41 min**	~5 min (arranque, carga, parada y export)
paso 15 (SPP IS + OOS)	**25 min**	~9 min (el export reexportaba todo; ya no)
paso 19 (WFM)	**1 h 07 min**	~8 min (export de la WFM: 918.194 operaciones)

La suma de SQX para 21 estrategias, del 13 al 19, es de unas 3 h 15 min, y el MC Retest es la mitad: OHLC y Stress solos son 71 minutos. Con 200 estrategias el MC Retest pasaba de 10 horas (se midió MCR 2 Spread a 21 s por estrategia): escala casi en línea recta con la población.

Del 16.5 al 20, cinco madres (una con 4.514 variantes y cuatro con 500, `--sample 500`):

paso	qué	tiempo
16.5	`make` (fabricar)	2-3 s por madre
16.5	`execute`, 4.514 variantes: retest de las tres patas WFC	**35 min**
16.5	`execute`, 4.514 variantes: SQX escribiendo a disco (un solo hilo)	**30 min**
16.5	`execute`, 500 variantes, todo	**17-21 min** (retest ~4 min; el resto, cargar los 9 mercados, escribir y exportar)
16.5	`equity` + `collect` + vaciar	20 s (500) · 1,5 min (4.514)
17 / 18 / 18.5	WFC, CSCV y superficies, 6 madres	segundos cada uno
19	WFM limpia, 21 estrategias	**58-67 min**
20	análisis conjunto, 5 madres completas	2 s

Con 4.514 variantes una madre cuesta más de una hora y media de custodio; con 500, un tercio. Para desarrollo, `--sample 500`.

Los pasos de Python, sobre las mismas 21, tardan segundos: ingesta del MC Retest 17 s (539 corridas, 0,5 GB), análisis del MC Retest 6 s, SPP 8 s, WFM 20 s, análisis conjunto 1 s, exposición 5 s, edge por coste unos segundos.

Lo que NO está medido

Los pasos 17 (WFC), 18 (CSCV) y 19 (WFM) no se han corrido, porque gastan `oos2`. Tampoco el retest del lote de variantes del 16.5, que es el que los alimenta y el que se prevé más caro de todos: 240 variantes × 3 tramos × 9 mercados adicionales. Cualquier presupuesto de la cadena completa que salga de esta página está incompleto por ese lado, y no de poco.

Lo que cuesta NUESTRO Python, con 500 estrategias — medido 2026-09-24

La sección anterior mide la cadena entera con una población de juguete. Esta mide **sólo la capa que hemos escrito nosotros**, con 500 estrategias construidas y retesteadas a propósito para esto (proyecto `PerfUSDJPY_Python_v1`, USDJPY H1), y mide **escalado**, no un punto suelto.

Máquina: 96 núcleos, 125 GB. **Todo lo que sigue usa UN núcleo.**

La unidad correcta no es la estrategia, es la operación

Medido: las 8 estrategias de una misma corrida llevan entre **5.338 y 43.109 operaciones** en los nueve mercados ajenos — un factor de 8. Un «segundos por estrategia» sobre esa población no dice nada. Por eso todo lo de abajo está normalizado por operación.

`gate.*` — el paso 8, cribar la población

N estrategias	`studies.screening.gate.report`	RSS
25	3,5 s	505 MB
50	5,2 s	545 MB
100	8,2 s	636 MB
250	15,9 s	819 MB
500	**32,3 s**	**1,32 GB**

Lineal y barato: unos **60 ms por estrategia** de coste marginal, sobre un fijo de ~2 s. Incluye las 8 cribas y el mono de cada superviviente a 2.000 sorteos. 500 estrategias se criban en medio minuto.

`studies.screening.gate.harvest`, que es la mitad que conduce SQX: **112 s y 4,56 GB** con 1.000 ficheros (500+500), contra 58 s y 1,6 GB con 67. El tiempo lo domina el arranque de la JVM; **la memoria sí crece con la población** y es el número a vigilar.

El gate por niveles y el guardado para fondeo — medido 2026-10-07

Desde el 2026-10-07 el paso 8 corre por niveles (0, 1a, 1b, 2, 3ab, 3c: el mono) y el catálogo tiene un objetivo por nivel (`python3 -m perf.catalogue --only gate`). La tabla de arriba describe la puerta vieja: no se compara con esto.

Muestra: las 300 primeras estrategias de la cosecha `Test_USA500_H1_indicesIBS/Results/2026-10-05` (3.000, H1, 113 operaciones de mediana), con la cascada apagada (todas pasan por todos los niveles). Es `sample.gate` en `perf/config.yaml`: cambiarla rompe la comparación con las medidas anteriores. La máquina tenía carga 15-35 de otras sesiones durante la medida: léelo como orden de magnitud.

Cada objetivo prepara la muestra y corre el nivel 1a (que escribe la tabla por operación que leen los demás), y encima su nivel: **el coste de un nivel es su fila menos la de `gate.level1a`**.

objetivo	pared (300 estr.)	por estrategia sobre 1a	qué es
`gate.stage`	6,5 s	—	leer la cosecha y las barras: lo fijo de todos
`gate.level0`	6,5 s	~0	presencia, sanidad, estáticas
`gate.level1a`	6,9 s	1,5 ms	R3: exceso por operación y su t
`gate.level1b`	7,7 s	2,5 ms	los tres módulos del 1b (centradas, una pasada, calidad de datos)
`gate.level2`	7,8 s	3 ms	repreciado con una barra de retraso
`gate.level3ab`	11,2 s	14 ms	remuestreos contra el build
`gate.monkey`	10,3 s	~11 ms de pared, **0,23 s de núcleo**	el mono a 5.000 sorteos × 4 escalones, sobre todos los núcleos

`cpu_s` del catálogo cuenta desde el 2026-10-07 todo el árbol de procesos, hijos incluidos (dueño: «todos los recursos computacionales»). Las filas medidas antes de esa fecha sólo contaban el padre: la del mono enseña 17 s de CPU donde el trabajo real son ~70 s de núcleo (300 × 0,23), y se corrige sola la próxima vez que se mida el catálogo. Con la cascada encendida casi nadie llega al 1b: las 3.000 de USA500 se criban en ~1 s (140 s apagada).

- `gate.kept`: guardar para fondeo (`pipeline/autopilot/kept.py`, el cruce del scorecard y el corte de las curvas oos1) sobre las **3.000** enteras: **2,5 s pared, 2,2 GB**. Medido sin escribir: `kept.save` no se llama porque escribiría en `kept/` de la raíz de datos.
- `falsepositives.donor`: un donante, 50 monos de su huella, por la **cadena entera** con la cascada apagada y el mono a 5.000: **8,3 s pared**, 12.650 operaciones. La población de prueba entera (`per_level` 5.000) son 100 donantes por nivel de ventaja: del orden de 15 minutos por nivel en esta máquina vacía; no se lanza desde el catálogo.

Dónde se le va el tiempo al gate (cProfile, N=500)

	s	%
total	33,4	100
`monkey.mono`	**29,5**	**88**
↳ `simulate.nulls`	15,1	45
↳ `simulate.fixed`	9,9	30
↳ ↳ **`calibrate.atr`**	**9,2**	**28**
comparación de columnas `object`	4,2	13

Dos cosas concretas, y las dos son trabajo repetido, no trabajo necesario:

1. **El ATR se recalcula una vez por estrategia sobre las MISMAS barras.**
   `engines/nulls/simulate.py:fixed()` llama a `calibrate.atr(frame, …)` y `frame` es idéntico en las 500 llamadas: 234 llamadas × 39 ms = **9,2 s de los 33**, y crece lineal con la población haciendo siempre la misma

cuenta. Calcularlo una vez por (barras, periodo) lo deja en 39 ms totales.

2. □ **Se filtra la tabla entera de operaciones por identidad, una vez por estrategia** (`studies/screening/gate/monkey.py:31`, `oos[oos["identity"] == name]`), y la columna es de tipo `object`: 4,2 s en 236 comparaciones de cadenas. Un `groupby` una sola vez lo elimina.

Juntas son **~40 % del paso 8** y ninguna cambia un número: es la misma cuenta hecha una vez.

`crossmarket.report` — el paso 10, y el cuello de botella real

Tres medidas limpias, con SQX parado y a 500 sorteos:

N	operaciones	tiempo	ms/operación	RSS
2	44.660	170,6 s	**3,82**	6,0 GB
4	133.240	507,4 s	**3,81**	6,8 GB
8	158.415	630,1 s	**3,98**	6,8 GB

La constante aguanta: 3,8–4,0 ms por operación. Y el coste crece con los sorteos, medido sobre una población fija de 122.045 operaciones: 4,19 ms/op a 500 sorteos y 5,03 a 1.000. Ajustando:

> **coste ≈ (3,36 + 0,00168 × sorteos) ms por operación, en un núcleo**

Lo que significa para el export completo de las 500 estrategias, que lleva **12.010.976 operaciones** en diez mercados:

sorteos	un núcleo	con 90 núcleos
500	**14 h**	9 min
2.000	**22 h**	15 min
10.000 (el de su `config.yaml`)	**67 h**	**45 min**

⚠ El ajuste de los sorteos sale de dos puntos medidos **con SQX corriendo a la vez**, así que las tres cifras de la derecha son el orden de magnitud, no una promesa.

Dónde se le va el tiempo (cProfile, 8 × 9 mercados, 500 sorteos)

	tottime	cumtime
`simulate/metrics.py:paths`	64,8 s	**184,1 s**
↳ `_losing_run`	65,8 s	—
`verdict/stress.py:degraded`	65,4 s	65,5 s
`np.add.at` (`mechanics/equity.py:42`)	52,6 s	—
`model/holdfit.py:fit`	38,7 s	—
`cumsum` · `ufunc.reduce` (1,78 M llamadas)	28,5 · 28,1 s	—

□ **`np.add.at` → `np.bincount` medido, y NO es el premio que parece**: 1,6x a 2,5x según el tamaño, no el 10x de la sabiduría popular. Verificado dando el mismo resultado.

□ **`_losing_run` ya está vectorizado por caminos** — el bucle recorre operaciones, no caminos, y cada paso es una operación vectorial. No es código ingenuo: es el coste inherente del barrido.

Conclusión: aquí no hay una micro-optimización que salve el día. El coste es `operaciones × sorteos × mercados × modelos` y está donde tiene que estar. Lo que sobra es que **corre en 1 de 96 núcleos**, y el bucle exterior sobre estrategias es independiente por construcción. Paralelizarlo es un cambio que no toca ni una fórmula y convierte 67 horas en 45 minutos. **Es la única optimización que importa de todo el proyecto.**

Los demás pasos de Python, a la escala a la que se pudieron medir

paso	comando	población	tiempo	RSS
10.5	`variants.scale`	12 → 12	0,7 s	125 MB
12	`crossTF.report`	24 celdas	29 s	580 MB
14	`retest.ingest`	36 corridas, 36 k sims	10 s	**3,2 GB**
14	`retest.report`	4	3,4 s	1,1 GB
16	`sppUltra.report`	4 perfiles, 49 k filas	17 s	910 MB
16.5	`variants.make`	1 madre, 60	1,3 s	300 MB

⚠ **Estos seis NO están medidos a 500**, y no por pereza: cada uno necesita trabajo de SQX proporcional a la población que no cabe en una sesión. El MC Retest son ~1.955 s por cada 8 estrategias, o sea **unas 34 horas para 500**; el SPP son ~20 s por madre, **2,8 h para 500**. Lo que sí se puede afirmar de ellos es la forma: `retest.ingest` escala con corridas × simulaciones y ya pica 3,2 GB con 36 corridas, así que 500 estrategias por 8 tareas (4.000 corridas) es el número que hay que medir antes de lanzarlo, no después.

Exportaciones, que son parte del coste de Python

comando	población	tiempo	RSS	escrito
`export_metrics`	8	15 s	20 MB	—
`export_retest`	8 × 9	17 s	1,9 GB	—
`export_retest`	**500 × 9**	**154 s**	**6,3 GB**	**365 MB**

Y un regalo de la medición: SQX es MÁS eficiente con lotes grandes

tarea	población	tiempo	por estrategia
build	50	16 s	0,32 s
build	**500**	**30 s**	**0,06 s**
retest OOS	50	5,4 s	0,11 s
retest OOS	**500**	**13,9 s**	**0,03 s**
crossmarket 9 mercados	17	84 s	4,9 s
crossmarket 9 mercados	**500**	**293 s**	**0,59 s**

Extrapolar linealmente desde una población pequeña **sobreestima SQX entre 4x y 8x**: paraleliza sobre los 96 núcleos y el coste fijo por tarea se diluye. Lo contrario que nuestro Python.

Lo mismo, ya paralelizado — medido 2026-09-25

Todo lo de la sección anterior corría en **un núcleo de 96**. Esta sección es la misma medida después de repartir el trabajo, sobre la misma población y los mismos ficheros. **Ninguna cifra del análisis cambia**: cada optimización se verificó comparando el resultado contra el de antes.

Qué se cambió, y por qué no cambia ningún número

cambio	dónde	por qué es exacto
el ATR se calcula una vez por fichero de barras, no una por estrategia	`engines/market/calibrate.py`	depende sólo de `(barras, ventana)`; se cachea el mismo array
la tabla OOS se agrupa una vez por identidad, no se filtra una vez por estrategia	`studies/screening/gate/monkey.py`	`groupby` y la máscara booleana devuelven las mismas filas en el mismo orden
`stats.measure()` construye sólo la estadística pedida, y la puerta pide la única que lee	`engines/nulls/stats.py`, `studies/screening/gate/monkey.py`	el filtro estaba **después** del cálculo. `studies.readings.monkey.report`, que lee las cinco, cuesta lo mismo que antes
el bucle sobre estrategias se reparte entre los núcleos	`studies/screening/gate/monkey.py`, `studies/transfer/crossmarket/report.py`	las estrategias no comparten estado ni escriben nada
el estrés de ejecución se trocea en lotes de 500 corridas	`studies/transfer/crossmarket/simulate/stress.py`	los tres sorteos se siguen tomando enteros y en el mismo orden; sólo el precio va por lotes

Los cuatro usan `fork`: el padre lee el export entero y las barras **una vez** y los procesos hijos los heredan sin copiarlos. Mandárselos por `pickle` costaría más que el cálculo.

Paso 8 — `studies.screening.gate.report`, 500 estrategias

	antes	después
tiempo	**31,1 s**	**5,2 s**
RSS	1,30 GB	1,17 GB
CPU	143 %	2.437 %

5,9x. Verificado: el `scorecard.parquet` de 500 filas × 29 columnas sale **idéntico**, columna a columna, y sobreviven las mismas 229 estrategias.

El techo no es el reparto: de los 5,2 s, unos 2,5 s son leer la cosecha y correr las otras siete cribas, que ya eran baratas, y 0,48 s la maquinaria de reparto. El mono en serie baja de **29,5 s a 8,54 s** en tres pasos: el ATR cacheado y el `groupby` lo dejan en 15,7 s, y pedir una estadística en vez de cinco lo baja a 8,54 s.

Dentro del mono, y la dispersión que la media esconde

Reparto del mono ya optimizado, agregado sobre las 234 estrategias que llegan a él:

fase	%
precio de cada operación (`barrier.pnl`)	29,1
numpy suelto: `repeat` / `tile` , `reduce` , `std` , `mean`	28,9
sortear las entradas aleatorias (`model._place`)	27,7
barrido de barreras	13,2
reconciliar · fontanería · estadísticas · rejilla · p	1,1

 Las estadísticas eran el 46 % de este paso y cuatro de las cinco se tiraban. `stats.measure()` calculaba `net, sharpe, pf, retdd, dd` para las 2.000 corridas de cada estrategia y la puerta lee `sharpe` . Medido sobre 2.000×570: **22,45 ms las cinco, 1,75 ms sólo** `sharpe` — y quien pide las cinco no paga nada por el cambio.

Y una por una, con 2.000 sorteos cada una:

	mín	p25	mediana	p75	máx	media
ms por estrategia	3,2	14,0	29,2	48,0	**268,3**	36,5
operaciones OOS	39	497	879	1.289	6.192	1.001
µs por operación	25,3	29,4	34,2	40,5	84,1	**36,0**

⚠ **La más cara cuesta 83,4x la más barata.** Presupuesta por operación, no por estrategia: la correlación entre tiempo y operaciones es **r = 0,982**.

Paso 10 — `crossmarket.report`

Con 8 estrategias × 9 mercados a 500 sorteos (158.415 operaciones), que es la medida que la sección anterior dejó hecha en un núcleo:

	antes	después
tiempo	**623,5 s**	**156,1 s**
CPU	102 %	375 %

4,0x con sólo 7 procesos, porque siete estrategias en siete procesos duran lo que la más larga de las siete — y ese lote va de 50 a 44.771 operaciones. `verdict.csv` sale **idéntico byte a byte**.

Y la población entera, que es el número que importa: 499 estrategias × 9 mercados a 500 sorteos, 72 procesos, `2.285 s` **— 38,1 minutos.** Contra 12,8 h en un núcleo.

 El coste por operación es una constante de verdad, comprobado con un control. Ocho estrategias del medio de la población, con casi las mismas operaciones que las ocho primeras, en un solo proceso:

lote	operaciones	segundos, 1 proceso	ms/operación
las 8 primeras	158.415	623,50	3,94
8 del medio	157.571	593,94	3,77

De ahí el «antes» de las 499: 12.010.976 × 3,85 ms = 46.278 s ≈ **12,8 h**.

El reparto del paso 10 se mueve con el tamaño de la estrategia

Perfiladas tres del mismo lote, con un factor de 8 en operaciones. Dar una sola habría engañado:

fase	5.563 ops	8.988 ops	44.771 ops
numpy suelto (percentiles, `reduce`, `cumsum`, `add.at`)	54,0 %	46,8 %	31,6 %
estadísticas acumuladas de cada camino	17,9 %	20,9 %	**33,3 %**
sortear: los 4 modelos de colocación	9,4 %	10,2 %	13,9 %
estrés de ejecución	6,6 %	7,5 %	9,7 %
test pareado contra estar largo	4,9 %	6,4 %	1,4 %
preparar el mercado y reconciliar el fill	3,0 %	3,4 %	4,4 %
todo lo demás	4,1 %	3,9 %	2,7 %
ms por operación	5,09	4,46	3,64

Abierto por función, **nada pasa del 11 %** (`_losing_run` 11,1 %, `np.partition` 8,6 %, `metrics.paths` 6,7 %, `np.add.at` 6,4 %, `stress.degraded` 6,2 %, `holdfit.fit` 6,1 %). Aquí no hay bala de plata: el premio fue el reparto.

Lo que cuesta la maquinaria de paralelizar

Padre de 1,02 GB, 234 tareas, tarea vacía para medir sólo la fontanería:

procesos	crear el pool (`fork`)	repartir y recoger 234 tareas
16	0,00 s	0,11 s
48	0,00 s	0,26 s
96	0,00 s	0,48 s

`fork` es instantáneo porque no copia, y cada resultado del paso 8 son **45 bytes**. La comunicación no es el cuello de botella, precisamente porque no se manda nada grande.

La memoria era el límite de verdad, y estaba en un sitio concreto

 `stress.simulate` construía la matriz entera de **25.000 corridas × operaciones del mercado** de una vez, y tres de los arrays eran `float64` donde los valores son booleanos:

operaciones del mercado	pico antes	pico después
684	**816 MB**	**140 MB**
607	724 MB	137 MB
563	672 MB	136 MB

⚠ **Esto no era un lujo.** El primer intento de correr las 500 estrategias con 96 procesos llegó a **94,5 GB de los 125** y hubo que abortarlo; un segundo intento con 48 procesos llegó a **89 GB** y el núcleo mató la ventana de VSCode. Con el troceado, **72 procesos ocupan 22 GB**.

> **La regla:** `--workers` es el mando que cambia RAM por reloj. Cuenta **~0,35 GB por proceso** más lo que ocupe el export en el padre, y deja margen: la máquina también está siendo usada.

Cuántos procesos conviene usar — y no son 96

96 estrategias (2,43 M operaciones), 500 sorteos, el mismo trabajo a cinco tamaños de reparto. La referencia de un núcleo son 9.370 s por la constante de 3,85 ms/operación:

procesos	segundos	CPU	aceleración	eficiencia	horas-CPU
12	1.041,4	851 %	9,0x	**75 %**	2,5
24	759,2	1.324 %	12,3x	51 %	2,8
48	695,1	2.317 %	13,5x	28 %	4,5
72	673,7	3.293 %	13,9x	19 %	6,2
96	660,4	3.801 %	14,2x	15 %	7,0

De 24 a 96 procesos se gana un 13 % de reloj y se gastan 2,5x más horas-CPU.

 Y la causa no es Amdahl. El tramo en serie —leer el export y las barras antes de lanzar a nadie— está medido y es **0,83 s de 695** con 96 estrategias, **2,38 s** con las 499. Eso autorizaría 800x, no 14x.

 La causa es el tamaño de la tarea. Una tarea es una estrategia, y dentro del mismo lote la mayor lleva **117.612 operaciones (453 s ella sola)** y la menor **12**: un factor de 10.000. A partir de 24 procesos el reloj no lo manda el reparto, lo manda la estrategia más larga:

procesos	suelo teórico	medido	quién manda
12	781 s	1.041 s	el reparto
24	**453 s**	759 s	la más larga
96	**453 s**	660 s	la más larga

El hueco entre 453 y 660 sí es contención, pero aunque se borrara no se bajaría de 453 s. **El siguiente paso real es repartir por** `(estrategia, mercado)`: los nueve mercados son independientes dentro de `analyse_market` y eso divide la tarea más larga por ~9.

> **Recomendación:** para el paso 10, **24-32 procesos**. Dan el 87 % del reloj de 96 con una fracción de la memoria, y la memoria es lo que mató la ventana de VSCode.

Lo que se midió y se decidió NO hacer

- `_losing_run` **recorriendo filas contiguas en vez de columnas con salto**: 4,5x sobre la matriz de 25.000×600, pero **sólo 1,5x** sobre los lotes de 500 en los que ahora se trabaja, y es ~13 % del tiempo. Resultado idéntico verificado. Queda medido por si el troceado cambia de tamaño.
- `np.add.at` → `np.bincount`: 1,6x–2,5x, ya medido el 2026-09-24. No cambia el orden de magnitud.

Segunda ronda: numba y balanceo de carga — medido 2026-09-25

Misma máquina, mismos ficheros y mismas órdenes que arriba, con la base **re-medida el mismo día** desde el commit anterior (`f1bf407`) en una copia aparte del código. `PYTHONHASHSEED=0`, un hilo de BLAS. Datos crudos y scripts: `AlgoData/reports/perf-optim-2026-09-25/`.

Qué se cambió

#	cambio	dónde
1	**kernel numba** que valora cada run y calcula sus estadísticas en una pasada, sin matrices intermedias	`engines/nulls/kernel.py`
2	barrido de barreras que **se para en el primer toque**	`engines/nulls/kernel.py:touched`
3	**balanceo de carga**: lo más caro primero (LPT), un hilo de BLAS por proceso	`core/fanout.py`
4	el paso 10 reparte por **(estrategia, mercado)** y calcula solo lo que publica el `verdict.csv`	`studies/transfer/crossmarket/report.py`, `market_run.verdict_row`
5	`studies.readings.monkey.report` en paralelo y leyendo el export **una vez**, no una por estrategia	`studies/readings/monkey/report.py`
6	semilla estable por (estrategia, peldaño, bloque) y bloque medido en trades	`engines/nulls/simulate.py`
7	la cosecha exporta IS y OOS en **un** ciclo del conductor y **un** `orderstocsv`	`studies/screening/gate/collect.py`

El resultado

tarea	antes	después	factor	RAM del sistema, antes → después
`studies.readings.monkey.report`, 757 estrategias × 4 peldaños × 2.500	**569,0 s**	**11,1 s**	**51x**	1,0 → 5,3 GB
`crossmarket` 8 × 9, 7 procesos	155,7 s	**12,9 s**	**12x**	8,6 → 1,5 GB
`crossmarket` 96 × 9, 24 procesos	755,4 s	**52,2 s**	**14x**	20,8 → 7,3 GB
`crossmarket` 96 × 9, 96 procesos	674,9 s	59,4 s	11x	37,4 → 21,1 GB
`crossmarket` **499 × 9**, 72 procesos	2.285 s (38 min)	**255 s (4 min)**	**9x**	53,8 → 32,7 GB
`studies.screening.gate.report`, 500 estrategias	5,6 s	4,9 s	1,15x	5,3 → 1,1 GB
`studies.screening.gate.harvest`, 500 + 500 (export)	114,1 s	**81,5 s**	1,4x	4,4 → 4,4 GB
`export_retest`, 499 × 9 (export)	160,4 s	160,0 s	1,0x	6,0 → 5,8 GB

Cuántos procesos, ahora

procesos	96 × 9, después
12	90,8 s
24	52,2 s
48	**44,6 s**
72	51,7 s
96	59,4 s

> **Recomendación nueva: 48 procesos, uno por núcleo físico.** Por encima el hyperthreading y la caché compartida lo hacen **más lento**, no igual, y además gasta más memoria.

 2026-09-29, confirmado sobre datos reales guardados (no sintéticos): `crossmarket.report` sobre `Test_USDJPY_donchianUpperCrossUp_M30 / Retest_Markets_-_Family / 2026-09-28`, 200 estrategias reales × 9 mercados a 10.000 sorteos, 48 procesos — **135 s de reloj** (2:15), 6.121 s de CPU (45,3×), `verdict.csv` idéntico antes/después de la repetición: 200/200 `DESCARTAR`, mismas razones. Todo el lote de 200 estrategias de hoy (135 s) es más rápido que la tarea más larga sola del reparto antiguo por estrategia (453 s en el lote de 96): el reparto por (estrategia, mercado) ya está en producción — issues 69 y 70 del `OPEN.md`, cerradas.

Por qué los números son los mismos

- `crossmarket`: el `verdict.csv` sale **idéntico byte a byte** al de antes en 8 × 9 y en 96 × 9, con 12, 24, 48, 72 y 96 procesos. El 499 × 9 no se pudo comparar: su salida anterior se sobrescribió.
- Los kernels se compararon contra el código numpy con los mismos sorteos: drawdown, curvas, racha y trades **idénticos bit a bit**; net, Sharpe y PF a ≤ $5{,}5 \cdot 10^{-11}$ (numpy suma por pares, el kernel en orden).
- `nulls` y el mono de la puerta **cambian de monos**, a propósito (semilla nueva). 15.140 p de `studies.readings.monkey.report`: correlación 0,991 con los de antes, y solo el 0,23 % se aleja más de 3 errores Monte Carlo (se espera ~0,3 %). La puerta: las cinco cribas deterministas idénticas; supervivientes 229 → 228, una estrategia en el umbral.
- La cosecha: métricas y equity idénticas; operaciones idénticas en valores, con `Sample type` ahora categórica en vez de texto.

Lo que se midió y NO funcionó

- **Parsear los CSV del export con 16 hilos es más lento**: `export_retest` 179,5 s contra 160,4 s en serie, misma salida. Lo que se hace con cada fichero después de `read_csv` retiene el GIL. Se deshizo.
- En el export manda SQX: el `orderstocsv` y la carga del databank. La cosecha solo ganó lo que cuesta un arranque de JVM; `export_retest` ya tenía uno solo y no ganó nada.

Dónde está el cuello de botella ahora, y qué se estropea al doblar procesos

 Medido el 2026-09-25 cronometrando cada fase de cada tarea (estrategia, mercado) de 96 × 9, a 24, 48 y 96 procesos (`AlgoData/reports/perf-optim-2026-09-25/phases_*.parquet`):

- **No es el reparto**: los procesos están ocupados el 99 % del reloj en los tres casos, y la última tarea dura menos de 3 s. La parte en serie (leer el export y compilar) es 1,5 s.
- **Cada tarea se vuelve más lenta cuantas más corren a la vez**, y sin esperar a nada (CPU/reloj = 1,00). El mismo trabajo cuesta 1.155 s de proceso con 24, 1.940 con 48 y **5.219 con 96**. De 48 a 96 dos procesos comparten núcleo físico. De 24 a 48 ya se pierde un 68 %: se comparten L3 y memoria, y probablemente baja el turbo.
- **Lo que más se estropea de 48 a 96**: el kernel numba de los nulos, **6,2 veces** más lento, y el estrés de slippage, 5,6 veces. Son las fases que leen los arrays de precios a saltos. El resto empeora entre 2,2 y 3,4 veces.
- **Lo que más pesa, con 48 procesos**: el **test pareado** (`paired.run`), el **42 %** del tiempo de cada tarea; la exposición (bootstrap), el 15 %; la reconciliación (`backtest.setting`), el 12 %; el bootstrap de PF y esperanza, el 10 %. Los nulos enteros (sortear, kernel, diagnósticos) ya son sólo el 12 %. **El siguiente objetivo es `paired.run`**, no los nulos.

Tercera ronda: todo lo demás — medido 2026-09-25

Todos los procesos de Python del proyecto que no se habían optimizado, con la base re-medida el mismo día desde el commit anterior en una copia aparte del código. **Memoria = PSS del árbol de procesos** (`/proc/<pid>/smaps_rollup`), no RSS: con `fork`, el RSS cuenta una vez por hijo las páginas que comparten

(`knowhow/perf/`). Datos crudos, scripts y validaciones: `AlgoData/profiling/bench-2026-09-25/` .

proceso	antes	después	factor	memoria, antes → después	qué se cambió
Monte Carlo, 36 estrategias × 20.000 caminos	**477 s**	**27 s**	**18x**	1,7 → 2,7 GB	cada estrategia en su proceso, LPT; kernel numba de los estadísticos sin la matriz reunida
`studies.screening.filters.report` , 10.000 estrategias	38,7 s	**3,2 s**	12x	0,4 → 1,5 GB	los 420 filtros candidatos, repartidos
CSCV (`pbo`), 962 variantes	30,8 s	**9,5 s**	3,3x	igual	vecinos de la rejilla calculados una vez; las tres reglas a la vez
crossTF, 48 celdas	29,2 s	**5,7 s**	5,1x	igual	caché de los YAML de `assets/` (54 de 60 s eran parsearlos)
`sqx.variants.make` , 5.000 variantes	11,2 s	**2,1 s**	5,3x	igual	fabricar y releer los `.sqx` en paralelo
`studies.readings.monkey.report` (tras la ronda 2)	11,1 s	**3,6 s**	3,1x	—	la misma caché de YAML
`retest.ingest` , 5 × 8 tareas	7,9 s	**1,9 s**	4,0x	**2,9 → 1,9 GB**	cada proceso escribe su P&L; 16 procesos
`sppUltra`	7,0 s	**1,9 s**	3,7x	igual	la prueba de inertes, vectorizada
`retest.report`	5,3 s	**2,0 s**	2,6x	0,4 → 0,85 GB	una estrategia por proceso
`entryQuality`	2,5 s	2,1 s	1,2x	**1,3 → 0,47 GB**	lee sólo la columna M1 que usa
`index_sqx` , 19.328 `.sqx`	2,2 s	1,65 s	1,3x	igual	abre cada zip una vez, no dos (−28 % CPU)
los diez restantes (WFM, WFC, parameterCloud, profitShape, exposure, decay, is_oos, tasks.nulls, ledger, variants.scale)	0,4–1,9 s	igual	1x	igual	su tiempo es importar pandas y scipy (~0,7 s)

Suma de los 20: 623 s → 67 s.

Por qué los números son los mismos

- **Idénticos byte a byte o celda a celda:** los ficheros de `sppUltra` y `retest.report` (hoy `spp` y `mcRetest`), el `cscv.json` , las 48 celdas de crossTF, el `improvement.md` de `filters` (hoy `filters/filters.md`), el `nulls.csv` , el índice de `.sqx` , el manifiesto de `variants.make` y el contenido de sus 5.000 `.sqx` , la salida de `entryQuality` , y las cinco tablas de la ingesta, incluidas las **35.125.203 filas de P&L**.
- **Kernel de Monte Carlo** contra `metrics.paths` con los mismos índices, en los 5 modelos de sorteo y los 4 de estrés: ≤ $2{,}5 \cdot 10^{-15}$ relativo, drawdown y racha idénticos.
- **Monte Carlo es aleatorio sin semilla**, así que se validó contra su propio ruido: dos corridas nuevas y una base dan el **mismo tier en las 36 estrategias**, y la base se separa de la nueva lo mismo que la nueva de sí misma.

Lo que se encontró por el camino

- □ `@njit` **por defecto lanza** `ZeroDivisionError` donde numpy da `nan`. Los tres kernels llevan ya `error_model="numpy"`, también los de la ronda 2.
- `retest.ingest` no gastaba memoria en el padre, sino en los trabajadores (~85 MB cada uno). Con 40 va en 1,4 s y 3,4 GB; con 16, en 1,9 s y 1,9 GB; con 8, en 2,7 s y 1,1 GB. Se dejó en 16.
- `sqx.variants.equity` y `collect` no se pudieron medir: no hay en disco ninguna carpeta de variantes con el formato de tres tramos actual.

El paso 16.5 de punta a punta — medido 2026-09-25

Tres madres de USDJPY H1, 5.000 variantes cada una, retesteadas en 3 tramos × 10 mercados en el custodio. Tabla completa en `09-optimizacion.pdf › 19-wfc`. Lo que cambió en la cosecha, medido sobre la misma escala y validado contra la lógica original en 200 ficheros de cada tramo:

fase	antes (madre 1)	después (madres 2 y 3)
`equity`	385 s, **31,8 GB**	**83–88 s, 3,7–4,4 GB**
`collect`	196 s	**17 s**
vaciar el custodio	206 s arrancando SQX, que carga las 20.000 estrategias (JVM 47 GB) solo para borrarlas	**3–4 s** borrando con SQX parado
Python por madre	**~13 min**	**~1,8 min**

Lo que manda es SQX: 63–71 min de retest por madre, con el JVM pegado a su techo de 80 GB.

La ventana mientras corren análisis — medido 2026-09-28

Tu revisión de la ventana: al lanzar los análisis IS/OS de Python la ventana tardaba **un minuto** en cambiar de vista y el servidor estaba tranquilo. Se midió antes de tocar nada, sobre `Test_USDJPY_donchianUpperCrossUp_M30`, 200 estrategias:

qué	antes	después
abrir el proyecto (la ventana vuelve a responder)	4,4 s congelada	**0,006 s**; los datos llegan detrás
recargar el panel de databanks tras un análisis	3,4 s congelada, **una vez por cada trabajo terminado** (16 por sondeo → ~1 min)	en segundo plano, **una vez cada 20 s** como mucho mientras quedan trabajos, y otra al acabar
`profitShape` sobre las 200 estrategias	207 procesos, 16 a la vez: **~43 s**	**un** proceso con 44 trabajadores: **~9 s**
peor parón de la ventana durante esa tanda	~60 s	**0,32 s** (una vez; el percentil 99, 0,18 s)

Dónde estaba el cuello de botella. No en cargar estrategias: la tabla del databank sale del demonio en 0,1–0,3 s. Estaba en que la ventana esperaba al demonio **en el mismo hilo que pinta**, y en que cada trabajo que acababa hacía releer el panel, el embudo y el carril enteros. Y los núcleos estaban quietos porque cada estrategia era un proceso de ~1 s, del que 0,7 s es Python importando pandas y el estudio, y solo entraban 16 a la vez.

Qué cambió.

- La ventana pide todo lo que se repite (sondeos, panel, curva agregada, embudo, carril, «En marcha», el pulso, la barra de carga) en hilos aparte, y solo pinta cuando llega la respuesta. Puedes cambiar de pestaña y seguir trabajando mientras corre.
- Un análisis «sobre la población» de un estudio de una estrategia (profitShape, entryQuality, edgeCost…) es ahora **un solo trabajo**: importa el estudio una vez y reparte las estrategias entre los núcleos físicos **menos 4**, que quedan para la ventana, el demonio y el sistema. Si la RAM libre por encima de los 20 GB de Python no da para tantos, usa menos (0,6 GB por trabajador).
- Todo análisis de Python corre con prioridad baja (`nice 10`): a plena carga, la ventana y el demonio van primero.
- Los estudios que ya reparten ellos solos entre núcleos (monkey, gate, crossmarket, cscv…) siguen siendo un trabajo por estrategia: meter un reparto dentro de otro satura la máquina.

Cómo verlo y cancelarlo. La tira de trabajos de la barra de estado enseña el lote como un solo trabajo con su barra y «74/200 estrategias · 2 fallaron»; su ✕ lo cancela entero, trabajadores incluidos. Debajo de los pasos del proyecto, el carril dice cuántos siguen en marcha y su media.

El comando que corre por debajo lo lanza la ventana; a mano solo sirve para depurar:

```
python3 -m ui.daemon.runner.batchrun --plan AlgoData/logs/ui/batches/<fecha>-profitShape.json --workers 44
```

flag	obligatorio	qué hace
`--plan`	sí	el JSON que escribe el demonio: una línea `{strategy, argv}` por estrategia
`--workers`	sí	cuántos procesos; el demonio lo calcula con los núcleos y la RAM libre

Cada estrategia deja su log en `AlgoData/logs/ui/batches/<fecha>-<estudio>/NNNN.log`; el log del lote copia las últimas líneas de cada una que falla. Termina con código 0 aunque fallen algunas (lo dice su última línea); con 1 solo si fallan todas.

Cómo comprobar que la ventana ya no se congela. Abre un proyecto con un databank de 200 estrategias, lanza «▶ todas las pruebas Python pendientes» o un paso con varias pruebas y, mientras corre, cambia de zona (Databanks, Estrategia, En marcha) y vuelve: cada cambio debe ser inmediato. Mira `htop`: durante el lote verás ~44 procesos `python3` con prioridad 10.

64. El archivo de estrategias — guardar una estrategia con todos sus resultados y recuperarla sin recalcular nada

Qué pregunta responde

Una estrategia que ha llegado lejos en la secuencia ha costado muchas horas de CPU: el Monte Carlo, el MC Retest, el SPP, el cruce de timeframes... El archivo la guarda «como oro en paño»: el `.sqx` de SQX, sus operaciones, **el resultado entero de cada estudio** (no solo «pasa / no pasa») y lo que hace falta para juzgarla más adelante. Después se recupera tal cual, sin volver a correr ningún estudio, aunque su proyecto se haya retirado y sus informes hayan desaparecido.

Cuándo lo usas, y cuándo no

- **Sí:** cuando una estrategia merece guardarse, en cualquier paso del workflow. Se archiva a mano (decisión tuya, 2026-09-27): no hay archivado automático. Lo normal es hacerlo desde el botón «Archivar» de la ficha de la ventana; este capítulo explica el comando que hay debajo.
- **Sí, otra vez:** si vuelves a correr un estudio sobre ella, archívala de nuevo. Cada archivado es una **versión nueva**; la anterior no se toca.
- **No** para guardar un databank entero: se archiva **una** estrategia cada vez.
- **No** es una copia de seguridad del proyecto de SQX: eso lo hace `/retirar-proyectos`.

Antes de empezar

- La estrategia tiene que tener informes en `AlgoData/reports/<proyecto>/`. Lo normal es que al menos haya pasado la puerta IS/OOS (paso 7-8), que es quien deja la cosecha que la ficha dibuja.
- El proyecto tiene que estar en `AlgoData/projects/registry.csv` (lo apunta el constructor de proyectos): de ahí se saca el activo y el timeframe.
- **SQX no tiene que estar corriendo ese proyecto.** El `.sqx` se copia leyendo el fichero de la carpeta del databank; si SQX lo está escribiendo, el archivo se niega y te pide esperar. No se manda ningún comando a SQX: puedes tener la GUI abierta.

Cómo se ejecuta

Archivar:

```
python3 -m core.archive archive --project Test_USDJPY_donchianUpperCrossUp_M30 \
    --databank Results --identity 4d679e0c2ce2a63ee53bc6cb305830bcaf5a74997e6e6e2c1deeb4c30f307048 \
    --step 16 --family donchianUpperCrossUp --note "la mejor del lote"
```

flag	obligatorio	qué hace
`--project`	sí	el proyecto de SQX, como aparece en SQX
`--databank`	sí	el databank de construcción, el que tiene la cosecha (normalmente `Results`)
`--identity`	sí	la huella de la estrategia; la ventana la pasa sola, tú no la tienes que escribir nunca
`--step`	sí	el paso del workflow en el que estaba al archivarla (`13`, `16.5`...)
`--family`	sí	la familia de plantillas con la que se cuenta su Ledger (ver «Qué NO te dice»)
`--note`	no	tu comentario, se guarda tal cual
`--sqx`	no	el `.sqx` a mano, solo cuando ningún install guarda ya el databank (proyecto retirado). Se comprueba que es la misma estrategia; si no, se niega

Ver lo archivado:

```
python3 -m core.archive list                    # una línea por versión
python3 -m core.archive show <identidad>        # la versión más nueva
python3 -m core.archive show <identidad> --version 2026-09-27T2024
```

Tarda un par de segundos. No toca SQX ni lanza ningún estudio.

Qué produce

Una carpeta por versión: `AlgoData/archive/<identidad>/<AAAA-MM-DDTHHMM>/`. **No sobrescribe nunca**: si ya hay una versión en ese mismo minuto, se niega. Al terminar la carpeta queda **de solo lectura**; para borrarla hace falta `chmod -R u+w` antes, a propósito.

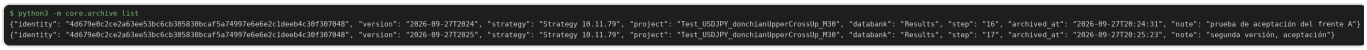
fichero	qué es
`strategy.sqx`	la estrategia, byte a byte
`reports/<databank>/<día>/<estudio>/`	cada resultado de estudio que lleva su identidad, de **todos** los databanks del proyecto: el JSON del estudio, el de su población, su manifiesto y sus filas de cada tabla (por ejemplo, sus 2.094 operaciones con spread real). Sin figuras: la ventana las redibuja
`harvest/`	sus filas de la cosecha: métricas IS/OOS, curva diaria y operaciones
`view.json`, `tearsheet/`	exactamente lo que la ventana recibe para esta estrategia en vivo: el resultado de cada estudio, la fila de la matriz, la ficha y la puerta
`asset/`	la ficha de costes del activo **tal como estaba el día de archivar**, con su huella sha256
`meta.json`	los campos legibles de la estrategia (dirección, entradas, salidas, gestión monetaria...)
`manifest.json`	todo lo anterior listado, más el paso, tu nota, el commit de git, la huella de configuración (`config_hash`) de cada estudio y **la cuenta del Ledger**

Cómo se lee el resultado

```
$ python3 -m core.archive show 4d679e0c2ce2a63ee53bc6cb305830bcaf5a74997e6e6e2c1deeb4c30f307048
versión 2026-09-27T2039 · Test_USDJPY_donchianUpperCrossUp_M30/Results · paso 17 · archivada 2026-09-27T20:39:17 · código 7d038a6-dirty
  estrategia: Strategy 10.11.79
  .sqx: ~/Desktop/AlgoData/raw/Test_USDJPY_donchianUpperCrossUp_M30/SPP_IS/2026-09-27/strategies/Strategy 10.11.79.sqx (a mano)
  activo: USDJPY sha256 79ef15423a3e
  ledger USDJPY_M30_donchianUpperCrossUp: 4 búsquedas, N=0 candidatos puntuados
  CrossTF                   2026-09-27 crossTF      hash 874339be  4 ficheros
  MCR_All                   2026-09-27 mcRetest     hash 7c87851e  4 ficheros
  OOS                       2026-09-27 curate       hash None  4 ficheros
  Results                   2026-09-27 curate       hash None  4 ficheros
  Results                   2026-09-27 gate         hash 2a2a3079  7 ficheros
  Results                   2026-09-27 isOos        hash 5c618829  2 ficheros
  Results                   2026-09-27 spread       hash 91570e0e  4 ficheros
  suelto: pre_mcr_identity.csv
  suelto: MCR_All/2026-09-27/capacity3_pre_mcr_identity_verdict.csv
  suelto: Results/2026-09-27/capacity8_verdict.csv
  no archivado: OOS/2026-09-27/isOos — sin filas por estrategia: solo resultado de población o tablas sin columna strategy
  no archivado: Retest_Markets_-_Family/2026-09-27/crossmarket — sus filas no llevan identidad: emparejar por nombre sería adivinar
  no archivado: SPP_IS/2026-09-27/spp — sus filas no llevan identidad: emparejar por nombre sería adivinar
  no archivado: workflow-steps.csv — no nombra estrategias por identidad
  cosecha: {'metrics': 1, 'equity': 4032, 'trades': 2094}
  gate: sí · metadatos: sí
```

- **Primera línea:** la versión, de qué proyecto y databank viene, en qué paso se archivó, cuándo y con qué versión del código.
- **`.sqx`:** de dónde se copió; «(a mano)» si se pasó con `--sqx`.
- **`activo`:** la huella de la ficha de costes congelada. Si mañana cambias los costes de USDJPY, la huella de hoy no coincidirá, y así sabes que la estrategia se juzgó con otros costes.
- **`ledger`:** cuántas búsquedas y cuántos candidatos llevaba el estudio (activo + timeframe + familia) al archivar. Es lo que el módulo de portfolio necesita para deflactar su Sharpe: tres supervivientes de diez mil no valen lo mismo que tres de cincuenta.
- **Una línea por estudio:** el databank, el día, el estudio, su huella de configuración y cuántos ficheros se congelaron.
- **`suelto`:** tablas que no están dentro de un estudio (los cortes de aforo, la identidad antes del MC Retest) y que nombran la estrategia: se guardan solo sus filas.
- **`no archivado`:** lo que no se ha podido emparejar con la estrategia, y por qué. Nada se queda fuera en silencio.
- **`cosecha`:** cuántas filas de métricas, días de curva y operaciones se guardaron.

`list` da una línea por versión; dos líneas con la misma identidad son dos versiones de la misma estrategia:



Un ejemplo completo

La estrategia `Strategy 10.11.79` de `Test_USDJPY_donchianUpperCrossUp_M30`, archivada el 2026-09-27. Su proyecto ya no tiene databanks en el custodio, así que sin `--sqx` el archivo se niega y explica qué hacer:

```
$ python3 -m core.archive archive --project Test_USDJPY_donchianUpperCrossUp_M30 \
    --databank Results --identity 4d679e0c2ce2a63ee53bc6cb305830bcaf5a74997e6e6e2c1deeb4c30f307048 \
    --step 16 --family donchianUpperCrossUp
FileNotFoundError: Ningún install guarda esa estrategia en Test_USDJPY_donchianUpperCrossUp_M30/Results: pasa el .sqx a mano con --sqx (se comprueba su identidad)
```

Con el `.sqx` que guardó la exportación del SPP (`--sqx ~/Desktop/AlgoData/raw/Test_USDJPY_donchianUpperCrossUp_M30/SPP_IS/2026-09-27/strategies/Strategy 10.11.79.sqx`) se archivó tres veces (pasos 16, 17 y 17), y quedaron tres versiones. La prueba de aceptación se hizo **sobre una copia** de los datos, no sobre tu `reports/` real: otro trabajo estaba escribiendo en ese proyecto en ese

momento y apartar sus carpetas lo habría roto. En la copia se archivó, se apartaron las carpetas de `reports/`, de `harvest/` y el `.sqx` original, y se leyó el archivo: los datos salieron **idénticos** a los que la ventana recibía antes, sin cargar ningún estudio ni lanzar ningún proceso. Luego se devolvieron las carpetas a su sitio y la copia se borró.

Qué NO te dice

- **Un estudio que no guarda la identidad no se archiva, y se dice.** El cruce de mercados de este proyecto escribió su `verdict.csv` sin identidades, y su databank tenía dos estrategias con casi el mismo nombre (`Strategy 10.11.79` y `Strategy 10.11.79(1)`). Emparejar por nombre sería adivinar, así que no se hace: `show` lo lista como «no archivado» con el motivo. Lo mismo el SPP que se volvió a correr a las 20:33 (firmó sin identidad), el `isOos` de OOS (solo tiene el resultado de población) y cualquier estudio que nombre las estrategias solo por nombre.
- **«Obsoleto» es a fecha de archivar.** Si luego cambias la configuración de un estudio, el archivo no lo sabe: para saberlo habría que cargar el estudio, y el archivo nunca lo hace.
- **La cuenta del Ledger es la de ese día.** Si el fichero del Ledger del estudio no existe (este ejemplo: el de USDJPY M30 donchianUpperCrossUp se movió a `AlgoData/ledger/archive/`), cuenta 0.
- **`--family` lo eliges tú** mientras no decidas cómo se nombra la familia del Ledger (pregunta Q9 del plan 24). Una familia mal escrita da un estudio vacío: 0 búsquedas.
- **La ficha de costes es la de hoy, no necesariamente la de cada backtest:** `assets/` no guarda historia. La huella sirve para saber si ha cambiado desde el archivado.

Reloj y comisión de un miembro de cartera

Al archivar, el manifiesto guarda `clock` (la zona horaria del feed) y `commission` (la comisión de la ficha, por tramo). La cartera los lee de ahí, nunca en vivo, así que cambiar la ficha o el registro de SQX no rompe a los miembros ya archivados. Los archivados antes de este cambio se rellenan con `python3 -m portfolio.common.construct.inputs.backfill`: sin `--yes` solo muestra la tabla (qué es rellenable y qué no, con el motivo) y no escribe nada; con `--yes` escribe. La comisión sale de la propia ficha congelada del miembro; el reloj solo se rellena si su proyecto se creó el 2026-10-01 o después. Antes el feed tenía otro reloj que ningún fichero guarda, con una excepción por orden del dueño (2026-10-13): los miembros del feed antiguo `USDJPY_DukasM1_the5ers` reciben el reloj del feed USDJPY actual (hora de FTMO). Los demás miembros anteriores se listan y no se tocan: los 5 USDJPY de finales de septiembre (comisión de la ficha sin tramos, un 8.0 plano) siguen sin rellenar y fallan al leerse, que es lo correcto. Aviso: en 3 de los 30 miembros de octubre la reconciliación con la curva de SQX cuadra con Asia/Jerusalem (error máximo 0,3-3 $) y no con la hora de FTMO (430-2.000 $); pendiente de decisión.

Si algo falla

- `KeyError ... no guarda reloj ni comisión`: el miembro es anterior al cambio; pasa el backfill.
- `FileNotFoundError: No hay ninguna versión archivada`: esa identidad no se ha archivado nunca.
- `FileExistsError ... ya existe`: ya archivaste esa estrategia en este mismo minuto. Espera al minuto siguiente; la versión que existe no se toca.
- `FileNotFoundError: Ningún install guarda esa estrategia`: el proyecto ya no tiene ese databank en disco. Pasa el `.sqx` con `--sqx`; suele estar en `AlgoData/raw/<proyecto>/*/…/strategies/`.
- `ValueError ... es otra estrategia`: el `.sqx` que pasaste no es esa estrategia.
- `RuntimeError: SQX está escribiendo`: el proyecto está corriendo; archiva cuando termine.
- `KeyError ... no está en registry.csv`: el proyecto no lo creó el constructor, y no se sabe su activo ni su timeframe.

109. El almacén de estrategias — guardar lo que pasa la puerta y comprobar que está entero

Qué pregunta responde

Una estrategia que sale de un build genético no se puede volver a generar: si se pierde su archivo `.sqx`, se pierde la estrategia. El almacén guarda, para cada estrategia que pasa el paso 8 (limpia o en limbo), su `.sqx` y un archivo por cada juego de tests que se le haya hecho, y permite comprobar en cualquier momento que nada se ha perdido ni dañado.

Cuándo lo usas, y cuándo no

- Después de un run del workflow, para guardar sus supervivientes antes de retirar el proyecto. `retire` se niega a borrar un proyecto con supervivientes sin guardar y te dice este comando.
- Cuando quieras comprobar que lo guardado sigue entero.

No guarda las estrategias que mueren en el paso 8. No sustituye al archivo antiguo (capítulo 64): lo que se archivó allí a mano sigue allí.

Antes de empezar

El proyecto tiene que haber pasado el paso 8. Los `.sqx` tienen que seguir en algún sitio: en los databanks del proyecto en el worker, o en una carpeta que le indiques. No toca SQX.

Cómo se ejecuta

```
python3 -m core.store save Test_USDJPY_M30_doubleROCLong_10k
python3 -m core.store verify
python3 -m core.store audit Test_USDJPY_M30_doubleROCLong_10k
python3 -m core.store list
python3 -m core.store open 03fdea12773d --out /tmp/una
```

orden	qué hace
`save <proyecto>`	guarda las supervivientes del paso 8 de ese proyecto. Con `--sqx-dir <carpeta>` busca los `.sqx` ahí en vez de en los workers
`verify`	comprueba todo lo guardado; con una o más huellas detrás, solo esas
`audit <proyecto>`	comprueba que no se queda nada fuera: `FALTA` (código 1) por cada cosa que el proyecto dejó en disco y ningún zip guarda; `ajeno` por lo que no pertenece a ninguna estrategia guardada
`list`	una línea por estrategia guardada
`open <huella o nombre> --out <carpeta>`	desempaqueta una estrategia: su `.sqx` con su nombre y cada test en su carpeta; si varias coinciden, las lista y no elige

Guardar dos veces no duplica nada: solo añade lo nuevo. Guardar 41 estrategias de un lote de 10.000 tarda algo más de un minuto. `save` acaba con código 1 si alguna superviviente no tiene `.sqx` en ningún sitio o si algo guardado sale dañado.

Qué produce

```
AlgoData/store/USDJPY_M30_doubleROCLong/        la celda: activo, timeframe y plantilla
  runs.json                                     cada run de la celda: generadas, supervivientes,
guardadas
  .pool/                                        los bytes de cada zip, una sola vez
  <nombre de la estrategia> <8 primeras letras de su huella>/   p. ej. «Strategy 12.13.265 4cf97a7d»
    strategy.sqx                                la estrategia, una sola vez
    manifest.json                               de dónde salió, qué archivos hay y qué dijo cada paso
    <estudio>.<databank>.<día>.zip              sus filas de ese estudio y sus distribuciones
    <estudio>.<databank>.<día>.poblacion.zip    lo que es igual para todas las del run (compartido)
    operaciones.<databank>.<día>.zip            sus operaciones y las de sus hermanas en ese databank
    hermanas.<día>.zip                          las hermanas del paso 12: parámetros, sin .sqx
    exportacion.<databank>.<día>.zip            carpetas de una exportación: simulaciones MC, SPP,
celdas WFM
    variantes.<día>.zip                         el lote de variantes del paso 16.5
    optimizacion.<día>.zip                      los estudios hechos sobre ese lote
    metricas.zip, cosecha.….zip                 sus métricas, su curva y sus operaciones
    costes.USDJPY.zip                           la ficha de costes del día en que se guardó
```

Cada carpeta contiene todos sus archivos, pero los bytes iguales entre estrategias están una sola vez en `.pool` y las carpetas los enlazan: copiar una carpeta se lo lleva todo, y el disco no paga dos veces lo que comparten 41 estrategias. Los informes del WFM y de blindJoint se archivan bajo la estrategia madre (el WFM le da otra huella a la estrategia, y `lineage.csv` dice de quién es cada una). Cada archivo queda de solo lectura. Si un test se repite con otros resultados no pisa al anterior: se guarda al lado como `….2.zip`.

Distribuciones. Cuando un estudio saca una z o una p de simulaciones, guarda en `distribuciones.parquet` lo que salió: el valor observado, un histograma de 200 barras sobre todo el rango, los 100 valores más extremos de cada lado y, salvo que sean miles, los sorteos enteros. Sirve para ver de dónde sale una p sin repetir el test.

Hermanas. Las estrategias escaladas del paso 12 (la misma en M15, H1, H4…) no guardan su `.sqx`, solo sus parámetros. Para volver a tener el archivo: `python3 -m sqx.variants.rebuild` (ver `/rebuild-sister`).

Tamaño. Unos 4 MB por estrategia hasta el paso 12; una que llega al 16.5 añade unos 80 MB más, casi todo del lote de variantes.

Reanudar una estrategia guardada

Para seguir desde el paso 13 un run cortado o un proyecto ya retirado:

```
python3 -m pipeline.autopilot.resume Test_USDJPY_M30_doubleROCLong_10k 03fdea12773d
bin/workflow.sh run Test_USDJPY_M30_doubleROCLong_10k
```

`resume` vuelve a crear el proyecto con su mismo nombre y carga en sus databanks los `.sqx` guardados; después el workflow continúa en el paso 13. Un proyecto construido con `--until` no tiene tareas posteriores, así que no se puede restaurar así desde su `project.cfx` archivado.

Ejemplo de lista

```
$ python3 -m core.store list | head -8
03fdea12773d  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 8.10.434  37 archivos  pa…
1f2d8a9fc85d  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 19.8.312  37 archivos  pa…
2631ca905a83  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 12.21.437  37 archivos  p…
2f33bf21bf8c  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 12.19.288  37 archivos  p…
311efed279dc  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 12.18.376  37 archivos  p…
3f7ed23978df  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 8.5.311  37 archivos  pas…
4040ceb9bf3a  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 2.25.295  37 archivos  pa…
40f12988eea9  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 19.7.431  37 archivos  pa…
```

Cómo se lee el resultado

```
$ python3 -m core.store verify | tail -5
ok  ccc98e4155e4  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 19.7.389  37 archivos…
ok  d628c58de74e  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 14.11.270  37 archivo…
ok  df7346eac9e4  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 2.22.293  37 archivos…
ok  f0e5d3330802  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 19.10.469  37 archivo…
ok  f6170e270326  USDJPY_M30_doubleROCLong  Test_USDJPY_M30_doubleROCLong_10k  Strategy 12.18.253  37 archivo…
```

- **`41 guardadas, 15 sin .sqx en ningún sitio`** (al guardar): de las 56 supervivientes del lote, 15 ya no tenían archivo en ninguna parte y no se han podido guardar. Se nombran una a una; nunca se callan.
- **`ok`** delante de una línea de `verify`: su `.sqx` tiene la huella que apunta su manifiesto (la carpeta lleva el nombre y sus 8 primeras letras) y cada archivo tiene la huella que anotó el manifiesto.
- **`MAL`**: detrás dice qué falla — falta un archivo, un archivo no tiene su huella, hay un archivo que el manifiesto no nombra, o el `.sqx` no se puede leer.
- **`paso 12: MANTENER (limbo)`**: el último paso del workflow que la juzgó y lo que dijo.

Si una estrategia falla un paso posterior **no se borra**: queda anotado en su manifiesto. Borrar es decisión tuya.

Un ejemplo completo

El de la captura: el lote 1 de doubleROCLong · USDJPY · M30. Se guardan sus 41 supervivientes con `.sqx`, cada una con 37 archivos (la puerta, otros mercados, otros timeframes, hermanas, exportaciones y el resto de estudios que la nombran). `verify` da `ok` en las 41.

Qué NO te dice

- **No demuestra que la estrategia se reproduce en SQX.** Comprueba que el archivo guardado es el mismo que se juzgó, no que al cargarlo y volver a probarlo salgan los mismos números. Esa prueba está pendiente.
- **No guarda sola todavía.** El workflow aún no llama al almacén: hay que lanzar `save` a mano antes de retirar un proyecto (y `retire` lo exige).
- **No guarda todavía los pasos 21 en adelante** ni tiene zona en la ventana.
- **La versión de criterios del manifiesto es la del día en que se guardó**, no necesariamente la del día en que se juzgó.

Si algo falla

- `KeyError: … no está en …registry.csv`: el proyecto no está en el registro de proyectos; sin su fila no se sabe su activo ni su plantilla.
- `su identidad … no es la que juzgó el paso 8`: el `.sqx` encontrado con ese nombre es otra estrategia. No se guarda; busca el archivo correcto con `--sqx-dir`.

73. Catálogo de fondeo — qué vende cada empresa, a qué precio y con qué reglas

Qué pregunta responde

¿Cuánto cuesta exactamente una cuenta de fondeo con los add-ons que yo elija, y qué reglas me deja (objetivo, pérdida diaria, pérdida máxima, reparto, días mínimos, noticias, fin de semana)? Y, de paso: ¿ha cambiado algo desde la semana pasada? Hoy cubre **Hantec Trader** y **FTMO**. Todo queda en una base de datos con historial, de donde leerá el cálculo de la esperanza del fondeo (encargo 33).

Qué empresas. Las del registro `AlgoData/funding/firms.yaml`. Cada una tiene un estado: **activa** (la has admitido: entra en tu universo de compra, en los avisos de ofertas y en los estudios) o **candidata** (se sigue su catálogo y sus ofertas mientras se comprueba si sirve, pero no entra en nada más). Hoy: Hantec y FTMO activas; FundedNext y FundingPips candidatas.

Añadir una empresa es un protocolo, la skill `/firm-onboard`: dile a Claude «añade la empresa X». La registra como candidata y comprueba seis puertas en las páginas de la propia empresa: que permita EAs propios en MT5 en tus tamaños (y cuánto cobra por ello), MT5, que acepte residentes en España, su historial de pagos, que sus reglas se puedan modelar, y las trampas para EAs (operaciones idénticas entre cuentas, riesgo máximo abierto, noticias, fin de semana). Monta su catálogo — leído de su web si la web lo permite; si la web bloquea la lectura automática, tecleado y marcado sin confirmar para que lo compruebes tú, **nunca forzando esa protección** —, sus reglas y sus ofertas, y te deja una hoja de admisión en `docs/AgentPDFs/fondeo-admision-<empresa>-<fecha>.md` que termina en «¿La doy de alta como activa?». **Sólo tú la pasas a activa.**

Lo normal: no haces nada

Cada domingo a las 06:30 pasa solo el agente `fundingWatcher` (`bin/weekly-funding-refresh.sh`, en el crontab). Hace dos cosas:

1. **Vuelve a leer las webs** y actualiza precios, planes y add-ons. Esto lo hace el código, sin interpretar nada: Hantec publica su catálogo entero como JSON; FTMO lo lleva dentro de su portada.
2. **Investiga las reglas que las webs no traen en el catálogo** (si se permiten EAs, cómo se mide la pérdida diaria, si devuelven la cuota, el objetivo de la fase 2…), empezando por las que siguen desconocidas, sin confirmar, en conflicto o con más de cuatro semanas. Sólo marca una regla como **confirmada** si lo dice la propia empresa en su web; si dos fuentes discrepan la deja en **conflicto** y no elige.

Te deja el resumen en `AlgoData/audit/AAAA-MM-DD-fondeo.md`: precios que se movieron, reglas que cambiaron, lo que sigue pendiente y cualquier fallo. El log está en `AlgoData/logs/weekly-funding-refresh.log`. Sólo lee páginas públicas: nunca inicia sesión, ni compra, ni rellena formularios.

Cuándo lo usas, y cuándo no

Úsalo para consultar el precio y las reglas de una cuenta antes de comprarla, o para comparar planes. **No** te dice si una cuenta te conviene: eso depende de tus EAs y es el trabajo del encargo 33. Y los precios son **de lista**: los descuentos se ignoran a propósito (tu decisión, 2026-09-29).

Antes de empezar

Nada. No toca SQX ni MetaTrader. Necesita internet sólo para refrescar; para consultar, no.

Cómo se ejecuta

Consultar un plan con sus add-ons:

```
python3 -m portfolio.funded.catalog.show hantec:express:25000:USD MAX_DRAWDOWN PROFIT_TARGET
```

Ver todos los planes de una empresa, con sus add-ons y lo que cuesta cada uno:

```
python3 -m portfolio.funded.catalog.show hantec
python3 -m portfolio.funded.catalog.show ftmo
```

Refrescar a mano (lo mismo que hace el domingo la primera mitad del agente):

```
python3 -m portfolio.funded.catalog.refresh
```

argumento	obligatorio	qué hace
`hantec` / `ftmo`	uno de los dos, o un plan	lista los planes de esa empresa
`firma:familia:tamaño:divisa`	—	un plan concreto, p. ej. `ftmo:2step:100000:USD`
claves de add-on	no	los add-ons que sumas: `MAX_DRAWDOWN`, `PROFIT_TARGET`, `PROFIT_SHARE_FLEXIBLE` … (salen en la lista de la empresa)

Familias: Hantec `express` (1 paso), `enhanced` y `enhancedx` (2), `endurance` (3), `instant-funding`, `instant-lite`, `instant24`; FTMO `1step` y `2step`. Tarda un segundo.

Qué produce

- `AlgoData/funding/funding.sqlite` — la base de datos. Las tablas están explicadas en `portfolio/funded/catalog/README.md`. Cada cambio cierra la fila vieja y abre otra: cualquier precio pasado sigue consultable.
- `AlgoData/funding/catalogs/<empresa>/<fecha>.json` — el catálogo tal cual lo sirvió la web, sólo cuando cambió.
- `AlgoData/funding/csv/` — **lo que abres en Excel o LibreOffice**: `combos.csv` (cada cuenta con cada combinación de add-ons, su precio y sus reglas; filtra por `family`, `size` y `options`), `plans.csv`, `rules.csv` (el compendio con fuente y estado) y `changes.csv` (qué cambió y cuándo, lo más reciente arriba). Se reescriben en cada refresco, también el del domingo: no los edites, lo que cambies ahí se pierde.
- `AlgoData/funding/rules/<empresa>.yaml` — el compendio de reglas, con la fuente y el estado de cada una. Lo edita el agente; también puedes corregirlo tú y relanzar el refresh.

Cómo se lee el resultado

La primera tabla es la cuenta con tus add-ons: `price` (lo que pagas), `p1_target_pct` (objetivo de la fase 1), `daily_loss_pct`, `max_loss_pct` y su tipo (`static`, `trailing` o `eod_trailing`, que es el de fin de día de FTMO 1-Step), `reward_share_pct` (tu parte de los cobros), `payout_days` (cada cuántos días cobras), `news_ok` y `weekend_ok` (1 = permitido), `eas_allowed`.

Dos convenciones: **0 significa que la regla no existe; vacío significa que todavía no se sabe.** En `filled_from`, un valor puesto por el compendio lleva el nombre de la regla, y un `?` al final si no está confirmada por la propia empresa. La tabla final, `rules`, es el compendio: cada regla con su `status` (`confirmed`, `unconfirmed`, `conflict`).

Un ejemplo completo

Una 1-step de 25k de Hantec con pérdida máxima +2 % y objetivo −2 %:

```
$ python3 -m portfolio.funded.catalog.show hantec:express:25000:USD MAX_DRAWDOWN PROFIT_TARGET
hantec:express:25000:USD  add-ons: MAX_DRAWDOWN,PROFIT_TARGET  →  308.45 USD

... p1_target_pct  daily_loss_pct  max_loss_pct  max_loss_mode  min_days  reward_share_pct  payout_days
news_ok  weekend_ok
... 8.0            5.0             8.0           trailing       0         80.0              14
0        1

rules
rule_key           value                                                          status
daily_loss_basis   max(balance, equity) at previous 00:00 server time             unconfirmed
fee_refund                                                                        unconfirmed
eas_allowed        True                                                           confirmed
min_days           0                                                              confirmed
news_window        none in challenge; funded: no open/close 3 min around news     confirmed
reward_share_base  80                                                             confirmed
trailing_lock      locks at starting balance once profit equals the max loss      confirmed
weekend_holding    True                                                           confirmed
```

Se lee: pagas 308,45 $; necesitas +8 % (2.000 $) sin pasar de −5 % en un día ni de −8 % trailing, que se queda fijo en el balance inicial al llegar al beneficio de la pérdida máxima (+6 % sin add-on; con él, previsiblemente +8 %, sin confirmar); sin días mínimos; cobrarías el 80 %, cada 14 días. En la fondeada no puedes abrir ni cerrar 3 minutos antes y después de una noticia fuerte (`news_ok` 0) salvo que añadas `NEWS_TRADING`, que la sube a 358,20 $. Con `PROFIT_SHARE_FLEXIBLE` sube a 368,15 $ y el reparto al 95 %. Lo que queda en `unconfirmed` (si devuelven la cuota) hay que mirarlo antes de comprar.

Qué NO te dice

- Si la cuenta te sale rentable. El precio y las reglas son la mitad; la otra mitad es cómo se comportan tus EAs contra esas reglas (encargo 33).
- Descuentos: no los guarda.
- Las reglas en `unconfirmed` o `conflict` son la mejor información disponible, no la verdad. Antes de comprar, mira la fuente que da cada una.
- La cuenta Swing de FTMO aparece como opción, pero su precio no está publicado en la tabla, así que no entra en las combinaciones hasta que se sepa.

Si algo falla

- **El refresh se para con un error al leer una empresa**: la web cambió de forma. Los precios de esa empresa se quedan como estaban; el informe del domingo trae el diagnóstico y el arreglo que hace falta, para que una sesión lo aplique.
- **`UNMODELLED add-on X`**: la empresa ha sacado un add-on nuevo. Ya tiene precio y combinaciones, pero su efecto sobre las reglas aún no está escrito en `combos.py`; el informe dice cuál sería.
- **`IndexError` en `show`**: la combinación de add-ons no existe para ese plan (no todos los planes ofrecen los mismos). Mira la lista con `show hantec`.

74. Ofertas de fondeo — el aviso diario y «¿merece la pena?»

Qué pregunta responde

¿Ha salido hoy un descuento en alguna cuenta que yo compraría? Y si ha salido: ¿cuánto cambia lo que me cuesta de verdad una cuenta aprobada? Cubre Hantec Trader y FTMO, y sólo avisa de lo que toca tu universo: **cuentas de 10k como máximo, en USD, donde se permiten EAs**.

Lo normal: no haces nada

Cada día a las 10:00 (`bin/daily-funding-deals.sh` , en el crontab) pasan tres cosas:

1. **El escaneo** mira los sitios donde cada empresa enseña sus ofertas: la tabla de precios de FTMO (sus precios rebajados), el banner de la página de compra de Hantec, y vuelve a comprobar cada código de Hantec que ya conocemos. Lo que ya no está se marca como caducado.
2. **El agente** `dealHunter` busca en internet códigos que ninguna página de la empresa enseña (agregadores, cupones, redes). Un código de Hantec **sólo se guarda si la propia Hantec lo acepta**: se le pregunta con la misma comprobación que hace su web cuando escribes un código, sin comprar nada. FTMO no tiene esa comprobación pública: un código suyo sólo se guarda si lo dice la propia FTMO o dos fuentes independientes coinciden, y el aviso dice «sin verificar».
3. **El aviso**: una notificación en el escritorio por cada oferta nueva que **mejora el precio actual** de alguna cuenta de tu universo. Un −20 % no te avisa si ya hay un −50 % vigente. Si una oferta mejora (más descuento, o vuelve a estar activa), te avisa otra vez.

El aviso dice la empresa, el código y el descuento, las cuentas de 10k primero con su precio antes y después, y la línea que tienes que copiar para saber si merece la pena. El log está en `AlgoData/logs/daily-funding-deals.log` , con el resumen del agente: qué códigos probó, cuáles aceptó la empresa, cuáles rechazó y dónde buscó.

Cuándo lo usas, y cuándo no

Cuando te salte un aviso, o cuando quieras ver qué hay vigente. Dice dos cosas. Siempre, cuánto mejora el precio de cada cuenta aprobada si no tuvieras edge (el suelo). Y, **si ya has simulado la caja de tu cartera** (capítulo 88), cuánto sube el valor esperado de tu banco en cada plan con el precio rebajado. Si no hay ninguna simulación de caja lo dice en una línea y te quedas con el suelo: la mitad de la respuesta.

Antes de empezar

Nada. Para que el aviso salga, la sesión del escritorio tiene que estar abierta a las 10:00; si no, la oferta queda guardada y la ves con `worth` .

Cómo se ejecuta

```
python3 -m portfolio.funded.deals.worth 16      # el número que trae el aviso
python3 -m portfolio.funded.deals.worth         # el tablero, sin destacar ninguna oferta
python3 -m portfolio.funded.deals.add hantec CODIGO --source https://donde-lo-viste
```

comando	qué hace
`worth [número]`	el tablero de tu universo; con el número de una oferta, cuánto mueve cada cuenta
`add hantec CODIGO --source URL`	un código que has visto tú: Hantec lo comprueba plan a plan y sólo se guarda si lo acepta
`add ftmo CODIGO --pct N --terms "…" --source URL`	un código de FTMO: se guarda sin verificar
`python3 -m portfolio.funded.deals.scan`	la etapa 1 a mano: mira las webs y vuelve a comprobar los códigos conocidos
`python3 -m portfolio.funded.deals.notify`	la etapa 3 a mano: avisa de lo nuevo que mejora un precio

Tarda unos segundos (`add` de Hantec, medio minuto: pregunta por cada plan).

Qué produce

La tabla `deals` de `AlgoData/funding/funding.sqlite`: una fila por oferta y cuenta, con precio de lista, precio con la oferta, porcentaje, condiciones, de dónde salió, estado (`valid`, `unverified`, `expired`), cuándo se vio por primera y última vez y cuándo se te avisó.

Cómo se lee el resultado

`worth` ordena tus cuentas por **$/k**: las cuotas que esperas pagar por cada 1.000 $ de cuenta aprobada **si tu estrategia no tuviera ningún edge**. Sale de la geometría de las reglas: sin edge, la probabilidad de tocar el objetivo +a antes que la pérdida −b es b/(a+b) en cada fase. Se divide por el tamaño porque una cuenta de 10k cobra el doble que una de 5k, y se descuenta la cuota cuando la empresa la devuelve con el primer cobro. **Cuanto más bajo, mejor.** `puesto 3 → 1` es el sitio de esa cuenta a precio de lista y con las ofertas de hoy. Una `≤` delante de la probabilidad avisa de que el trailing la hace aún peor. Los euros de FTMO se pasan a dólares al cambio del BCE del día.

Un ejemplo completo

El aviso de DROP50 en Hantec (−50 %), y su comprobación:

```
$ python3 -m portfolio.funded.deals.worth 18
plan                          lista $    hoy $  P suelo  $/k lista  $/k hoy   puesto  reembolso
hantec:express:10000:USD        99.00    49.50     ≤38%       26.4     13.2   3 → 1   ?
hantec:enhancedx:10000:USD      99.00    49.50      33%       29.7     14.8   4 → 2   ?
hantec:enhanced:10000:USD       99.00    49.50      33%       29.7     14.8   5 → 3   ?
...
ftmo:1step:10000:USD            89.89    89.89     ≤50%       18.0     18.0   1 → 9   no
ftmo:2step:10000:USD           101.26   101.26      33%       20.3     20.3   2 → 10  yes
```

Se lee: a precio de lista la cuenta aprobada más barata por cada 1.000 $ era la FTMO 1-step (18 $); con DROP50 las Hantec de 10k pasan por delante (13,2 $ la Express). Es la mitad de la respuesta: la Express tiene drawdown trailing (el `≤`) y el reembolso de Hantec está sin confirmar (`?`), mientras que la FTMO 2-step devuelve la cuota. Con tus estrategias el orden puede cambiar: eso lo dice el bloque de abajo.

Debajo del suelo, tu cartera. Si existe una tabla de caja (capítulo 88), `worth` añade un bloque con el VE_banco de cada plan a precio de lista y con la oferta:

```
Con tu cartera — VE_banco de la última tabla de caja de cada plan, con h = 50% (los planes que no salen aquí no tienen tabla: para ellos vale sólo el suelo de arriba):
plan                        recompra   VE lista ± e.t. $  VE hoy $  Δ oferta  ¿positivo hoy?
hantec:express:2000:USD     none            -33.4 ± 0.2     -13.9     +19.5  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:express:2000:USD     same_plan       -73.3 ± 0.4     -34.7     +38.6  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:express:5000:USD     none            -45.0 ± 0.5     -15.5     +29.5  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:express:5000:USD     same_plan      -107.0 ± 0.6     -48.6     +58.4  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:express:10000:USD    none            -71.0 ± 0.9     -21.5     +49.5  no (AJUSTADO: no es un resultado)
hantec:express:10000:USD    same_plan      -176.4 ± 1.2     -78.3     +98.0  no (AJUSTADO: no es un resultado)
hantec:enhancedx:5000:USD   none            -56.8 ± 0.4     -27.3     +29.5  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:enhancedx:5000:USD   same_plan       -70.5 ± 0.3     -35.1     +35.4  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:enhancedx:10000:USD  none            -94.6 ± 0.8     -45.1     +49.5  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:enhancedx:10000:USD  same_plan      -118.3 ± 0.4     -58.9     +59.4  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:enhanced:5000:USD    none             -6.4 ± 2.0     +23.0     +29.5  sí — LA OFERTA LO CAMBIA (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:enhanced:5000:USD    same_plan       -57.7 ± 0.8     -23.1     +34.6  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:enhanced:10000:USD   none             +6.1 ± 3.9     +55.6     +49.5  sí (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:enhanced:10000:USD   same_plan       -82.6 ± 3.7      +9.7     +92.3  sí — LA OFERTA LO CAMBIA (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:endurance:5000:USD   none            -18.2 ± 0.8      -3.7     +14.5  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:endurance:5000:USD   same_plan       -34.6 ± 0.3     -16.4     +18.1  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:endurance:10000:USD  none            -37.4 ± 1.5      -7.9     +29.5  no (cartera de otro plan, AJUSTADO: no es un resultado)
hantec:endurance:10000:USD  same_plan       -70.4 ± 0.5     -33.5     +36.9  no (cartera de otro plan, AJUSTADO: no es un resultado)
Tabla(s): 20261005_113333_2026-10-05T1000_Test_nikkei_ref_hantec_express_10000_USD_s2. Δ = ahorro por compra no reembolsada; exacto con «none», con «same_plan» a compras cons…
```

(Salida real del 2026-10-05 sobre la **raíz de prueba**: cartera del pool de prueba del Nikkei, umbrales relajados y decisiones tuyas inventadas para probar. **No es un resultado.**)

- **VE lista ± e.t.**: el valor esperado de tu banco a precio de lista con el recorte que decide (h = 50 %), y su error típico de simulación.
- **VE hoy** y **Δ oferta**: lo mismo con el precio de hoy. La oferta te ahorra la rebaja en cada compra que no te reembolsan: `Δ = (lista − hoy) × (compras − reembolsos)`. Con recompra «none» (una sola cuenta) es exacto; con «same_plan» supone que compras las mismas veces, y en realidad con cuotas más baratas la caja da para alguna más — es una aproximación.
- **¿positivo hoy?**: «sí — LA OFERTA LO CAMBIA» es el caso que importa: un plan que a precio de lista pierde y con la oferta gana. Entre paréntesis, los avisos: «cartera de otro plan» (la tabla se simuló con la cartera elegida para otra cuenta: orienta, no decide) y «AJUSTADO».

Si no hay ninguna tabla de caja, en su lugar sale:

> `Sin tabla de caja de ningún plan (python3 -m portfolio.funded.sim.run no ha corrido sobre tu cartera): el juicio de arriba es SÓLO el suelo sin edge, no el valor esperado con tus estrategias.`

Qué NO te dice

- Si te conviene con **tus** estrategias **mientras no hayas simulado la caja** (capítulo 88). El suelo es el coste sin edge; un edge real lo baja, y no igual en todos los planes (las reglas de consistencia, noticias o fin de semana pesan distinto).
- Qué comprar: eso es `python3 -m portfolio.funded.decide.run` (capítulo 102), que decide a precio de lista. `worth` sólo dice cuánto mueve una oferta cada plan.
- Que un código de FTMO funcione: sin verificar quiere decir exactamente eso.
- Ofertas en redes sociales sin código: el agente las menciona en el log, pero no avisa.

Si algo falla

- **No salta el aviso a las 10:00**: mira el log. Si la sesión del escritorio no estaba abierta, la oferta está guardada; `worth` la enseña.
- **`add` dice que no es válido**: Hantec lo ha rechazado hoy (caducado, o sólo para otros planes que ya no existen). No se guarda.
- **`database is locked`**: coincidió con el refresco del catálogo del domingo; espera un minuto.